

UNIVERSIDAD DE SALAMANCA

GRADO EN INGENIERÍA INFORMÁTICA

Anexo III: Análisis y diseño del sistema

Sistema de Gestión Documental mediante Blockchain



David Pérez Velasco

Tutores:

Gabriel Villarrubia González

Sergio García González

Julio 2026

Trabajo de Fin de Grado

Índice

1. Introducción	9
2. Ámbito del software	9
2.1. Motivación y Problemática	9
2.2. Subsistemas Principales	10
2.2.1. Subsistema de Usuario Final (Frontend Web)	10
2.2.2. Subsistema de Gestión Backend	10
2.2.3. Subsistema de Almacenamiento y Persistencia	11
2.2.4. Subsistema Blockchain	11
2.2.5. Subsistema de Seguridad y Cifrado	12
3. Diseño de datos	12
3.1. Objetos de datos y estructuras de datos	12
3.1.1. Diagrama de clases de análisis	12
3.2. Estructuras de archivo y bases de datos	13
3.3. Diagrama Entidad-Relación	13
3.4. Diagrama conjunto: base de datos y blockchain	18
3.5. Persistencia coordinada: PostgreSQL, IPFS y blockchain	19
3.5.1. Base de datos relacional (PostgreSQL)	20
3.5.2. Almacenamiento distribuido (IPFS configurable)	24
3.5.3. Blockchain (Smart Contract en Ethereum)	25
3.5.4. Estructuras de datos del smart contract	26
3.5.5. Flujo de persistencia coordinado	28
4. Diseño arquitectónico	28
4.1. Arquitectura de capas	28
4.1.1. Capas del Sistema	28
4.1.2. Diagrama de paquetes	30
4.2. Patrones de diseño aplicados	30
4.2.1. Prepare/Confirm Pattern (Transacciones Blockchain)	30
4.2.2. Repository Pattern	31
4.2.3. Service Layer Pattern	31
4.2.4. Event-Driven Architecture	32
4.2.5. Salvaguardas de ejecución del contrato	32
4.2.6. Strategy (Multi-Proveedor de Wallet)	32
4.3. Diagrama de clases de diseño	33
4.3.1. Arquitectura del sistema por componentes	35
5. Diseño de la interfaz	37
5.1. Representación de las interfaces del sistema	37
5.1.1. Componentes principales	37
5.2. Prototipos de la interfaz de los componentes del sistema	38
5.2.1. Vista de inicio de sesión	38
5.2.2. Vista de registro	39
5.2.3. Vista principal de documentos	40
5.2.4. Vista de detalle de documento	41
5.2.5. Vista de compartir documento	42
5.2.6. Vista de documentos compartidos	43
5.2.7. Vista de administración (Solo Admin)	43

5.2.8.	Vista de auditoría blockchain	44
5.2.9.	Vista de verificación pública	45
5.2.10.	Vista de perfil de usuario	46
5.2.11.	Vista de configuración	47
6.	Modelo de analisis y diseño procedimental	47
6.1.	Clases de análisis (BCE)	48
6.2.	Diagramas de secuencia de análisis (patrón BCE)	48
6.2.1.	Análisis BCE: UC-0001 Registrar Usuario	49
6.2.2.	Análisis BCE: UC-0002 Iniciar Sesión	50
6.2.3.	Análisis BCE: UC-0003 Cerrar Sesión	50
6.2.4.	Análisis BCE: UC-0004 Conectar Wallet	51
6.2.5.	Análisis BCE: UC-0005 Eliminar Wallet	51
6.2.6.	Análisis BCE: UC-0006 Establecer Wallet Principal	52
6.2.7.	Análisis BCE: UC-0007 Renombrar Wallet	53
6.2.8.	Análisis BCE: UC-0008 Gestionar Perfil	54
6.2.9.	Análisis BCE: UC-0009 Eliminar Cuenta	55
6.2.10.	Análisis BCE: UC-0010 Cambiar Contraseña	56
6.2.11.	Análisis BCE: UC-0011 Verificar Email	57
6.2.12.	Análisis BCE: UC-0012 Reenviar Verificación de Email	58
6.2.13.	Análisis BCE: UC-0013 Subir Documento	59
6.2.14.	Análisis BCE: UC-0014 Listar Documentos	60
6.2.15.	Análisis BCE: UC-0015 Ver Detalle de Documento	60
6.2.16.	Análisis BCE: UC-0016 Descargar Documento	61
6.2.17.	Análisis BCE: UC-0017 Archivar Documento	62
6.2.18.	Análisis BCE: UC-0018 Desarchivar Documento	63
6.2.19.	Análisis BCE: UC-0019 Eliminar Documento	64
6.2.20.	Análisis BCE: UC-0020 Transferir Documento	65
6.2.21.	Análisis BCE: UC-0021 Buscar Documentos	66
6.2.22.	Análisis BCE: UC-0022 Crear Nueva Versión	67
6.2.23.	Análisis BCE: UC-0023 Listar Versiones	68
6.2.24.	Análisis BCE: UC-0024 Establecer Versión Operativa	69
6.2.25.	Análisis BCE: UC-0025 Restaurar Versión Anterior	70
6.2.26.	Análisis BCE: UC-0026 Descargar Versión Específica	70
6.2.27.	Análisis BCE: UC-0027 Firmar Documento	71
6.2.28.	Análisis BCE: UC-0028 Ver Firmas de un Documento	72
6.2.29.	Análisis BCE: UC-0029 Verificar Firma del Usuario	72
6.2.30.	Análisis BCE: UC-0030 Compartir Documento	73
6.2.31.	Análisis BCE: UC-0031 Revocar Acceso a Documento	74
6.2.32.	Análisis BCE: UC-0032 Ver Compartidos Conmigo	75
6.2.33.	Análisis BCE: UC-0033 Ver Timeline de Documento	75
6.2.34.	Análisis BCE: UC-0034 Ver Estadísticas Personales	76
6.2.35.	Análisis BCE: UC-0035 Ver Estadísticas Globales del Sistema	76
6.2.36.	Análisis BCE: UC-0036 Verificar Autenticidad de Documento	77
6.2.37.	Análisis BCE: UC-0037 Auditoría Blockchain	78
6.2.38.	Análisis BCE: UC-0038 Listar Usuarios	79
6.2.39.	Análisis BCE: UC-0039 Gestionar Roles de Usuario	80
6.3.	Diagramas de secuencia de diseno	80
6.3.1.	Secuencia: Registro de usuario	81
6.3.2.	Secuencia: Iniciar Sesión	81
6.3.3.	Secuencia: Consultar Notificaciones	82

6.3.4.	Secuencia: Cerrar Sesión	82
6.3.5.	Secuencia: Conectar Wallet	83
6.3.6.	Secuencia: Gestionar Perfil	84
6.3.7.	Secuencia: Cambiar Contraseña	85
6.3.8.	Secuencia: Verificar Email	86
6.3.9.	Secuencia: Subir documento	87
6.3.10.	Secuencia: Listar Documentos	87
6.3.11.	Secuencia: Ver Detalle de Documento	87
6.3.12.	Secuencia: Descargar documento	88
6.3.13.	Secuencia: Archivar Documento	88
6.3.14.	Secuencia: Eliminar Documento	89
6.3.15.	Secuencia: Transferir Documento	90
6.3.16.	Secuencia: Buscar Documentos	90
6.3.17.	Secuencia: Crear nueva versión	90
6.3.18.	Secuencia: Listar Versiones	91
6.3.19.	Secuencia: Establecer Versión Operativa	91
6.3.20.	Secuencia: Restaurar Versión Anterior	92
6.3.21.	Secuencia: Descargar Versión Específica	92
6.3.22.	Secuencia: Firmar Documento	93
6.3.23.	Secuencia: Ver Firmas de un Documento	93
6.3.24.	Secuencia: Verificar Firma del Usuario	93
6.3.25.	Secuencia: Compartir Documento	94
6.3.26.	Secuencia: Revocar Acceso a Documento	94
6.3.27.	Secuencia: Ver Documentos Compartidos Conmigo	95
6.3.28.	Secuencia: Ver Timeline de Documento	95
6.3.29.	Secuencia: Ver Estadísticas Personales	95
6.3.30.	Secuencia: Ver Estadísticas Globales del Sistema	96
6.3.31.	Secuencia: Auditoría Blockchain	96
6.3.32.	Secuencia: Listar Usuarios (Admin)	97
6.3.33.	Secuencia: Gestionar Roles de Usuario	97
6.4.	Subflujos y extensiones de los casos de uso principales	97
6.5.	Diagramas de caso de uso-diseño	97
6.5.1.	Colaboración: UC-0013 Subir Documento	98
6.5.2.	Colaboración: UC-0027 Firmar Documento	98
6.5.3.	Flujo complementario: Publicación pública durante la subida	99
6.5.4.	Flujo complementario: Compartición mediante enlace y código QR	99
6.6.	Diagrama de estados: Ciclo de vida del documento	99
6.7.	Diagrama de actividad: Subida de documento con firma	101
7.	Referencia cruzada a los requisitos	103
7.1.	Matriz IRQ – Componente de diseño	103
7.2.	Matriz UC – Clases de análisis	103
7.3.	Cobertura de requisitos no funcionales	104
8.	Plan de desarrollo e implementación	104
8.1.	API REST del sistema	104
8.2.	Diagrama de despliegue	105
8.3.	Tecnologías de cada componente	108
8.4.	Descripción de nodos de despliegue	109
	Referencias	109

Índice de figuras

1.	Diagrama de clases de análisis completo	13
2.	Diagrama Entidad-Relación de la base de datos	16
3.	Modelo integrado de persistencia: tablas PostgreSQL, referencias IPFS y estructuras/mappings del smart contract al mismo nivel	19
4.	Diagrama de paquetes y dependencias	30
5.	Diagrama del patrón Prepare/Confirm	31
6.	Diagrama de clases de diseño completo	34
7.	Diagrama de componentes y comunicación	36
8.	Boceto de la pantalla de inicio de sesión	38
9.	Boceto de la pantalla de registro de usuario	39
10.	Boceto de la vista principal de documentos propios	40
11.	Boceto de la vista de detalle de documento	41
12.	Boceto del modal de compartir documento	42
13.	Boceto de la vista de documentos compartidos	43
14.	Boceto del panel de administración	43
15.	Boceto de la vista de auditoría blockchain	44
16.	Boceto de la vista de verificación de autenticidad	45
17.	Boceto de la vista de perfil de usuario	46
18.	Boceto de la vista de configuración	47
19.	Diagrama de clases de análisis con estereotipos BCE	48
20.	Diagrama de secuencia de análisis BCE – UC-0001: Registrar Usuario	49
21.	Diagrama de secuencia de análisis BCE – UC-0002: Iniciar Sesión	50
22.	Diagrama de secuencia de análisis BCE – UC-0003: Cerrar Sesión	50
23.	Diagrama de secuencia de análisis BCE – UC-0004: Conectar Wallet	51
24.	Diagrama de secuencia de análisis BCE – UC-0005: Eliminar Wallet	51
25.	Diagrama de secuencia de análisis BCE – UC-0006: Establecer Wallet Principal	52
26.	Diagrama de secuencia de análisis BCE – UC-0007: Renombrar Wallet	53
27.	Diagrama de secuencia de análisis BCE – UC-0008: Gestionar Perfil	54
28.	Diagrama de secuencia de análisis BCE – UC-0009: Eliminar Cuenta	55
29.	Diagrama de secuencia de análisis BCE – UC-0010: Cambiar Contraseña	56
30.	Diagrama de secuencia de análisis BCE – UC-0011: Verificar Email	57
31.	Diagrama de secuencia de análisis BCE – UC-0012: Reenviar Verificación de Email	58
32.	Diagrama de secuencia de análisis BCE – UC-0013: Subir Documento	59
33.	Diagrama de secuencia de análisis BCE – UC-0014: Listar Documentos	60
34.	Diagrama de secuencia de análisis BCE – UC-0015: Ver Detalle de Documento	60
35.	Diagrama de secuencia de análisis BCE – UC-0016: Descargar Documento	61
36.	Diagrama de secuencia de análisis BCE – UC-0017: Archivar Documento	62
37.	Diagrama de secuencia de análisis BCE – UC-0018: Desarchivar Documento	63
38.	Diagrama de secuencia de análisis BCE – UC-0019: Eliminar Documento	64
39.	Diagrama de secuencia de análisis BCE – UC-0020: Transferir Documento	65
40.	Diagrama de secuencia de análisis BCE – UC-0021: Buscar Documentos	66
41.	Diagrama de secuencia de análisis BCE – UC-0022: Crear Nueva Versión	67
42.	Diagrama de secuencia de análisis BCE – UC-0023: Listar Versiones	68
43.	Diagrama de secuencia de análisis BCE – UC-0024: Establecer Versión Operativa	69
44.	Diagrama de secuencia de análisis BCE – UC-0025: Restaurar Versión Anterior	70
45.	Diagrama de secuencia de análisis BCE – UC-0026: Descargar Versión Específica	70
46.	Diagrama de secuencia de análisis BCE – UC-0027: Firmar Documento	71
47.	Diagrama de secuencia de análisis BCE – UC-0028: Ver Firmas de un Documento	72

48.	Diagrama de secuencia de análisis BCE – UC-0029: Verificar Firma del Usuario .	72
49.	Diagrama de secuencia de análisis BCE – UC-0030: Compartir Documento	73
50.	Diagrama de secuencia de análisis BCE – UC-0031: Revocar Acceso a Documento	74
51.	Diagrama de secuencia de análisis BCE – UC-0032: Ver Documentos Compartidos Conmigo	75
52.	Diagrama de secuencia de análisis BCE – UC-0033: Ver Timeline de Documento	75
53.	Diagrama de secuencia de análisis BCE – UC-0034: Ver Estadísticas Personales .	76
54.	Diagrama de secuencia de análisis BCE – UC-0035: Ver Estadísticas Globales del Sistema	76
55.	Diagrama de secuencia de análisis BCE – UC-0036: Verificar Autenticidad de Documento	77
56.	Diagrama de secuencia de análisis BCE – UC-0037: Auditoría Blockchain	78
57.	Diagrama de secuencia de análisis BCE – UC-0038: Listar Usuarios	79
58.	Diagrama de secuencia de análisis BCE – UC-0039: Gestionar Roles de Usuario .	80
59.	Diagrama de secuencia UC-0001: Registrar Usuario	81
60.	Diagrama de secuencia UC-0002: Iniciar Sesión	81
61.	Diagrama de secuencia UC-0003: Consultar Notificaciones	82
62.	Diagrama de secuencia UC-0004: Cerrar Sesión	82
63.	Diagrama de secuencia UC-0005: Conectar Wallet	83
64.	Diagrama de secuencia UC-0006: Eliminar Wallet	83
65.	Diagrama de secuencia UC-0007: Establecer Wallet Principal	83
66.	Diagrama de secuencia UC-0008: Renombrar/Etiquetar Wallet	84
67.	Diagrama de secuencia UC-0009: Gestionar Perfil	84
68.	Diagrama de secuencia UC-0010: Configurar Preferencias de Notificación	85
69.	Diagrama de secuencia UC-0010: Cambiar Contraseña	85
70.	Diagrama de secuencia UC-0011: Verificar Email	86
71.	Diagrama de secuencia UC-0015: Reenviar Email de Verificación	86
72.	Diagrama de secuencia UC-0013: Subir Documento	87
73.	Diagrama de secuencia UC-0014: Listar Documentos	87
74.	Diagrama de secuencia UC-0015: Ver Detalle de Documento	87
75.	Diagrama de secuencia UC-0016: Descargar Documento	88
76.	Diagrama de secuencia UC-0017: Archivar Documento	88
77.	Diagrama de secuencia UC-0018: Desarchivar Documento	89
78.	Diagrama de secuencia UC-0019: Eliminar Documento	89
79.	Diagrama de secuencia UC-0020: Transferir Documento	90
80.	Diagrama de secuencia UC-0021: Buscar Documentos	90
81.	Diagrama de secuencia UC-0022: Crear Nueva Versión	90
82.	Diagrama de secuencia UC-0023: Listar Versiones	91
83.	Diagrama de secuencia UC-0024: Establecer Versión Operativa	91
84.	Diagrama de secuencia UC-0025: Restaurar Versión Anterior	92
85.	Diagrama de secuencia UC-0026: Descargar Versión Específica	92
86.	Diagrama de secuencia UC-0027: Firmar Documento	93
87.	Diagrama de secuencia UC-0028: Ver Firmas de un Documento	93
88.	Diagrama de secuencia UC-0029: Verificar Firma del Usuario	93
89.	Diagrama de secuencia UC-0030: Compartir Documento	94
90.	Diagrama de secuencia UC-0031: Revocar Acceso a Documento	94
91.	Diagrama de secuencia UC-0032: Ver Documentos Compartidos Conmigo	95
92.	Diagrama de secuencia UC-0033: Ver Timeline de Documento	95
93.	Diagrama de secuencia UC-0034: Ver Estadísticas Personales	95
94.	Diagrama de secuencia UC-0035: Ver Estadísticas Globales del Sistema	96

95.	Diagrama de secuencia UC-0036: Verificar Autenticidad de Documento	96
96.	Diagrama de secuencia UC-0037: Auditoría Blockchain	96
97.	Diagrama de secuencia UC-0038: Listar Usuarios	97
98.	Diagrama de secuencia UC-0039: Gestionar Roles de Usuario	97
99.	Diagrama de colaboración para UC-0013: Subir Documento	98
100.	Diagrama de colaboración para UC-0027: Firmar Documento	98
101.	Flujo complementario de publicación pública integrado en UC-0013	99
102.	Flujo complementario de distribución por enlace y código QR integrado en UC-0015	99
103.	Diagrama de estados: ciclo de vida del documento	100
104.	Diagrama de actividad: subida de documento con firma (UC-0013)	102
105.	Diagrama de despliegue completo	107

Índice de cuadros

1.	Esquema de la tabla users	21
2.	Esquema de la tabla wallets	21
3.	Esquema de la tabla documents	22
4.	Esquema de la tabla versions	22
5.	Esquema de la tabla document_signatures	23
6.	Esquema de la tabla document_share_keys	23
7.	Esquema de la tabla events	23
8.	Esquema de la tabla folders	24
9.	Esquema de la tabla notifications	24
10.	Esquema de la tabla sessions	24
11.	Campos del struct Document en el smart contract	26
12.	Campos del struct Version en el smart contract	27
13.	Campos del struct Signature en el smart contract	27
14.	Mappings principales del smart contract DocumentRegistry	27
15.	Clasificación Boundary-Control-Entity por paquete funcional	48
16.	Matriz de trazabilidad IRQ – componentes de diseño	103
17.	Matriz de trazabilidad UC – clases de análisis	103
18.	Cobertura de requisitos no funcionales	104
19.	Endpoints principales de la API REST de DocumentChain	105

1. Introducción

Este anexo recorre el análisis y diseño de **DocumentChain**, traduciendo lo especificado en el Anexo I a una arquitectura concreta. El proceso va desde el modelo conceptual (qué tiene que hacer el sistema) hasta el diseño técnico (cómo lo hace), cubriendo todos los aspectos necesarios antes de ponerse a programar.

Se han abordado cuatro grandes áreas: el diseño de datos, que cubre entidades, relaciones y esquema de base de datos; el diseño arquitectónico, con patrones, capas y componentes del sistema; el diseño de interfaz, que incluye prototipos y flujos del frontend web; y el diseño procedimental, con diagramas de secuencia y colaboración para las operaciones más importantes.

La arquitectura de DocumentChain es distribuida y se apoya en un frontend SPA con React, TypeScript y Vite con diseño responsive, un backend API REST con Node.js y Express con su capa de acceso a datos, PostgreSQL con índices optimizados, smart contracts en Solidity y OpenZeppelin sobre Ethereum, almacenamiento distribuido en IPFS con nodo Kubo autoalojado (o Pinata) mediante adaptador, y criptografía AES-256-GCM para documentos privados, RSA-OAEP para proteger claves y publicación pública controlada. El diseño prioriza seguridad, escalabilidad y experiencia de usuario, usando patrones reconocidos y buenas prácticas.

Criterio de correspondencia con el catálogo funcional

El diseño de este anexo sigue el catálogo consolidado de **43 casos de uso principales** del Anexo I. En las secciones de procedimientos se ha puesto el foco en los flujos con más peso funcional, mientras que las operaciones de apoyo se describen como pasos internos o variantes de esos mismos casos de uso, sin inflar el catálogo.

Así, cosas como las carpetas y etiquetas, las notificaciones, las preferencias, la gestión avanzada de wallets o ciertas tareas de administración aparecen desde la perspectiva del flujo principal al que dan servicio. No se ha quitado funcionalidad del sistema: simplemente se ha reorganizado la documentación para que cada caso de uso represente una capacidad completa con sentido propio.

2. Ámbito del software

En este apartado se describe el alcance funcional y los subsistemas principales que componen DocumentChain.

DocumentChain es un sistema de gestión documental descentralizado que permite a organizaciones e individuos almacenar, versionar, compartir y firmar documentos de forma segura y auditable mediante tecnología blockchain.

2.1. Motivación y Problemática

Los sistemas tradicionales de gestión documental arrastran varios problemas conocidos: dependen de un servidor central que es punto único de fallo, no ofrecen trazabilidad real de quién modificó qué, el proveedor puede bloquear el acceso y, en muchos casos, el propio servidor puede ver el contenido de los documentos. Tampoco suelen ofrecer firmas con no-repudio criptográfico de verdad.

DocumentChain intenta resolver esto combinando varias ideas:

- Blockchain para tener inmutabilidad y trazabilidad verificable
- IPFS para descentralizar el almacenamiento
- Cifrado de documentos privados y publicación pública solo cuando el usuario lo decide

- Firmas on-chain con no-repudio

2.2. Subsistemas Principales

2.2.1. Subsistema de Usuario Final (Frontend Web)

Aplicación web de página única (SPA) que proporciona:

- Interfaz responsive para gestión de documentos
- Descifrado en navegador de documentos privados y previsualización pública por tipo MIME
- Integración con wallets compatibles para firmas blockchain
- Drag & drop para subida de archivos
- Vista de timeline y auditoría
- Vistas de documentos, compartición, auditoría y páginas públicas de documento

Características técnicas:

- React con hooks y context API
- TypeScript para type safety
- Tailwind CSS para diseño
- React Router para navegación
- Axios para API calls
- Ethers.js para blockchain

2.2.2. Subsistema de Gestión Backend

API RESTful que coordina:

- Autenticación y autorización (JWT y contraseñas; wallets para firma de operaciones)
- Comunicación con el nodo Kubo autoalojado mediante el adaptador IPFS del backend
- Preparación de transacciones blockchain
- Event listeners para sincronización
- Gestión de permisos y re-encryptación de claves

Arquitectura en capas:

- Controllers: Manejo de peticiones HTTP
- Services: Lógica de negocio
- Repositories: Acceso a datos vía capa de persistencia
- Middleware: Autenticación, validación, rate limiting

2.2.3. Subsistema de Almacenamiento y Persistencia

Base de Datos PostgreSQL Almacena:

- Usuarios, wallets, sesiones
- Metadatos de documentos (nombre, tamaño, hashes)
- Claves simétricas encriptadas por usuario
- Estadísticas agregadas
- Logs de eventos sincronizados de blockchain

Backend de almacenamiento IPFS configurable Servicio IPFS materializado mediante un nodo Kubo persistente que:

- Almacena archivos cifrados o en claro según la visibilidad seleccionada
- Genera CID content-addressable
- Mantiene persistencia del contenido sobre volúmenes propios
- Proporciona disponibilidad operativa dentro de la infraestructura autoalojada
- Permite unpinning al eliminar documentos

2.2.4. Subsistema Blockchain

Smart Contract (DocumentRegistry.sol) Contrato Solidity con OpenZeppelin que gestiona:

- Registro de documentos con metadata hash
- Versionado on-chain
- Control de acceso basado en roles (OWNER, VIEWER, EDITOR)
- Registro de firmas digitales con timestamp
- Transferencias de propiedad
- Salvaguardas de ejecución para integridad transaccional
- Emisión de eventos para sincronización

Event Listeners Servicios Node.js que:

- Escuchan eventos del smart contract
- Sincronizan estado con PostgreSQL
- Actualizan estadísticas
- Disparan notificaciones
- Mantienen consistencia eventual

2.2.5. Subsistema de Seguridad y Cifrado

Implementado de forma coordinada entre frontend y backend:

- Generación de pares RSA-OAEP de 4096 bits durante el alta de usuario
- Cifrado AES-256-GCM en backend para documentos privados
- Cifrado de claves privadas con AES-256-GCM derivando material desde contraseña o recovery key; hash de contraseñas con Argon2id
- Re-criptación de claves simétricas para compartición privada
- Publicación explícita de documentos públicos con `publicId`, enlace y código QR
- Descifrado en cliente y validación de checksums y firmas

Garantías de confidencialidad en la versión final:

- Claves privadas de usuarios
- Contenido privado ya descifrado por el usuario durante la descarga
- Distribución pública accidental, por lo que exige selección explícita y aviso previo

3. Diseño de datos

3.1. Objetos de datos y estructuras de datos

3.1.1. Diagrama de clases de análisis

El siguiente diagrama muestra las entidades principales del dominio y sus relaciones:

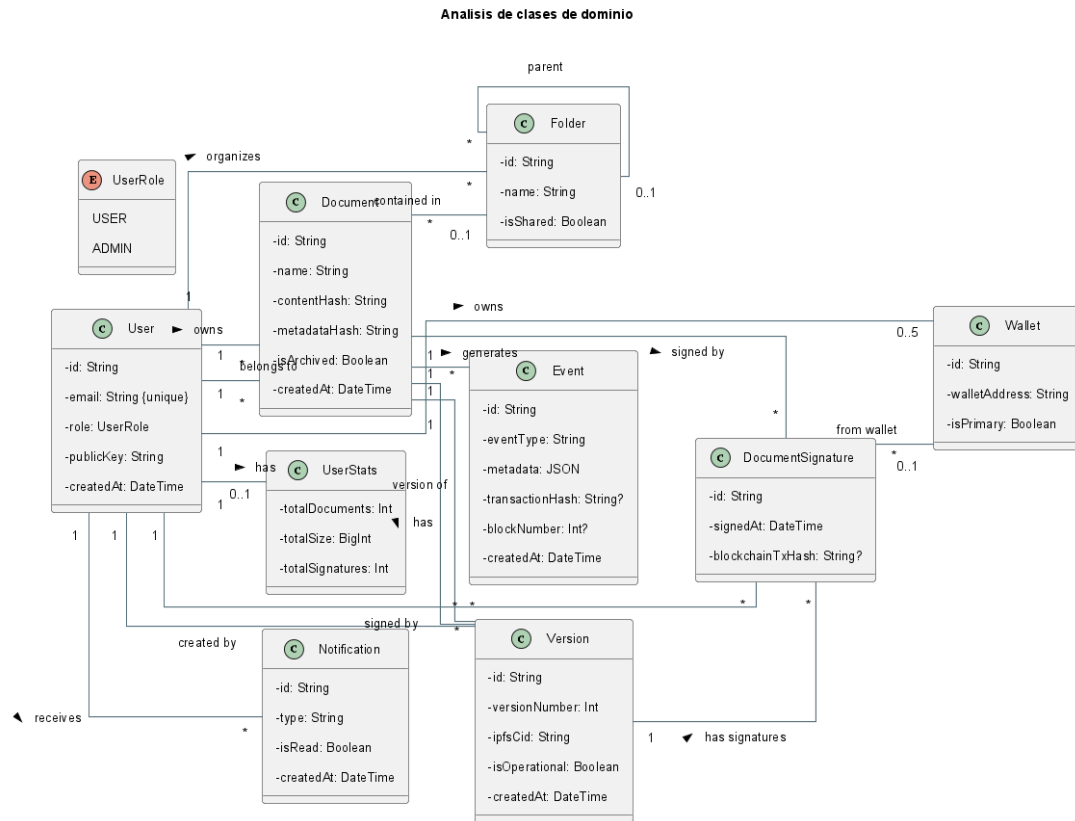


Figura 1: Diagrama de clases de análisis completo

3.2. Estructuras de archivo y bases de datos

El modelo de datos de DocumentChain comprende 16 tablas en PostgreSQL, organizadas en tres grupos funcionales:

- **Entidades principales:** `users`, `wallets`, `documents`. Representan los actores y recursos centrales del sistema.
- **Entidades derivadas:** `versions`, `document_signatures`, `folders`, `events`. Modelan las operaciones y la organización jerárquica de documentos.
- **Entidades de soporte:** `sessions`, `notifications`, `document_stats`, `system_stats`, `notification_preferences`, `email_verifications`, `password_resets`, `system_config`, `document_share_keys`. Cubren autenticación, auditoría, estadísticas, configuración y claves de compartición.

Las relaciones clave son: un `user` puede tener múltiples `wallets`, `documents` y `sessions`; un `document` tiene múltiples `versions` y `document_signatures`; los `events` registran toda la actividad auditada del sistema. La estructura completa, incluyendo las entidades blockchain e IPFS, se representa en el diagrama Entidad-Relación de la siguiente subsección.

3.3. Diagrama Entidad-Relación

El diagrama E-R muestra las relaciones entre las 16 tablas de la base de datos PostgreSQL. Las entidades principales son: `users`, `documents` y `wallets`; las entidades de relación incluyen `versions`, `document_signatures`, `folders` y `events`; las entidades de soporte son `sessions`,

notifications, document_stats, system_stats, notification_preferences, email_verifications, password_resets, system_config y document_share_keys.

Modelo entidad-relacion simplificado

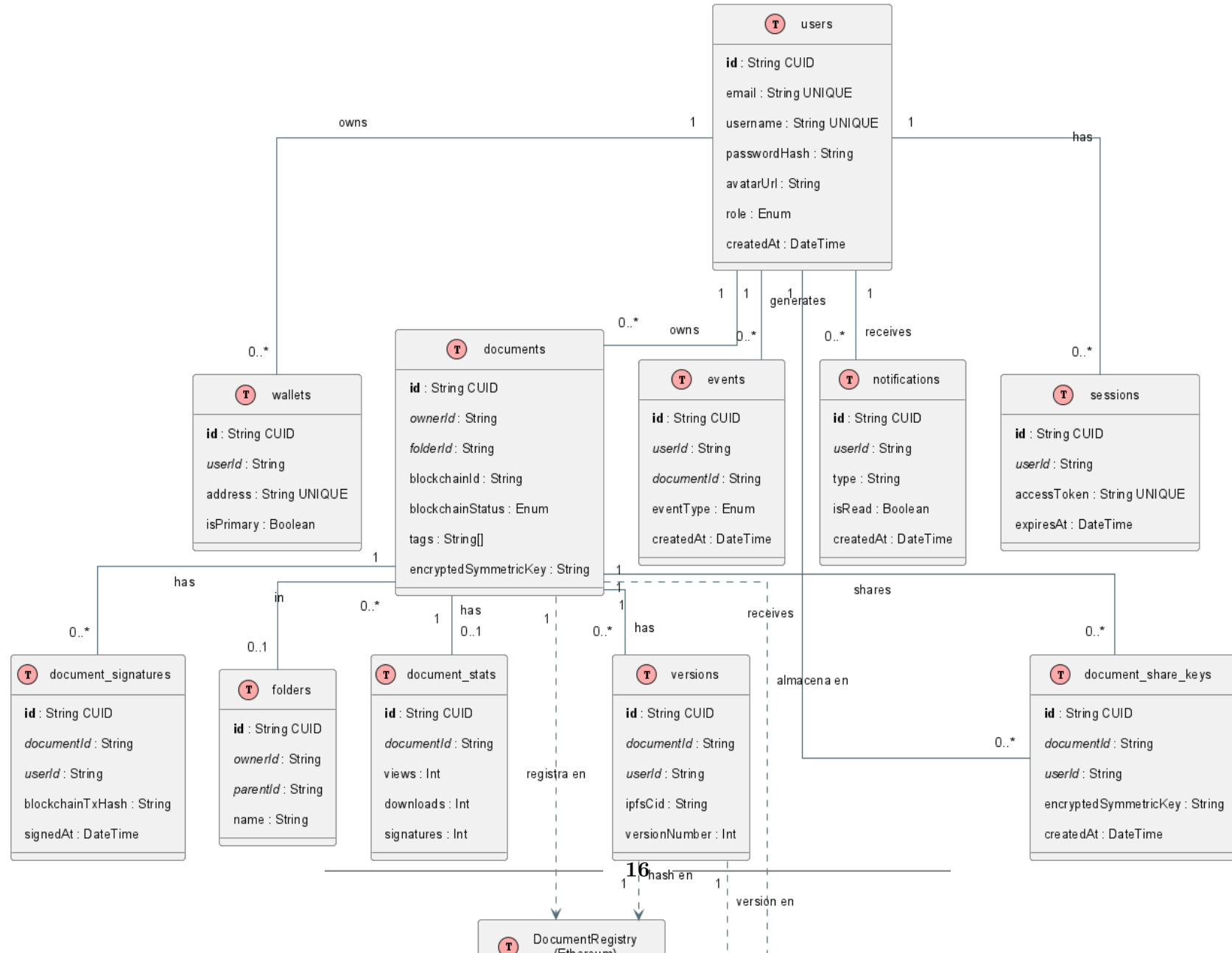


Figura 2: Diagrama Entidad-Relación de la base de datos

3.4. Diagrama conjunto: base de datos y blockchain

DocumentChain distribuye su información entre tres sistemas de persistencia complementarios: PostgreSQL, IPFS y Ethereum. En la arquitectura final no se consideran capas subordinadas entre sí, sino repositorios coordinados que operan al mismo nivel con responsabilidades diferentes. Por ello, el diagrama integrado representa conjuntamente tablas relacionales, referencias IPFS y estructuras *on-chain* como elementos equivalentes dentro del modelo global de persistencia.

El diagrama siguiente muestra las correspondencias entre los tres repositorios, destacando las claves lógicas que enlazan el estado operativo con la verdad verificable registrada en blockchain:

- **documents** ↔ **Struct Document**: el campo `blockchainId` de PostgreSQL corresponde al identificador `docId` registrado en el contrato.
- **versions** ↔ **Struct Version** ↔ **IPFS CID**: el CID de cada versión enlaza simultáneamente el registro operativo, el contenido cifrado y la prueba inmutable de existencia.
- **document_signatures** ↔ **Struct Signature**: la firma operativa se almacena en PostgreSQL y se enlaza con el firmante *on-chain* a través de la wallet asociada al usuario. La tabla conserva además una instantánea mínima de identidad (usuario mostrado y wallet usada) para seguir presentando el firmante en la interfaz aunque cambie el perfil o desaparezca la relación operativa original.
- **events** ↔ **Mapping Permissions**: la tabla `Event` conserva únicamente el registro de auditoría y timeline visual de concesiones, revocaciones y sincronización; el permiso activo y vigente se consulta exclusivamente en el mapping `DocumentRole` del contrato. PostgreSQL nunca se utiliza como fuente de verdad para autorización.
- **wallets** y **users** ↔ **owner / roles**: las direcciones Ethereum enlazan la identidad operativa con la titularidad y los permisos on-chain.

La coexistencia de estos repositorios no constituye redundancia, sino separación de responsabilidades: PostgreSQL resuelve consulta y actualización operativa, IPFS preserva el contenido cifrado y Ethereum conserva la verdad verificable sobre propiedad, permisos, firmas y versiones. Las líneas discontinuas del diagrama marcan las correspondencias transversales entre estos tres dominios de persistencia.

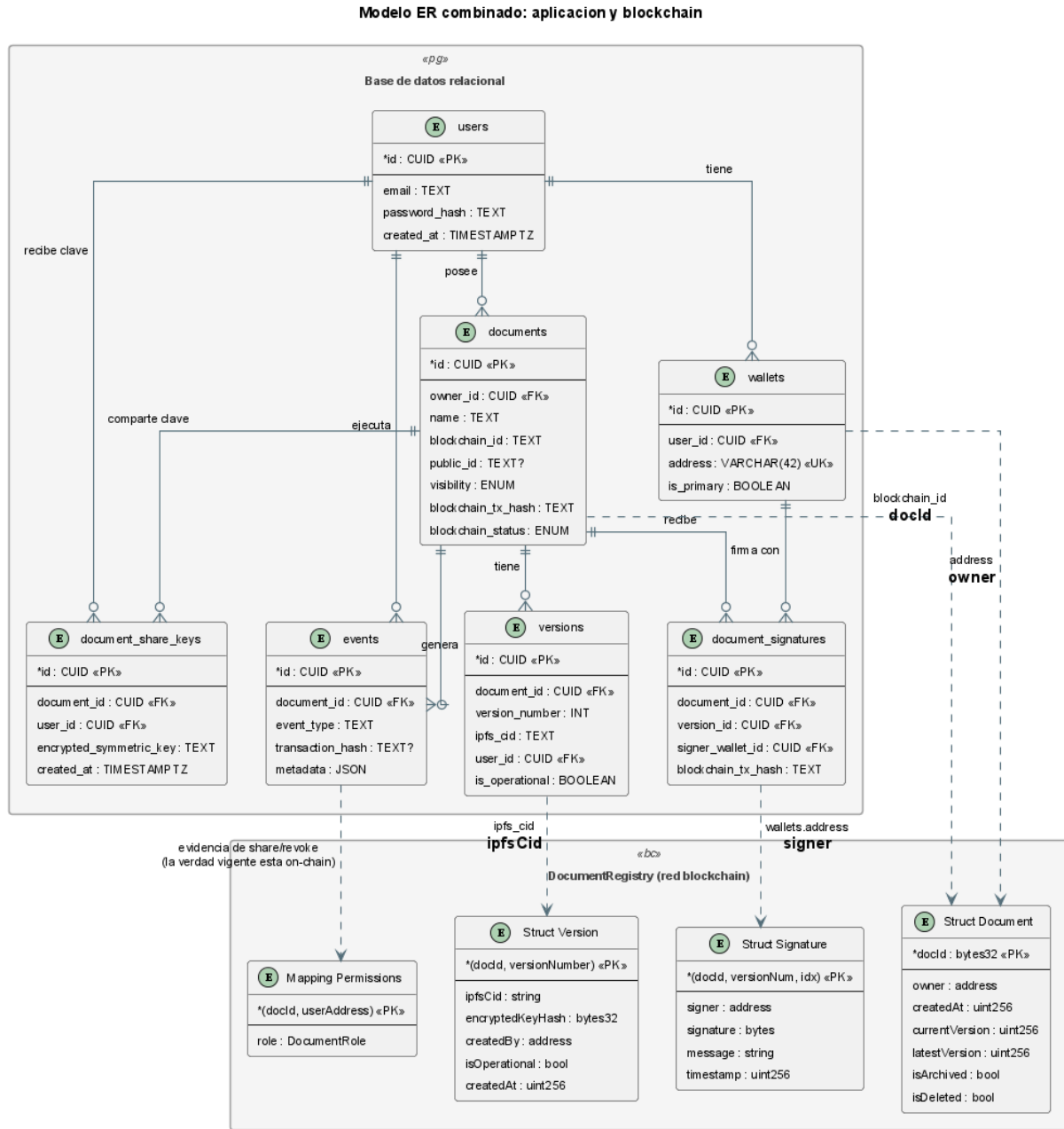


Figura 3: Modelo integrado de persistencia: tablas PostgreSQL, referencias IPFS y estructuras/mappings del smart contract al mismo nivel

3.5. Persistencia coordinada: PostgreSQL, IPFS y blockchain

El sistema usa tres formas de persistencia que se complementan, sin que una esté por encima de otra. La base de datos no sustituye a la blockchain, la blockchain no reemplaza a IPFS e IPFS no modela el estado operativo. Las tres juntas forman el subsistema de persistencia.

Cuando en este trabajo se habla de blockchain no se habla de una base de datos replicada cualquiera. Se habla de un libro mayor distribuido donde cada bloque enlaza criptográficamente con el anterior y el estado cambia mediante transacciones validadas por la red. Lo interesante no es solo que los datos queden guardados, sino que cualquier cambio relevante pasa a formar parte de un historial compartido, difícil de alterar y verificable sin depender del servidor. Eso es justo lo que se necesita para documentos, firmas y permisos que interesa poder contrastar

incluso fuera del contexto normal de la aplicación.

En la práctica, la arquitectura reserva la blockchain para los hechos con más valor probatorio y deja en PostgreSQL las consultas, relaciones y estados que cambian a menudo. IPFS completa el esquema con un almacenamiento distribuido orientado a contenido, donde cada versión se identifica por el hash de su binario, no por una ruta que puede cambiar. Con las tres capas, la aplicación funciona con fluidez en el día a día sin perder capacidad de auditoría externa.

3.5.1. Base de datos relacional (PostgreSQL)

PostgreSQL almacena toda la información que requiere consultas relacionales eficientes y puede ser modificada con el tiempo:

- **Datos de identidad:** cuentas de usuario, credenciales cifradas, configuración de perfil, preferencias de notificación.
- **Metadatos de documentos:** nombre, descripción, estado de blockchain (*PREPARING* / *TX_SUBMITTED* / *SYNCED* / *FAILED*), referencia al CID de IPFS, visibilidad, *publicId* cuando aplica, y marcas de tiempo de creación y modificación.
- **Control de versiones:** historial de versiones por documento, referencia a cada CID en IPFS, autor y fecha de cada versión.
- **Estructura organizativa:** carpetas y etiquetas para la organización jerárquica de documentos.
- **Compartición y auditoría operativa:** eventos de preparación, confirmación y revocación, notificaciones y datos auxiliares para la interfaz y la trazabilidad interna.
- **Sesiones y auditoría interna:** tokens de sesión activos, registro de eventos de acceso, estadísticas de uso.
- **Estado de sincronización:** referencia al hash de transacción blockchain para cada documento/firma, facilitando la reconciliación de estados.

A continuación se detallan las tablas principales de la base de datos con sus campos, tipos y descripciones. Las tablas se agrupan en tres categorías funcionales: entidades principales, entidades de relación y entidades de soporte.

Tabla: users		
Campo	Tipo	Descripcion
id	UUID	Identificador unico del usuario
email	VARCHAR(255)	Correo electronico (unico, verificado)
username	VARCHAR(100)	Nombre de usuario publico
password_hash	VARCHAR(255)	Hash de contrasena (Argon2)
email_verified	BOOLEAN	Estado de verificacion del email
suspended	BOOLEAN	Campo heredado para bloqueos defensivos internos no expuestos en la interfaz final
two_factor_enabled	BOOLEAN	Indicador legado reservado para compatibilidad de esquema
two_factor_secret	VARCHAR(255)	Dato legado no utilizado en el flujo autenticado final
avatar_url	VARCHAR(500)	URL del avatar del usuario
created_at	TIMESTAMP	Fecha de creacion de la cuenta
updated_at	TIMESTAMP	Fecha de ultima modificacion

Cuadro 1: Esquema de la tabla users

Tabla: wallets		
Campo	Tipo	Descripcion
id	UUID	Identificador unico de la wallet
user_id	UUID (FK)	Usuario propietario de la wallet
address	VARCHAR(42)	Direccion Ethereum (0x...)
label	VARCHAR(100)	Etiqueta descriptiva asignada por el usuario
is_primary	BOOLEAN	Wallet principal para operaciones de firma
created_at	TIMESTAMP	Fecha de vinculacion de la wallet

Cuadro 2: Esquema de la tabla wallets

Tabla: documents		
Campo	Tipo	Descripcion
id	UUID	Identificador unico del documento
owner_id	UUID (FK)	Usuario propietario del documento
name	VARCHAR(255)	Nombre del documento
description	TEXT	Descripcion opcional del contenido
blockchain_id	BIGINT	Identificador asignado por el smart contract
blockchain_tx_hash	VARCHAR(66)	Hash de la transaccion de registro on-chain
content_hash	VARCHAR(64)	Hash SHA-256 del contenido original
cid	VARCHAR(100)	Content Identifier en IPFS
status	VARCHAR(30)	Estado: DRAFT, PREPARING, TX_PENDING, SYNCED, ARCHIVED
visibility	VARCHAR(20)	Visibilidad: PRIVATE o PUBLIC
public_id	VARCHAR(50)	Identificador publico para verificacion sin autentificacion
folder_id	UUID (FK)	Carpeta contenedora (opcional)
operational_version_id	UUID (FK)	Version actualmente operativa del documento
created_at	TIMESTAMP	Fecha de creacion
updated_at	TIMESTAMP	Fecha de ultima modificacion

Cuadro 3: Esquema de la tabla documents

Entidades principales

Tabla: versions		
Campo	Tipo	Descripcion
id	UUID	Identificador unico de la version
document_id	UUID (FK)	Documento al que pertenece la version
version_number	INTEGER	Numero secuencial de version
cid	VARCHAR(100)	CID del contenido de esta version en IPFS
content_hash	VARCHAR(64)	Hash SHA-256 del contenido de esta version
blockchain_tx_hash	VARCHAR(66)	Hash de transaccion de registro on-chain
created_by	UUID (FK)	Usuario que creo la version
created_at	TIMESTAMP	Fecha de creacion de la version

Cuadro 4: Esquema de la tabla versions

Tabla: document_signatures		
Campo	Tipo	Descripcion
id	UUID	Identificador unico de la firma
document_id	UUID (FK)	Documento firmado
version_id	UUID (FK)	Version especifica firmada
signer_wallet_id	UUID (FK)	Wallet del firmante
signer_address	VARCHAR(42)	Direccion Ethereum del firmante
blockchain_tx_hash	VARCHAR(66)	Hash de la transaccion de firma on-chain
status	VARCHAR(20)	Estado: ACTIVE, REVOKED
signed_at	TIMESTAMP	Fecha de la firma

Cuadro 5: Esquema de la tabla document_signatures

Tabla: document_share_keys		
Campo	Tipo	Descripcion
id	UUID	Identificador unico del permiso
document_id	UUID (FK)	Documento compartido
shared_with_id	UUID (FK)	Usuario destinatario
shared_by_id	UUID (FK)	Usuario que concede el acceso
role	VARCHAR(20)	Rol asignado: VIEWER, EDITOR
encrypted_key	TEXT	Clave de contenido cifrada para el destinatario (ECDH)
blockchain_tx_hash	VARCHAR(66)	Hash de transaccion de comparticion on-chain
status	VARCHAR(20)	Estado: ACTIVE, REVOKED
created_at	TIMESTAMP	Fecha de concesion del permiso

Cuadro 6: Esquema de la tabla document_share_keys

Tabla: events		
Campo	Tipo	Descripcion
id	UUID	Identificador unico del evento
type	VARCHAR(50)	Tipo de evento (DOCUMENT_CREATED, DOCUMENT_SIGNED, etc.)
user_id	UUID (FK)	Usuario que origino el evento
document_id	UUID (FK)	Documento afectado (opcional)
blockchain_tx_hash	VARCHAR(66)	Hash de transaccion asociada (opcional)
metadata	JSONB	Datos adicionales del evento en formato JSON
created_at	TIMESTAMP	Fecha de ocurrencia del evento

Cuadro 7: Esquema de la tabla events

Tabla: folders		
Campo	Tipo	Descripcion
id	UUID	Identificador unico de la carpeta
name	VARCHAR(255)	Nombre de la carpeta
user_id	UUID (FK)	Usuario propietario de la carpeta
parent_id	UUID (FK)	Carpeta padre (para jerarquia, opcional)
created_at	TIMESTAMP	Fecha de creacion
updated_at	TIMESTAMP	Fecha de ultima modificacion

Cuadro 8: Esquema de la tabla folders

Entidades de relacion

Tabla: notifications		
Campo	Tipo	Descripcion
id	UUID	Identificador unico de la notificacion
user_id	UUID (FK)	Usuario destinatario
type	VARCHAR(50)	Tipo (DOCUMENT_SHARED, SIGNATURE_REQUESTED, etc.)
title	VARCHAR(255)	Titulo de la notificacion
message	TEXT	Contenido descriptivo
read	BOOLEAN	Estado de lectura
metadata	JSONB	Datos adicionales (ID de documento, etc.)
created_at	TIMESTAMP	Fecha de creacion

Cuadro 9: Esquema de la tabla notifications

Tabla: sessions		
Campo	Tipo	Descripcion
id	UUID	Identificador unico de la sesion
user_id	UUID (FK)	Usuario autenticado
access_token	TEXT	Token JWT de acceso (expiracion corta)
refresh_token	TEXT	Token JWT de renovacion
expires_at	TIMESTAMP	Fecha de expiracion del refresh token
created_at	TIMESTAMP	Fecha de inicio de sesion

Cuadro 10: Esquema de la tabla sessions

Entidades de soporte

3.5.2. Almacenamiento distribuido (IPFS configurable)

La capa de almacenamiento IPFS configurable conserva el contenido binario de los archivos de forma direccionable por contenido y persistente:

IPFS (*InterPlanetary File System*) funciona como una red de objetos enlazados por su huella criptográfica. En lugar de localizar un fichero por el servidor y la ruta donde se publicó, se localiza por el identificador derivado de su contenido. Si dos nodos conservan el mismo bloque, ambos pueden servir exactamente el mismo contenido, y si el binario cambia se obtiene necesariamente un nuevo CID. Por lo que, para un sistema con versionado documental, esta propiedad aporta una correspondencia directa entre versión lógica e identidad criptográfica del archivo.

- **Contenido según visibilidad:** los documentos privados se cifran con AES-256-GCM antes de subirse a IPFS; los documentos públicos se almacenan en claro porque se exponen deliberadamente mediante enlace compartible.
- **Contenido inmutable:** cada versión de un archivo genera un CID (Content Identifier) único derivado del hash SHA-256 de su contenido. Ningún CID puede modificarse una vez generado.
- **Persistencia controlada:** el sistema conserva los binarios en el backend IPFS configurado. Puede operar con nodo Kubo autoalojado o con un proveedor gestionado (Pinata), manteniendo la misma semántica de CID.
- El CID generado por IPFS se almacena en PostgreSQL y se registra en la blockchain, creando el puente entre los tres sistemas de persistencia.

Desde el punto de vista arquitectónico, IPFS no sustituye a la base de datos. No mantiene por sí mismo usuarios, permisos, nombres amigables ni estados de sincronización. Su papel consiste en garantizar que el binario asociado a una versión pueda ser recuperado y verificado por contenido. De este modo, PostgreSQL conserva la semántica operativa, IPFS conserva el objeto digital y Ethereum conserva la evidencia de que ese objeto concreto fue asociado a un documento y a una operación determinadas.

3.5.3. Blockchain (Smart Contract en Ethereum)

La blockchain actúa como registro inmutable e infalsificable para operaciones de alta criticidad:

La blockchain permite que cualquiera verifique los datos sin acceder al servidor.

- **Prueba de existencia de documentos:** para cada documento creado o nueva versión, el smart contract almacena el identificador documental, el CID de IPFS, la cuenta responsable y la marca temporal del bloque. Estos datos no pueden alterarse ni eliminarse una vez confirmados.
- **Registro de firmas digitales:** cada firma de documento queda inmutablemente registrada en la blockchain con la dirección Ethereum del firmante, el CID del documento firmado y la fecha/hora de la firma verificable criptográficamente.
- **Trazabilidad de propiedad:** el smart contract mantiene el histórico de transferencias de propiedad de documentos.
- **Validación independiente:** cualquier tercero puede verificar la autenticidad e integridad de un documento consultando directamente la blockchain, sin depender del sistema central.
- La blockchain NO almacena el contenido de los archivos (prohibitivo en coste y tamaño), sino únicamente las referencias (CIDs) y los hashes como prueba criptográfica.

3.5.4. Estructuras de datos del smart contract

El contrato `DocumentRegistry.sol` organiza sus datos en tres *structs* principales, varios *mappings* anidados y un sistema de roles granulares. A continuación se describe cada estructura:

La decisión de concentrar el comportamiento en un único contrato simplifica la coordinación entre frontend, backend y wallet del usuario. En lugar de repartir la lógica entre varios contratos especializados, se ha preferido una autoridad on-chain única para el ciclo de vida documental: creación, versionado, compartición, firma, transferencia, archivado y borrado lógico. Esta organización reduce saltos entre contratos, disminuye el número de transacciones por operación y facilita la correspondencia con los casos de uso descritos en la aplicación.

Tipos y roles El contrato define un tipo enumerado `DocumentRole` para expresar el nivel de acceso de cada usuario a un documento:

- `NONE` (0): sin acceso al documento.
- `VIEWER` (1): solo lectura; puede descargar y verificar firmas.
- `EDITOR` (2): lectura y escritura; puede subir nuevas versiones.
- `OWNER` (3): control total; puede compartir, firmar, transferir y eliminar.

Adicionalmente, el sistema de control de acceso basado en roles (RBAC) de OpenZeppelin gestiona `ADMIN_ROLE` para administradores de la plataforma. El contrato también define `OPERATOR_ROLE`, pero en el modelo relacional de PostgreSQL solo existen los roles `ADMIN` y `USER`, por lo que `OPERATOR_ROLE` no tiene representación operativa en la base de datos.

Struct Document Almacena los metadatos de cada documento registrado en la blockchain. El identificador principal es un `bytes32` derivado del UUID del documento en base de datos:

Campo	Tipo	Descripción
<code>docId</code>	<code>bytes32</code>	Identificador único del documento (hash del UUID)
<code>owner</code>	<code>address</code>	Dirección Ethereum del propietario actual
<code>createdAt</code>	<code>uint256</code>	Marca de tiempo de creación (Unix, <i>block.timestamp</i>)
<code>updatedAt</code>	<code>uint256</code>	Marca de tiempo de la última modificación
<code>currentVersion</code>	<code>uint256</code>	Número de la versión operativa activa
<code>latestVersion</code>	<code>uint256</code>	Número de la versión más reciente creada
<code>isArchived</code>	<code>bool</code>	Indica si el documento está archivado
<code>isDeleted</code>	<code>bool</code>	Eliminación lógica (los datos permanecen en blockchain)

Cuadro 11: Campos del struct Document en el smart contract

Struct Version Cada versión de un documento queda registrada de forma inmutable con su referencia a IPFS y, cuando procede, la metadata criptográfica correspondiente:

Campo	Tipo	Descripción
versionNumber	uint256	Número secuencial de la versión
ipfsCid	string	CID de IPFS del archivo almacenado (cifrado si es privado; en claro si es público)
encryptedKeyHash	bytes32	Hash de la clave de cifrado para verificación
createdBy	address	Dirección Ethereum del autor de la versión
createdAt	uint256	Marca de tiempo de la versión
isOperational	bool	Indica si es la versión activa del documento
restoredFrom	uint256	Versión origen si fue restaurada (0 si es nueva)

Cuadro 12: Campos del struct Version en el smart contract

Struct Signature Registra de forma inmutable cada firma digital aplicada sobre una versión concreta de un documento:

Campo	Tipo	Descripción
signer	address	Dirección Ethereum del firmante
signature	bytes	Firma criptográfica ECDSA en bytes
message	string	Mensaje o hash firmado
comment	string	Comentario opcional del firmante
timestamp	uint256	Marca de tiempo exacta de la firma

Cuadro 13: Campos del struct Signature en el smart contract

Mappings de estado El contrato utiliza mappings anidados para asociar eficientemente documentos con sus versiones, firmas y permisos de acceso:

Mapping	Propósito
_documents[docId]	Recuperar el struct Document por identificador
_versions[docId][versionNum]	Recuperar el struct Version de una versión concreta
_signatures[docId][versionNum]	Array de firmas sobre una versión dada
_hasSigned[docId][versionNum][address]	Evitar firmas duplicadas por direccion
_permissions[docId][address]	Rol (NONE/VIEWER/EDITOR/OWNER) de cada usuario
_documentUsers[docId]	Conjunto de direcciones con acceso al documento
_userDocuments[address]	Conjunto de documentos accesibles para un usuario
_suspendedUsers[address]	Salvaguarda interna heredada para controles defensivos no expuestos en la interfaz principal

Cuadro 14: Mappings principales del smart contract DocumentRegistry

Además de estas estructuras, el contrato apoya su comportamiento en eventos y restricciones de ejecución. Los eventos `DocumentCreated`, `VersionCreated`, `DocumentSigned`, `DocumentShared`, `PermissionRevoked`, `OwnershipTransferred` u `OperationalVersionChanged` permiten reconstruir la secuencia de acciones críticas y sirven como punto de reconciliación para el backend. En paralelo, los modificadores `nonReentrant` y `notDeleted` restringen la ejecución en condiciones incompatibles con la operación documental; el contrato conserva además ciertas salvaguardas históricas de bajo nivel no expuestas como casos de uso del sistema final. Se observa así una

combinación de trazabilidad y control preventivo: la cadena no solo registra lo ocurrido, sino que impone condiciones previas antes de aceptar cada transición de estado.

Modelo coordinado de verdad y operación La arquitectura de DocumentChain separa responsabilidades de forma nítida: **PostgreSQL gestiona el estado operativo que puede cambiar, IPFS guarda el contenido binario de cada versión y la blockchain garantiza la verdad inmutable y verificable.** PostgreSQL puede actualizar registros, recalcular estadísticas o mantener estados temporales; IPFS conserva cada archivo identificado por su CID, cifrado si es privado y en claro si es público; Ethereum fija la evidencia de propiedad, permisos, firmas, versiones y transferencias. Dicho esto, el flujo real cuando alguien sube un documento es el siguiente.

3.5.5. Flujo de persistencia coordinado

Cuando un usuario sube un documento, los tres sistemas actúan en secuencia:

1. El usuario envía el archivo y la visibilidad seleccionada; el backend cifra el contenido si es privado o lo mantiene en claro si es público antes de subirlo a IPFS, obteniendo un CID único.
2. El backend persiste el CID y los metadatos en PostgreSQL con estado *PREPARING*, junto con la visibilidad y el `publicId` cuando corresponde.
3. El usuario firma la transacción con su wallet conectada; el smart contract registra el CID y el hash en la blockchain.
4. El event listener detecta el evento blockchain, actualiza el estado en PostgreSQL a *SYN-CED* con el hash de la transacción y el identificador blockchain generado.

Este flujo refleja el patrón *prepare/confirm* adoptado por la aplicación. La fase de preparación permite validar, cifrar, subir y persistir el estado provisional sin custodiar la clave privada del usuario en el backend. La fase de confirmación incorpora la evidencia de la transacción firmada por la wallet y deja la operación lista para reconciliación posterior. El resultado es una separación nítida entre operación off-chain y consolidación on-chain, necesaria para mantener seguridad, trazabilidad y una experiencia de uso razonable.

4. Diseño arquitectónico

4.1. Arquitectura de capas

DocumentChain sigue una arquitectura en capas donde cada una tiene una responsabilidad clara y no se mezcla con las demás.

4.1.1. Capas del Sistema

1. Capa de Presentación (Frontend)

- Componentes React con diseño atómico
- Gestión de estado con Context API
- Lógica de cifrado/descifrado
- Interacción con la wallet conectada
- Consumo de API REST

2. Capa de Aplicación (Backend API)

- Controllars: manejo de peticiones HTTP
- Services: lógica de negocio
- Middleware: autenticación, validación, rate limiting
- Routers: enrutamiento de endpoints

3. Capa de Dominio

- Entidades del modelo de datos
- Reglas de negocio core
- Validaciones de dominio

4. Capa de Infraestructura

- Capa de acceso a base de datos (sobre PostgreSQL)
- Adaptador del nodo IPFS
- Cliente Ethereum (ethers.js)
- Event Listeners blockchain
- Sistema de email
- WebSocket server

5. Capa de Blockchain

- Smart Contract DocumentRegistry
- Nodos Ethereum (Hardhat local / Sepolia testnet)
- Eventos on-chain

6. Capa de Almacenamiento

- Backend IPFS compatible (nodo Kubo autoalojado)
- PostgreSQL database

4.1.2. Diagrama de paquetes

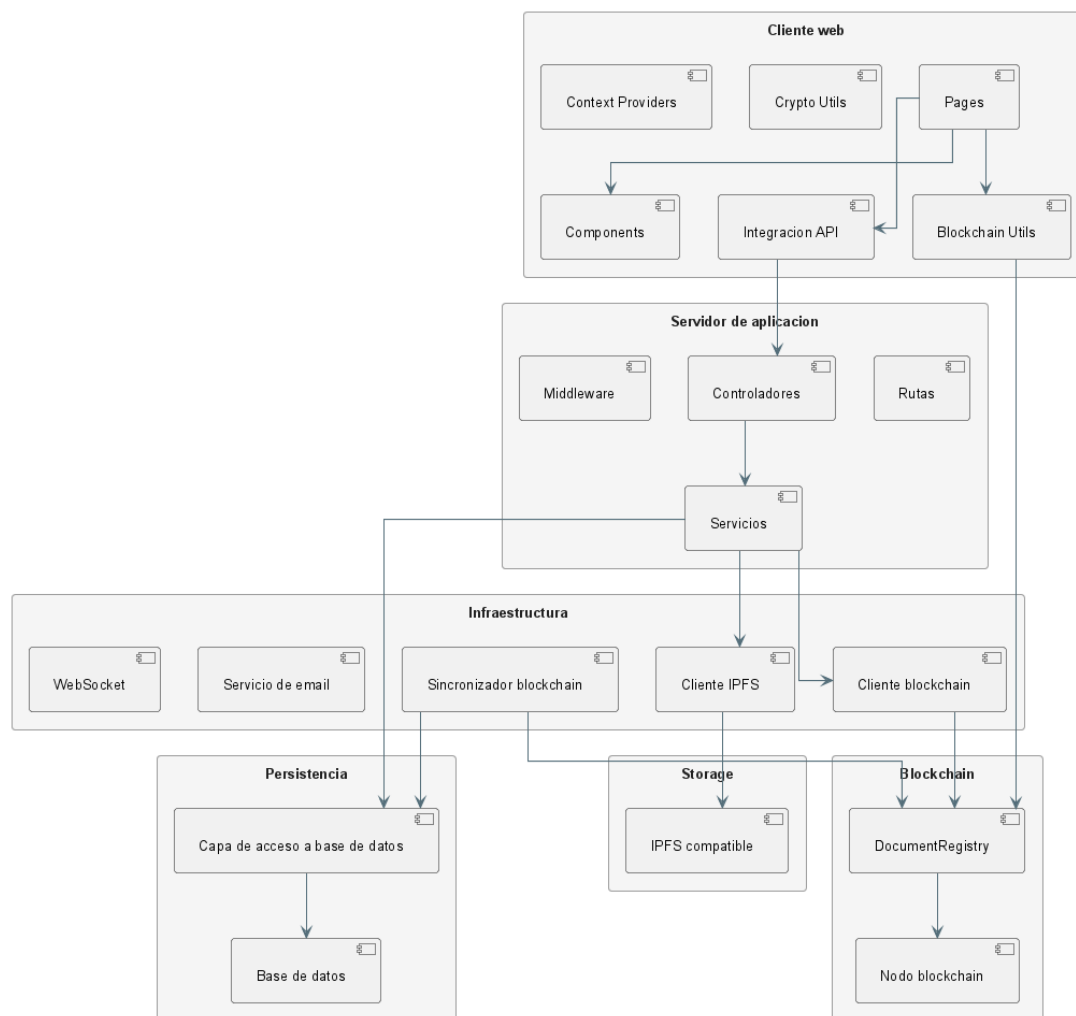


Figura 4: Diagrama de paquetes y dependencias

4.2. Patrones de diseño aplicados

4.2.1. Prepare/Confirm Pattern (Transacciones Blockchain)

Problema: Las transacciones en blockchain las tiene que firmar el usuario con su wallet. Esto crea un flujo asíncrono que puede ser complicado de manejar.

Solución: Se optó por dividir cada operación en dos fases, prepare y confirm:

1. **Prepare:** El backend lo deja todo listo (sube a IPFS, guarda en BD con estado PREPARING) y devuelve los datos que necesita la transacción
2. **Sign:** El frontend pide al usuario que firme con su wallet
3. **Confirm:** El frontend manda el txHash al backend, que cambia el estado a TX_SUBMITTED
4. **Sync:** Un event listener espera a que la transacción se minee y actualiza a SYNCED

Ventajas:

- Si el usuario rechaza la firma no se pierde nada
- Se puede reintentar si algo falla
- El estado nunca queda inconsistente
- El usuario mantiene el control de su wallet en todo momento

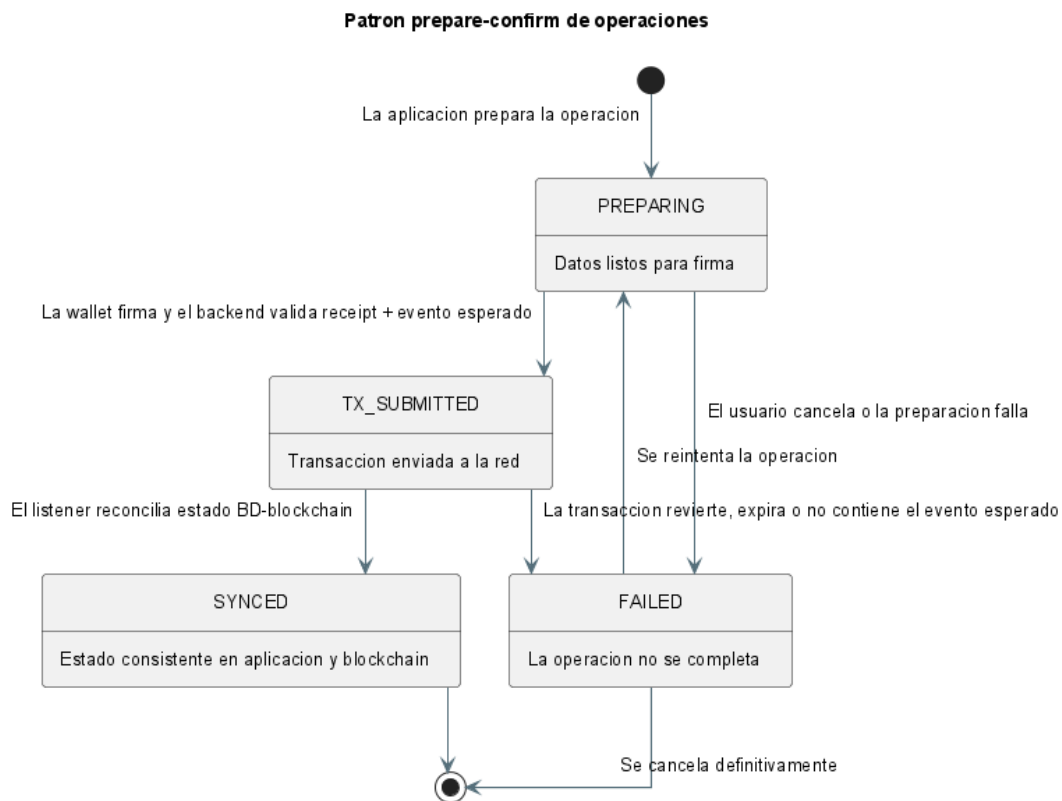


Figura 5: Diagrama del patrón Prepare/Confirm

4.2.2. Repository Pattern

Problema: Acoplamiento directo con la base de datos dificulta testing y cambios.

Solución: La capa de acceso a base de datos actúa como Repository abstrayendo la persistencia.

- Services solo dependen de la interfaz de acceso a datos
- Queries centralizadas y reutilizables
- Fácil mockeo para tests

4.2.3. Service Layer Pattern

Problema: Controllers con lógica de negocio complican reutilización y testing.

Solución: Controllers delegados que delegan en Services:

Los *controllers* reciben las peticiones HTTP, validan los parámetros de entrada y delegan inmediatamente en la capa de servicios. Los *services* concentran toda la lógica de negocio: validación de datos de dominio, orquestación de operaciones (cifrado, subida a IPFS, escritura en

base de datos, preparación de transacciones blockchain), y actualización de estadísticas. Esta separación facilita el testing unitario de la lógica de negocio sin dependencias HTTP.

4.2.4. Event-Driven Architecture

Event listeners escuchan eventos del smart contract y sincronizan estado:

La arquitectura se apoya en event listeners que vigilan el smart contract en tiempo real. Cuando salta un evento blockchain (por ejemplo, *DocumentCreated*), el listener busca la transacción pendiente en base de datos por su hash, pone el documento en estado *SYNCED* con el identificador on-chain, actualiza las estadísticas del propietario y manda una notificación al cliente. Así se mantiene la coherencia entre lo que hay en base de datos y lo que está registrado en blockchain.

4.2.5. Salvaguardas de ejecución del contrato

El smart contract incluye ciertas protecciones pensadas para la integridad de las transacciones y el control de acceso. Básicamente se trata de validar los roles antes de actuar, comprobar que el documento está en un estado válido y usar *ReentrancyGuard* en las funciones más delicadas. Son medidas internas que no forman parte del catálogo funcional como casos de uso, pero que evitan problemas graves en ejecución.

4.2.6. Strategy (Multi-Proveedor de Wallet)

El sistema debe soportar múltiples proveedores de wallet (MetaMask, Rabby, WalletConnect, Coinbase Wallet, entre otros) sin acoplar el código del frontend a las API específicas de cada uno. Para ello se adopta el **patron Strategy**, que define una interfaz común (*WalletProvider*) y delega la conexión, firma y desconexión en implementaciones concretas seleccionables en tiempo de ejecución.

El *WalletContext* del frontend actúa como contexto del patron. Al detectar los proveedores disponibles mediante el estándar EIP-6963, construye una colección de objetos *WalletProvider* que encapsulan la lógica específica de cada wallet:

- **Interfaz común (*IWalletProvider*):** define los métodos `connect()`, `disconnect()`, `signMessage(message, data)`, `signTypedData(domain, types, value)`, `getAddress()` y `getChainId()`.
- **Estrategia EIP-6963:** utiliza el evento `eip6963:announceProvider` para detectar wallets inyectadas en el navegador. Cada anuncio incluye un UUID único, nombre, icono y referencia al proveedor EIP-1193.
- **Estrategia WalletConnect v2:** para wallets móviles o de escritorio que no se inyectan en el navegador, se utiliza el protocolo WalletConnect mediante un proveedor remoto accesible vía QR o enlace profundo.
- **Selección dinámica:** el componente *WalletSelectorModal* presenta al usuario las estrategias disponibles (wallets detectadas). Al seleccionar una, el contexto instancia la estrategia correspondiente y todas las operaciones subsiguientes de firma utilizan esa instancia sin que el código cliente conozca los detalles del proveedor subyacente.

Este diseño presenta varias ventajas. En primer lugar, permite añadir soporte para nuevas wallets sin modificar el código de los componentes que consumen la firma, por lo que basta con implementar la interfaz *IWalletProvider* y registrarla en el contexto. En segundo lugar, evita el *vendor lock-in* con MetaMask, empoderando al usuario para elegir su proveedor de confianza. En tercer lugar, el desacoplamiento facilita el testing unitario al permitir la inyección de una estrategia mock que simule firmas sin interactuar con una wallet real.

4.3. Diagrama de clases de diseño

El diagrama de clases de diseño incluye métodos, visibilidad y relaciones técnicas:

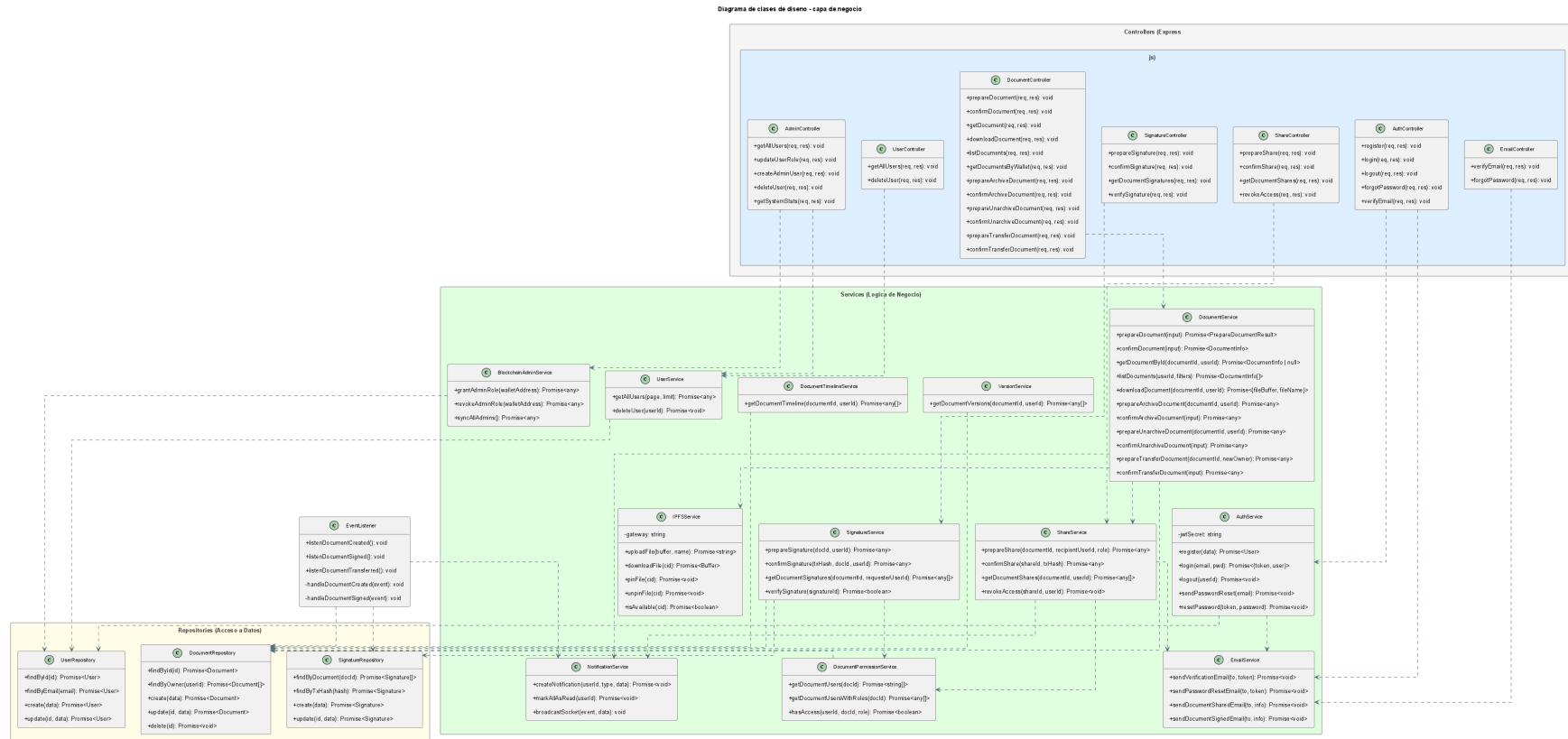


Figura 6: Diagrama de clases de diseño completo

4.3.1. Arquitectura del sistema por componentes

El diagrama siguiente muestra la comunicación entre todos los módulos del sistema: el servidor API (Node.js + Express) concentra los Blockchain Event Listeners, los Routers, el Auth Middleware JWT+RBAC y la capa de Services. Los clientes acceden mediante HTTPS REST y WebSocket (Socket.IO). Las tres capas de persistencia son la base de datos PostgreSQL, el Smart Contract en Ethereum y el backend IPFS configurable consumido mediante el adaptador de almacenamiento del backend.

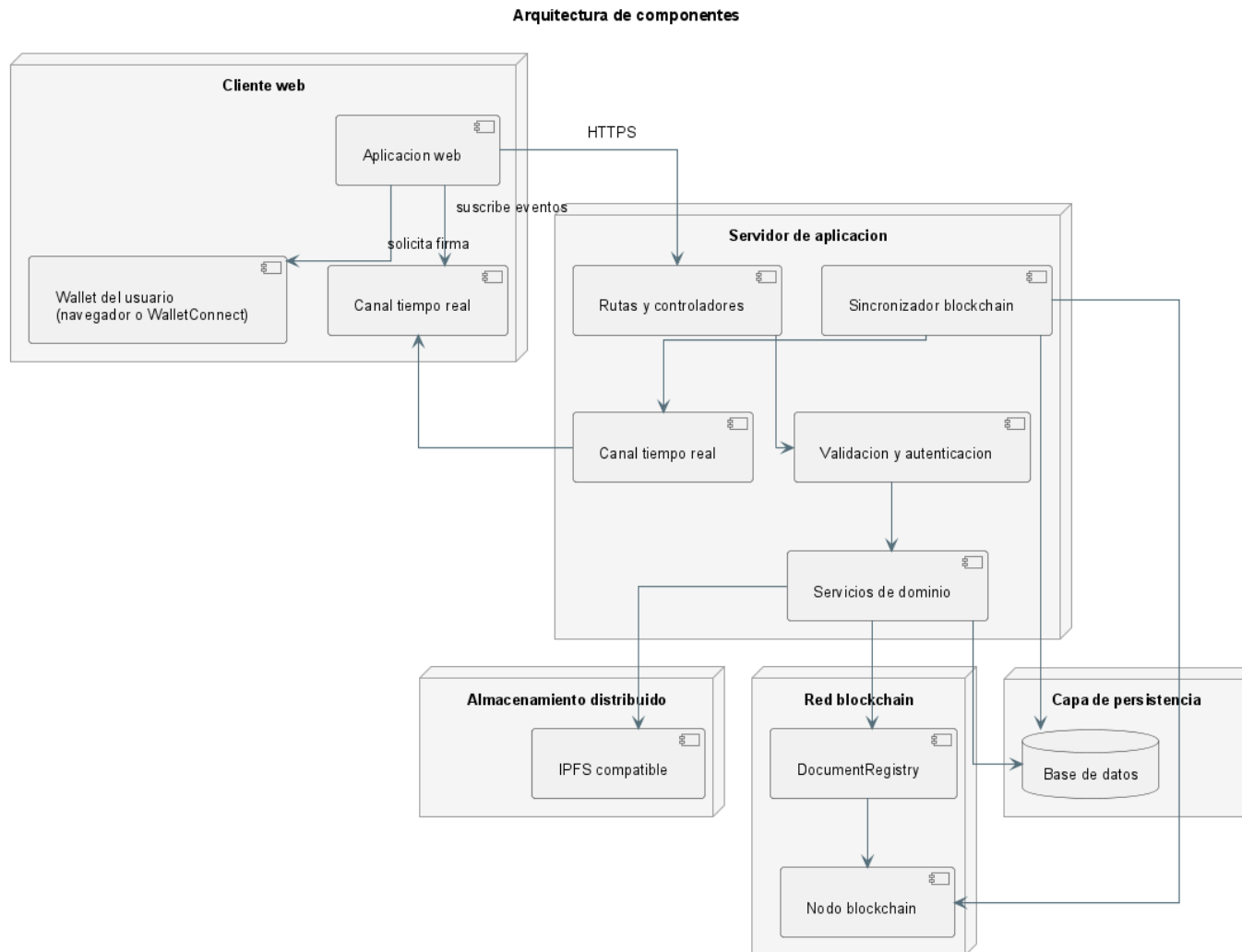


Figura 7: Diagrama de componentes y comunicación

5. Diseño de la interfaz

En este apartado se documentan los principios, componentes y prototipos que definen la interfaz de usuario del sistema.

5.1. Representación de las interfaces del sistema

La interfaz de DocumentChain se diseñó con los siguientes principios:

- **Material Design adaptado:** Componentes reconocibles y predecibles
- **Responsive first:** Diseño móvil primero, escala a desktop
- **Accesibilidad:** Contraste WCAG AA, navegación por teclado
- **Feedback inmediato:** Loading spinners, toasts, progress bars
- **Tema oscuro/claro:** Respeta preferencias del sistema

5.1.1. Componentes principales

1. Layout Components

- **Navbar:** navegación principal con logo, perfil, campana de notificaciones y acceso a ajustes
- **Sidebar:** navegación secundaria a documentos, compartidos, verificación, exploración blockchain y árbol contextual de carpetas
- **Footer:** links de ayuda, versión, estado del sistema

2. Document Components

- **DocumentCard:** tarjeta con preview, metadata, acciones
- **DocumentList:** listado con paginación y sorting
- **DocumentUploader:** drag&drop con progress y selector de visibilidad privada/pública
- **DocumentViewer:** modal de preview con acciones
- **PublicDocumentPreview:** previsualización inline de documentos públicos por tipo MIME
- **PublicLinkActions:** generación de enlace compartible y código QR

3. Form Components

- **Input, Textarea, Select** con validación
- **FileInput** con preview
- **WalletConnector:** punto de entrada para conectar una wallet compatible, incluyendo proveedores de navegador y WalletConnect v2 mediante código QR.
- **PasswordStrengthIndicator**

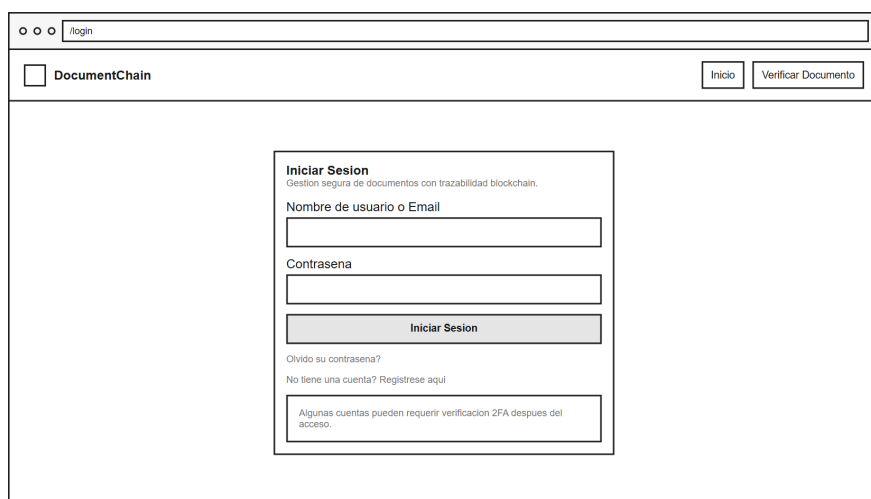
4. Feedback Components

- **Toast:** notificaciones temporales
- **Modal:** diálogos de confirmación
- **LoadingSpinner:** indicadores de carga
- **ProgressBar:** progreso de operaciones largas

5.2. Prototipos de la interfaz de los componentes del sistema

A continuación se incluyen bocetos de baja fidelidad de las vistas de mayor entidad funcional del sistema. Se han seleccionado únicamente las pantallas estructurales y los flujos principales cuya representación actual resulta suficientemente fiel al producto implementado, mientras que las capturas detalladas de la versión final se reservan para el Anexo V.

5.2.1. Vista de inicio de sesión



o o o /login

☐ DocumentChain

Iniciar Sesión
 Gestión segura de documentos con trazabilidad blockchain.

Nombre de usuario o Email

Contraseña

[Olvido su contraseña?](#)

[No tiene una cuenta? Regístrese aquí](#)

Algunas cuentas pueden requerir verificación 2FA después del acceso.

Figura 8: Boceto de la pantalla de inicio de sesión

Elementos:

- Formulario con nombre de usuario o email y contraseña.
- Recuperación de contraseña mediante enlace dedicado.
- Acceso al registro de nuevos usuarios.

5.2.2. Vista de registro

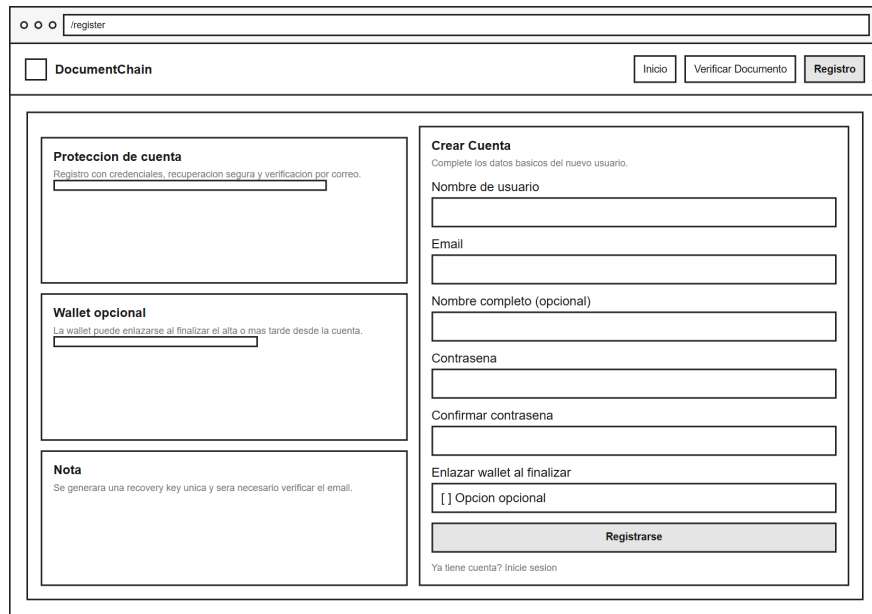


Figura 9: Boceto de la pantalla de registro de usuario

Elementos:

- Formulario con username, email, nombre completo opcional y doble confirmación de contraseña.
- Presentación obligatoria de la clave de recuperación al completar el alta.
- Pantalla de confirmación que informa de la necesidad de verificar el correo antes del primer inicio de sesión.
- Vinculación de wallet relegada al perfil una vez validado el acceso a la cuenta.

5.2.3. Vista principal de documentos

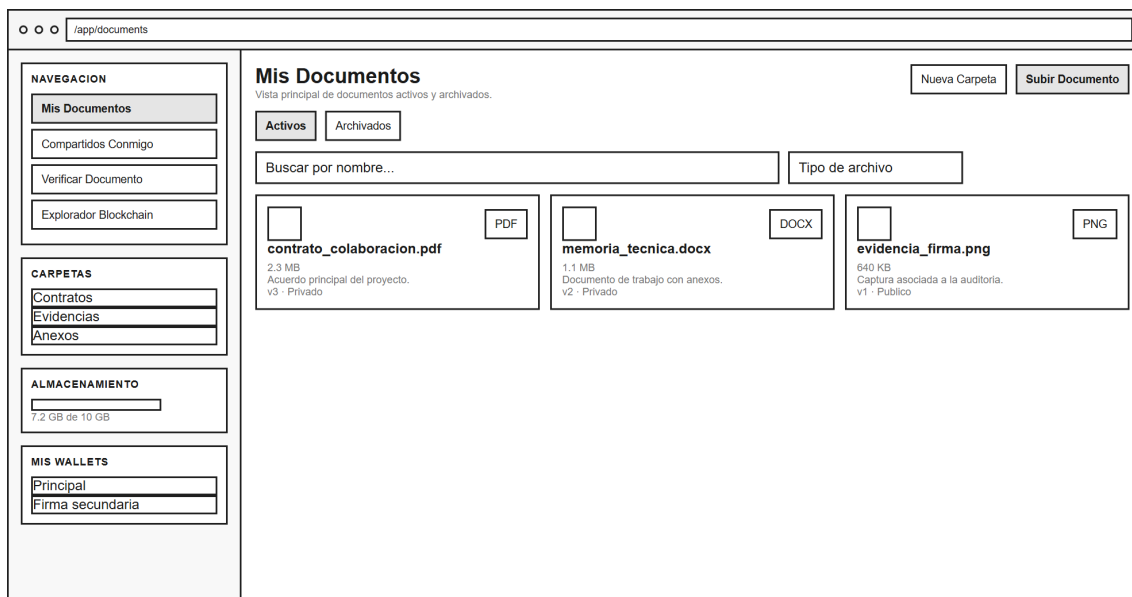


Figura 10: Boceto de la vista principal de documentos propios

Elementos:

- Barra lateral con accesos a documentos propios, compartidos, verificación, explorador blockchain y organización por carpetas cuando procede.
- Zona superior con acciones de crear carpeta y subir documento.
- Filtros por texto, tipo de archivo y estado archivado.
- Listado paginado de documentos del usuario autenticado.

5.2.4. Vista de detalle de documento

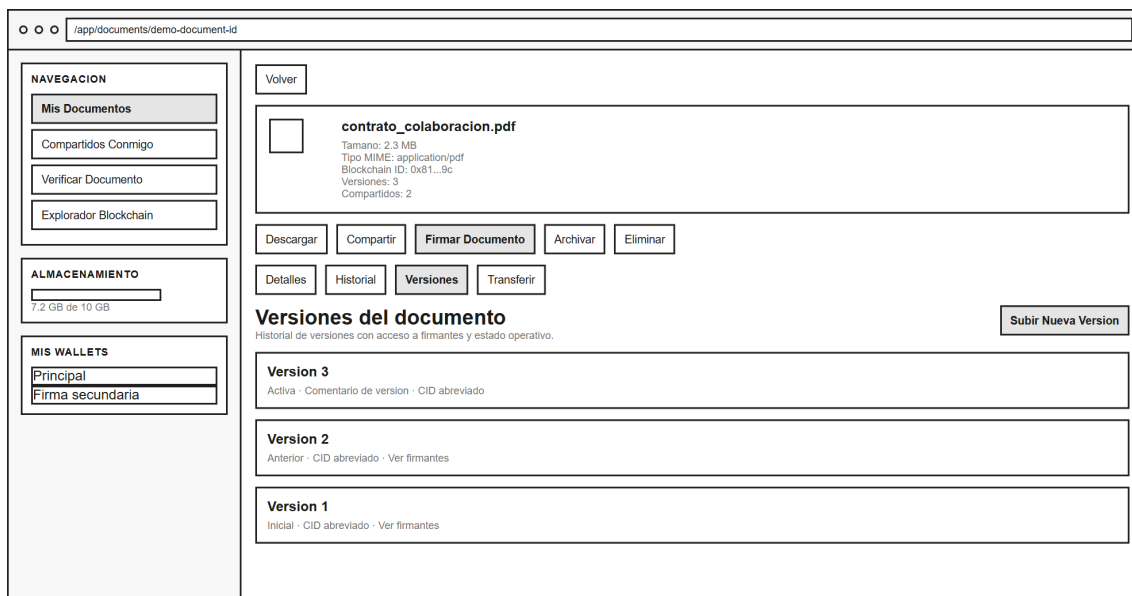
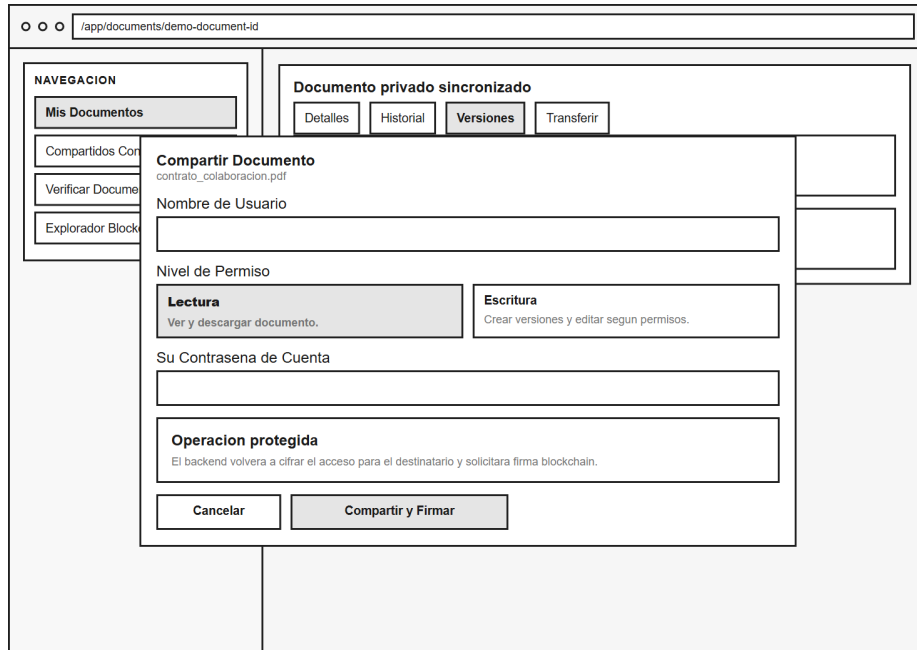


Figura 11: Boceto de la vista de detalle de documento

Elementos:

- Tarjeta de resumen con metadatos principales del documento seleccionado.
- Fila de acciones de descarga, compartición, firma, archivado y eliminación.
- Navegación por pestañas para detalles, historial, versiones y transferencia.
- Zona central orientada al historial de versiones y a la activación de una nueva versión operativa.

5.2.5. Vista de compartir documento



The wireframe shows a web application interface for sharing a document. At the top, there is a browser-like address bar with the URL "/app/documents/demo-document-id". Below this, the interface is divided into a left sidebar and a main content area. The sidebar, titled "NAVEGACION", contains a "Mis Documentos" button and a list of other options: "Compartidos Con...", "Verificar Docume...", and "Explorador Block...". The main content area is titled "Documento privado sincronizado" and includes sub-tabs: "Detalles", "Historial", "Versiones", and "Transferir". The "Compartir Documento" section is the primary focus, displaying the document name "contrato_colaboracion.pdf". It features a "Nombre de Usuario" input field, a "Nivel de Permiso" section with two options: "Lectura" (Ver y descargar documento.) and "Escritura" (Crear versiones y editar segun permisos.), and a "Su Contraseña de Cuenta" input field. A confirmation message states: "Operacion protegida. El backend volvera a cifrar el acceso para el destinatario y solicitara firma blockchain." At the bottom of this section are two buttons: "Cancelar" and "Compartir y Firmar".

Figura 12: Boceto del modal de compartir documento

Elementos:

- Búsqueda de usuario destino.
- Selección del nivel de permiso realmente soportado por el sistema.
- Confirmación con contraseña de cuenta y firma blockchain mediante wallet.
- Resumen del estado de compartición tras completarse la operación.

5.2.6. Vista de documentos compartidos

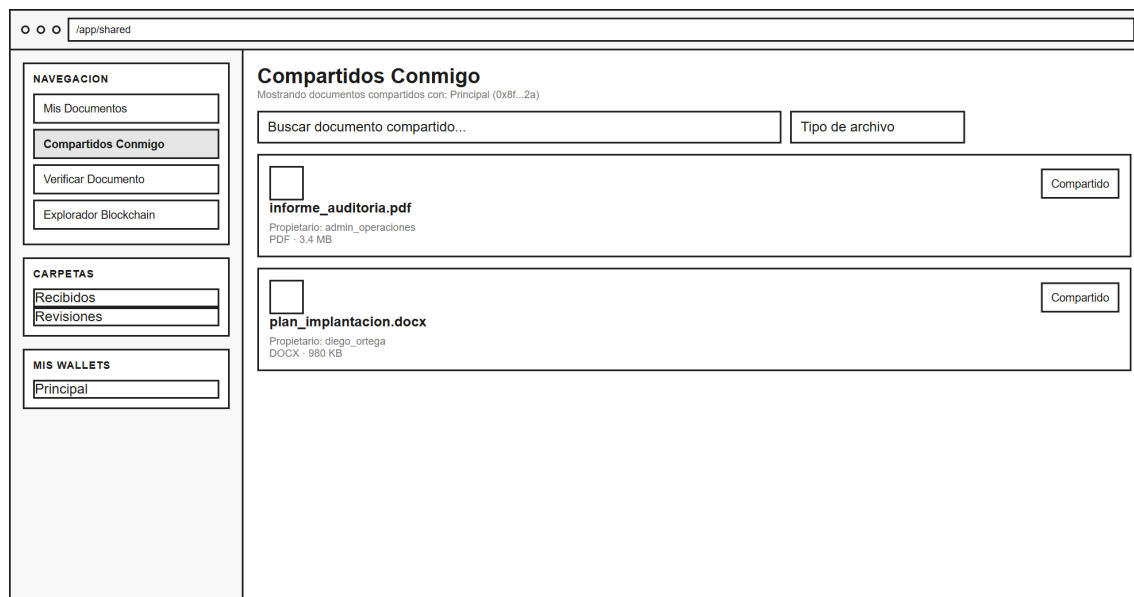


Figura 13: Boceto de la vista de documentos compartidos

Elementos:

- Misma barra lateral del usuario autenticado que en el resto de vistas principales.
- Indicación de la wallet activa desde la que se muestran los documentos recibidos.
- Búsqueda y filtro por tipo para reducir el conjunto compartido.
- Tarjetas compactas con nombre, extensión, propietario y condición de recurso compartido.

5.2.7. Vista de administración (Solo Admin)

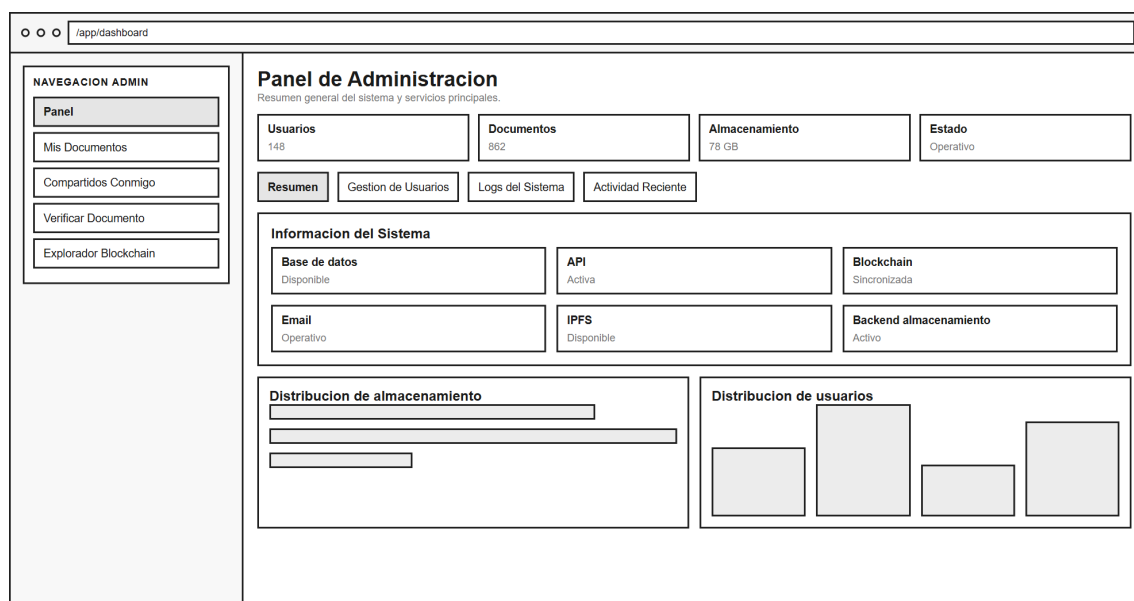
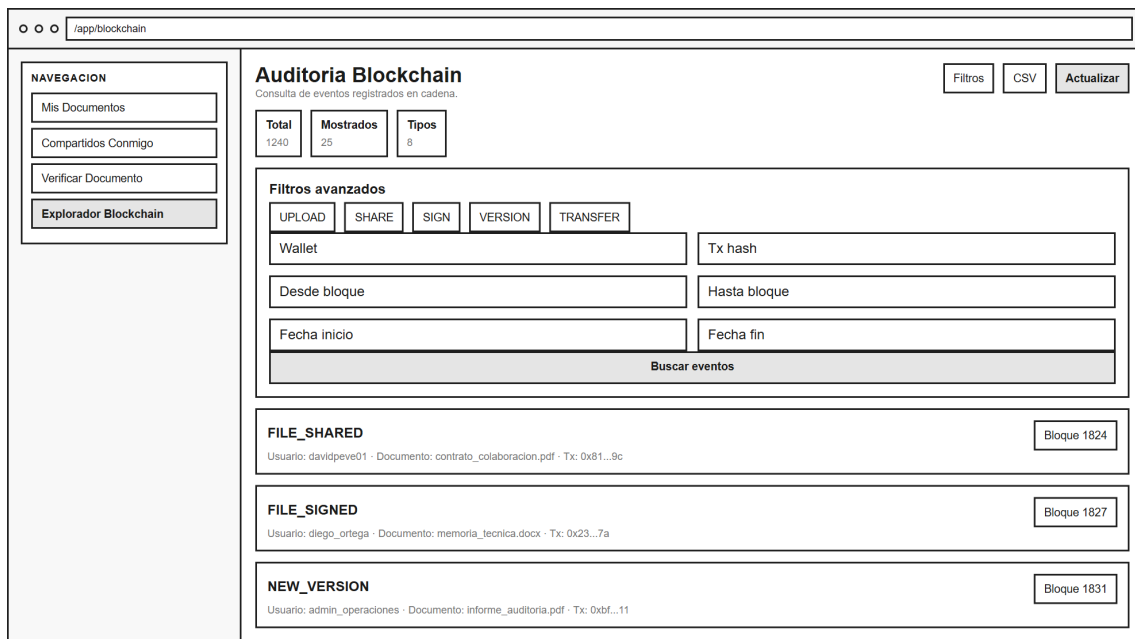


Figura 14: Boceto del panel de administración

Elementos:

- Tarjetas de resumen con usuarios, documentos, almacenamiento y estado general.
- Paneles de salud de servicios para base de datos, API, IPFS y blockchain.
- Pestañas separadas para resumen, gestión de usuarios, logs del sistema y actividad reciente.
- Distribución de almacenamiento y usuarios sin recurrir a gráficas ficticias no implementadas.

5.2.8. Vista de auditoría blockchain



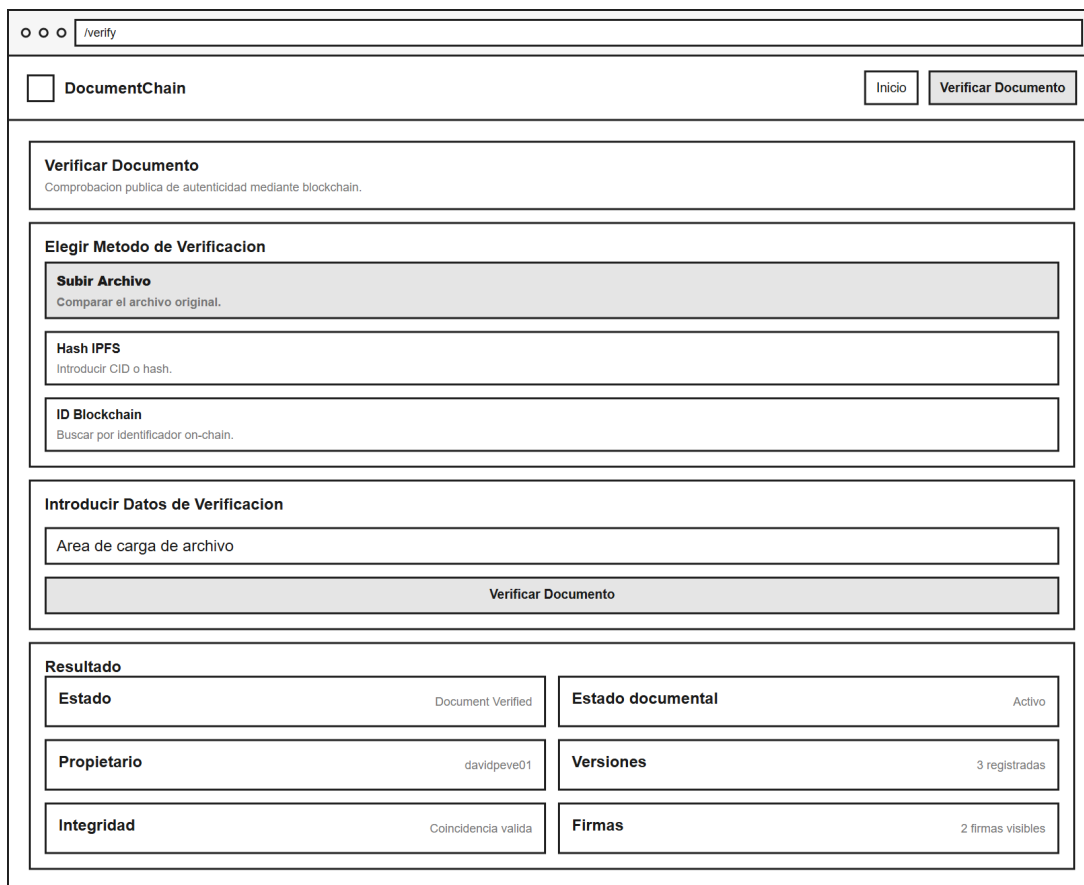
The wireframe shows a web application interface for 'Auditoria Blockchain'. It includes a navigation sidebar on the left with options like 'Mis Documentos', 'Compartidos Conmigo', 'Verificar Documento', and 'Explorador Blockchain'. The main content area has a header with 'Auditoria Blockchain' and a subtitle 'Consulta de eventos registrados en cadena.'. Below this are summary statistics: 'Total' (1240), 'Mostrados' (25), and 'Tipos' (8). There are buttons for 'Filtros', 'CSV', and 'Actualizar'. A section for 'Filtros avanzados' contains filters for 'UPLOAD', 'SHARE', 'SIGN', 'VERSION', and 'TRANSFER'. Below these are input fields for 'Wallet', 'Tx hash', 'Desde bloque', 'Hasta bloque', 'Fecha inicio', and 'Fecha fin'. A 'Buscar eventos' button is at the bottom of the filter section. The main list displays three items: 'FILE_SHARED' (Bloque 1824), 'FILE_SIGNED' (Bloque 1827), and 'NEW_VERSION' (Bloque 1831). Each item shows the user, document, and transaction hash.

Figura 15: Boceto de la vista de auditoría blockchain

Elementos:

- Cabecera de la vista con métricas rápidas de eventos registrados y resultados visibles.
- Acciones de filtrado, exportación y actualización del listado.
- Panel expandido de filtros avanzados por tipo de evento, wallet, transacción, bloque y rango temporal.
- Relación compacta de eventos de auditoría con usuario, documento, bloque y tx hash abreviado.

5.2.9. Vista de verificación pública



DocumentChain Inicio Verificar Documento

Verificar Documento
Comprobacion publica de autenticidad mediante blockchain.

Elegir Metodo de Verificacion

Subir Archivo
Comparar el archivo original.

Hash IPFS
Introducir CID o hash.

ID Blockchain
Buscar por identificador on-chain.

Introducir Datos de Verificacion

Area de carga de archivo

Verificar Documento

Resultado

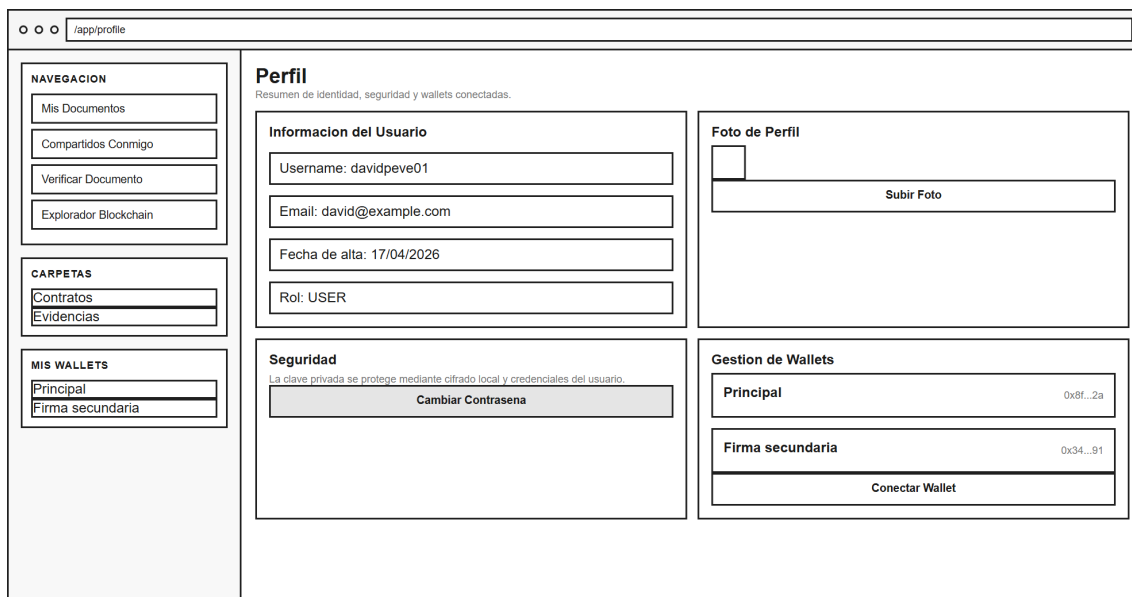
Estado	Document Verified	Estado documental	Activo
Propietario	davidpeve01	Versiones	3 registradas
Integridad	Coincidencia valida	Firmas	2 firmas visibles

Figura 16: Boceto de la vista de verificación de autenticidad

Elementos:

- Selección del método de verificación: archivo, CID IPFS o identificador blockchain.
- Formulario específico por método.
- Panel de resultados con existencia, integridad, propietario, versiones y firmas.
- Acceso público sin autenticación previa.

5.2.10. Vista de perfil de usuario



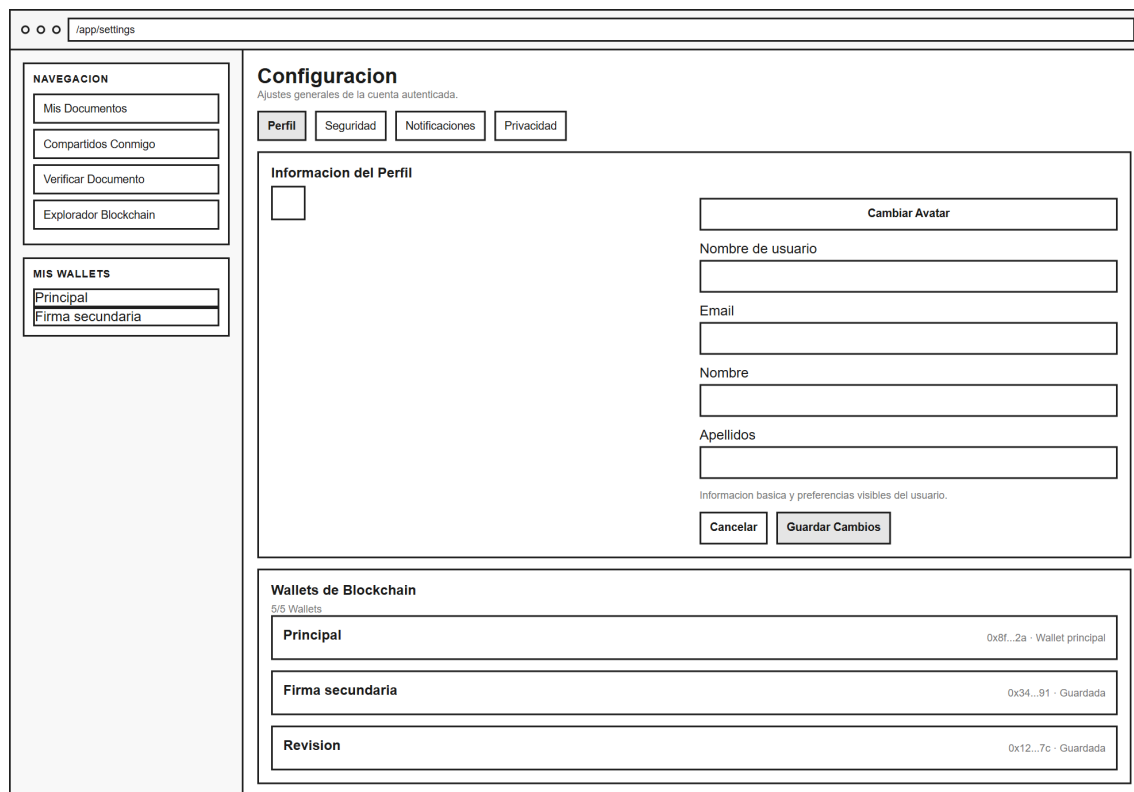
The wireframe shows a user profile page with a sidebar and a main content area. The sidebar contains three sections: 'NAVEGACION' with links for 'Mis Documentos', 'Compartidos Conmigo', 'Verificar Documento', and 'Explorador Blockchain'; 'CARPETAS' with 'Contratos' and 'Evidencias'; and 'MIS WALLETS' with 'Principal' and 'Firma secundaria'. The main content area is titled 'Perfil' and includes a subtitle 'Resumen de identidad, seguridad y wallets conectadas.' It is divided into four panels: 'Informacion del Usuario' (Username: davidpeve01, Email: david@example.com, Fecha de alta: 17/04/2026, Rol: USER), 'Foto de Perfil' (with a 'Subir Foto' button), 'Seguridad' (with a 'Cambiar Contraseña' button and a note about private key protection), and 'Gestion de Wallets' (showing 'Principal' and 'Firma secundaria' wallets with their respective addresses and a 'Conectar Wallet' button).

Figura 17: Boceto de la vista de perfil de usuario

Elementos:

- Bloques separados para identidad del usuario, avatar, seguridad y wallets enlazadas.
- Resumen de username, email, fecha de alta y rol actual.
- Acciones principales de actualización de foto y cambio de contraseña.
- Área de gestión simplificada de wallets conectadas al perfil.

5.2.11. Vista de configuración



The wireframe shows a web application interface for user configuration. It features a sidebar on the left with two sections: 'NAVEGACION' (Navigation) containing links for 'Mis Documentos', 'Compartidos Conmigo', 'Verificar Documento', and 'Explorador Blockchain'; and 'MIS WALLETS' containing 'Principal' and 'Firma secundaria'. The main content area is titled 'Configuracion' and includes tabs for 'Perfil', 'Seguridad', 'Notificaciones', and 'Privacidad'. The 'Perfil' tab is active, showing a form for 'Informacion del Perfil' with fields for 'Nombre de usuario', 'Email', 'Nombre', and 'Apellidos', along with an 'Avatar' placeholder and a 'Cambiar Avatar' button. Below this is a section for 'Wallets de Blockchain' showing a list of wallets: 'Principal' (0x8f...2a - Wallet principal), 'Firma secundaria' (0x34...91 - Guardada), and 'Revision' (0x12...7c - Guardada). The interface is clean and modern, using a light gray color scheme.

Figura 18: Boceto de la vista de configuración

Elementos:

- Cabecera de configuración con pestañas superiores para perfil y seguridad y cuenta; las preferencias de notificación se gestionan dentro de la pestaña de seguridad.
- Formulario principal de información de perfil con avatar, datos de cuenta y acciones de guardado.
- Bloque diferenciado de wallets de blockchain con recuento de límite y filas de wallets registradas.
- Reutilización de la misma navegación lateral del usuario autenticado para mantener consistencia estructural.

Con esta regeneración, la selección de bocetos vuelve a cubrir las principales vistas estructurales del frontend implementado, mientras que el Anexo V sigue reservándose para las capturas finales de alta fidelidad y para los estados operativos más detallados. La cobertura visual se considera suficiente para explicar las vistas esenciales del sistema, por lo que no se incorporan más bocetos específicos en esta versión del anexo.

6. Modelo de analisis y diseño procedimental

En este apartado se presentan los diagramas de secuencia de analisis y diseno que modelan las interacciones del sistema.

6.1. Clases de análisis (BCE)

El diagrama de clases de análisis de la Figura 19 muestra las clases del sistema estereotipadas según el patrón Boundary-Control-Entity. La Tabla 15 presenta la clasificación BCE para los siete paquetes funcionales del catálogo consolidado de 43 casos de uso.

Paquete	Clases Boundary	Clases Control / Entity
Acceso y Cuenta (UC-0001-0012)	LoginPage, RegisterPage, SettingsPage, WalletSelector	AuthController, UserService, TokenService, WalletService
Documentos (UC-0013-0021)	DocumentsPage, DocumentDetail, UploadModal, TransferModal	DocumentController, DocumentService, IPFSService
Versiones (UC-0022-0026)	VersionList, UploadVersionModal, VersionDetail	VersionController, VersionService
Firmas (UC-0027-0029)	SignatureList, SignModal	SignatureController, SignatureService
Comparticion (UC-0030-0032)	ShareModal, SharedWithMePage	ShareController, ShareService
Auditoria (UC-0033-0037)	AuditPage, BlockchainAuditor, VerifyPage, TimelinePage	AuditService, VerificationService, DocumentTimelineService
Administracion (UC-0038-0039)	AdminPanel, AdminDashboard, AdminUsersTab	AdminController, BlockchainAdminService

Cuadro 15: Clasificación Boundary-Control-Entity por paquete funcional

El diagrama de la Figura 19 complementa esta tabla mostrando visualmente las relaciones entre las clases de análisis. Las clases *boundary* representan las pantallas y modales de la interfaz de usuario. Las clases *control* coordinan las operaciones sin conocer detalles de persistencia ni de interfaz. Las clases *entity* representan las entidades del dominio definidas en los requisitos de información IRQ-0001 a IRQ-0007.

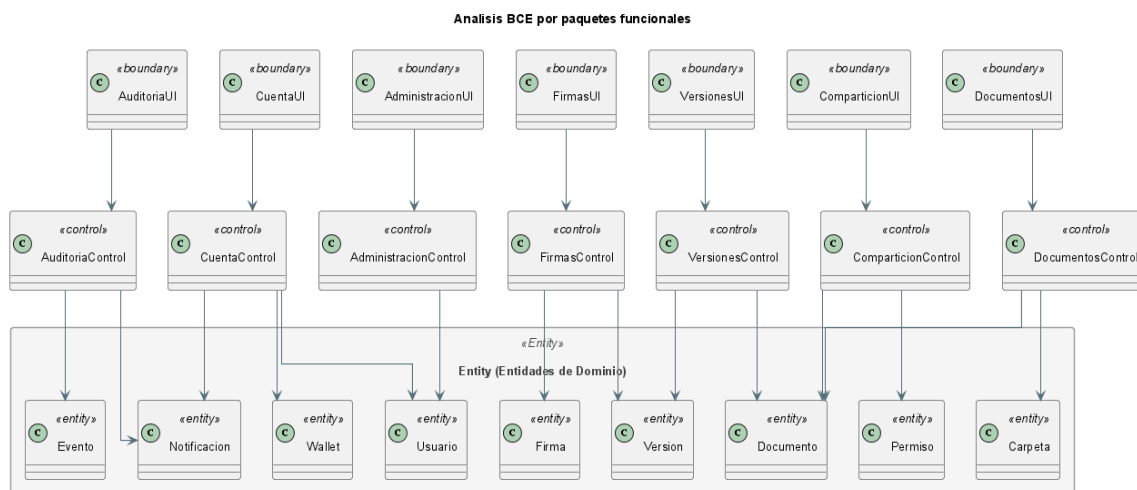


Figura 19: Diagrama de clases de análisis con estereotipos BCE

6.2. Diagramas de secuencia de análisis (patrón BCE)

Los diagramas de secuencia de análisis modelan cómo interactúan los actores con los objetos del dominio usando el patrón BCE (Boundary-Control-Entity). Son la primera aproximación a las interacciones del sistema, antes de asignar componentes tecnológicos concretos. Sirven sobre

todo para comprobar que la arquitectura conceptual cubre todos los requisitos funcionales del Anexo I.

Convenciones: los estereotipos **boundary** (naranja) son la interfaz de usuario; **control** (verde) son los controladores que coordinan cada operación; y **entity** (morado) son las entidades del dominio.

6.2.1. Análisis BCE: UC-0001 Registrar Usuario

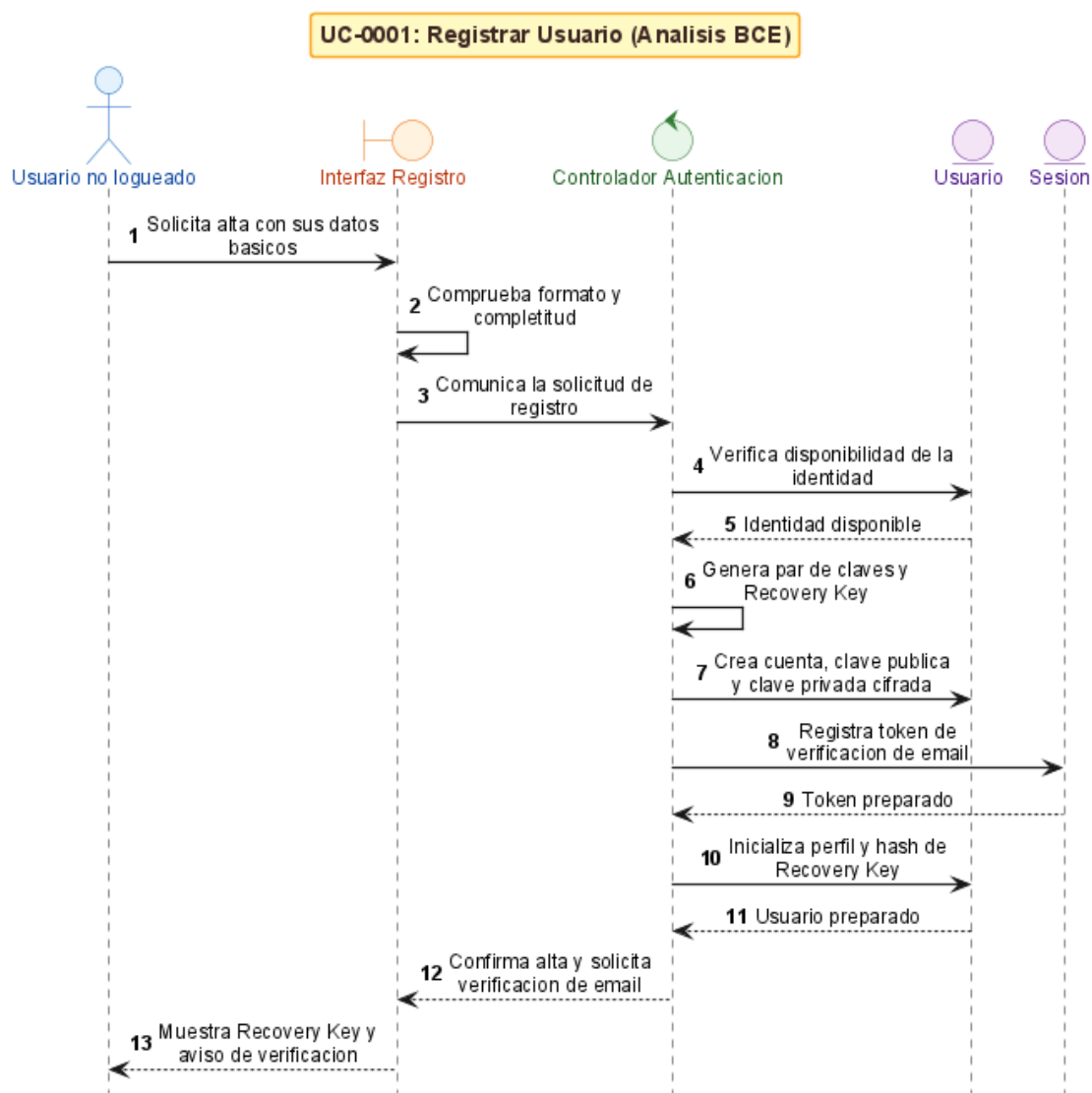


Figura 20: Diagrama de secuencia de análisis BCE – UC-0001: Registrar Usuario

6.2.2. Análisis BCE: UC-0002 Iniciar Sesión

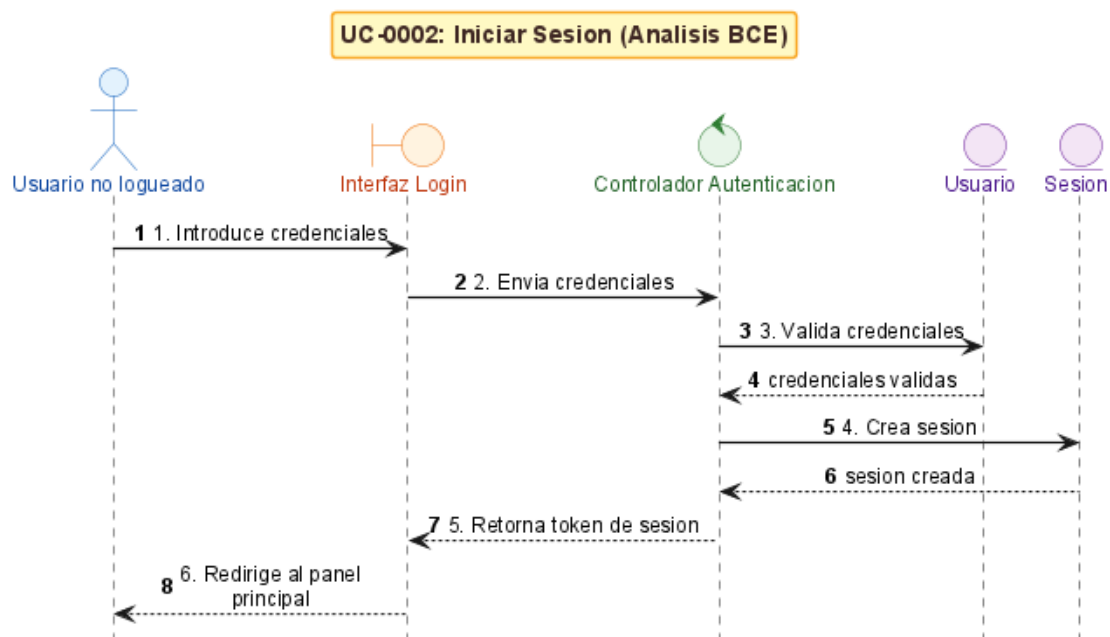


Figura 21: Diagrama de secuencia de análisis BCE – UC-0002: Iniciar Sesión

6.2.3. Análisis BCE: UC-0003 Cerrar Sesión

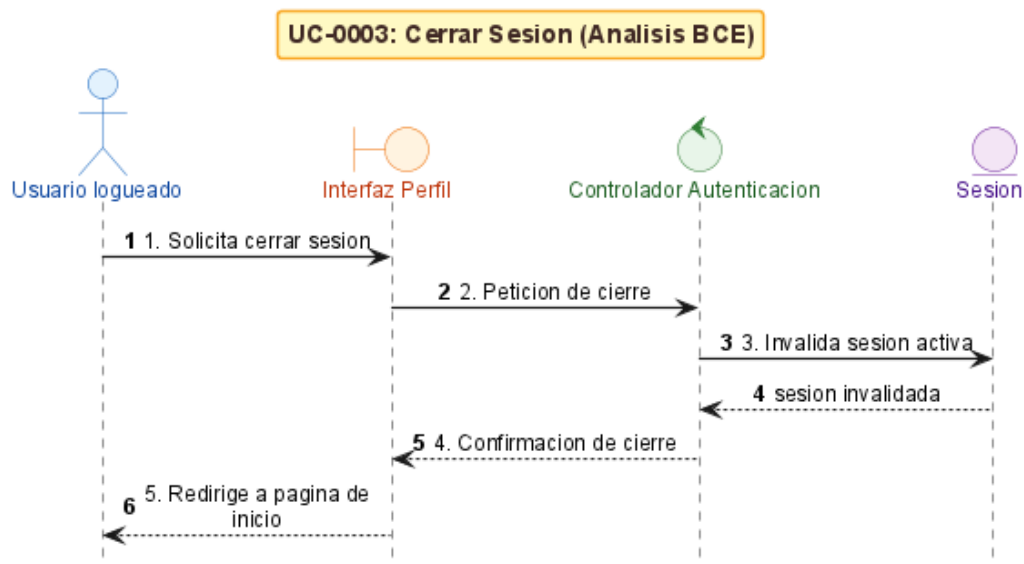


Figura 22: Diagrama de secuencia de análisis BCE – UC-0003: Cerrar Sesión

6.2.4. Análisis BCE: UC-0004 Conectar Wallet

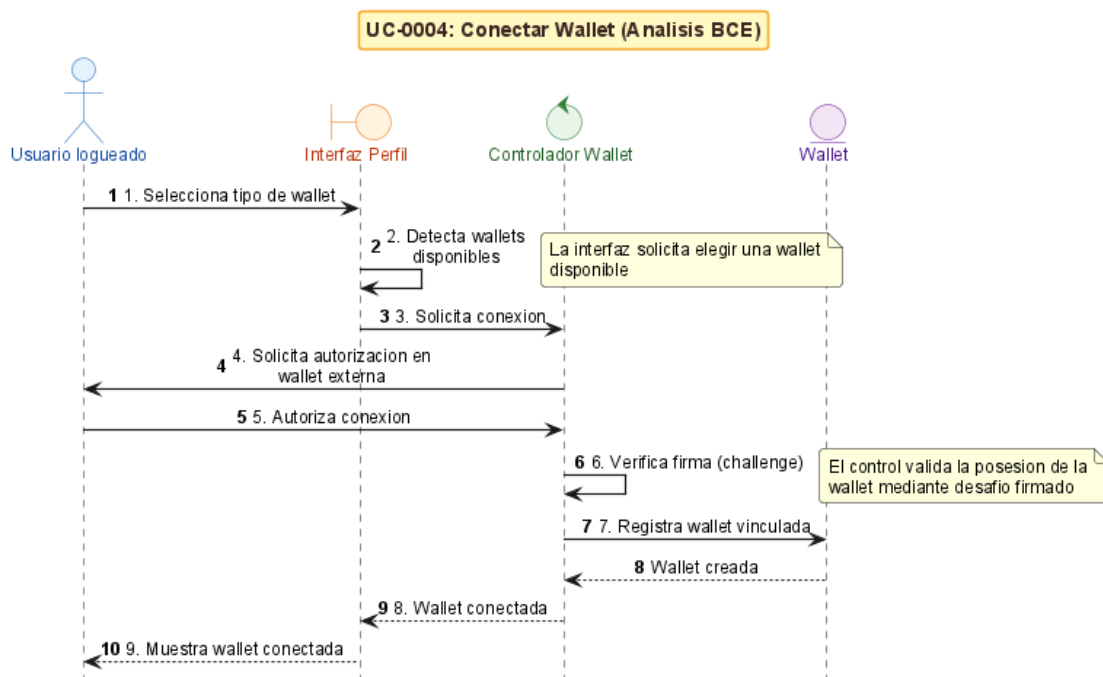


Figura 23: Diagrama de secuencia de análisis BCE – UC-0004: Conectar Wallet

6.2.5. Análisis BCE: UC-0005 Eliminar Wallet

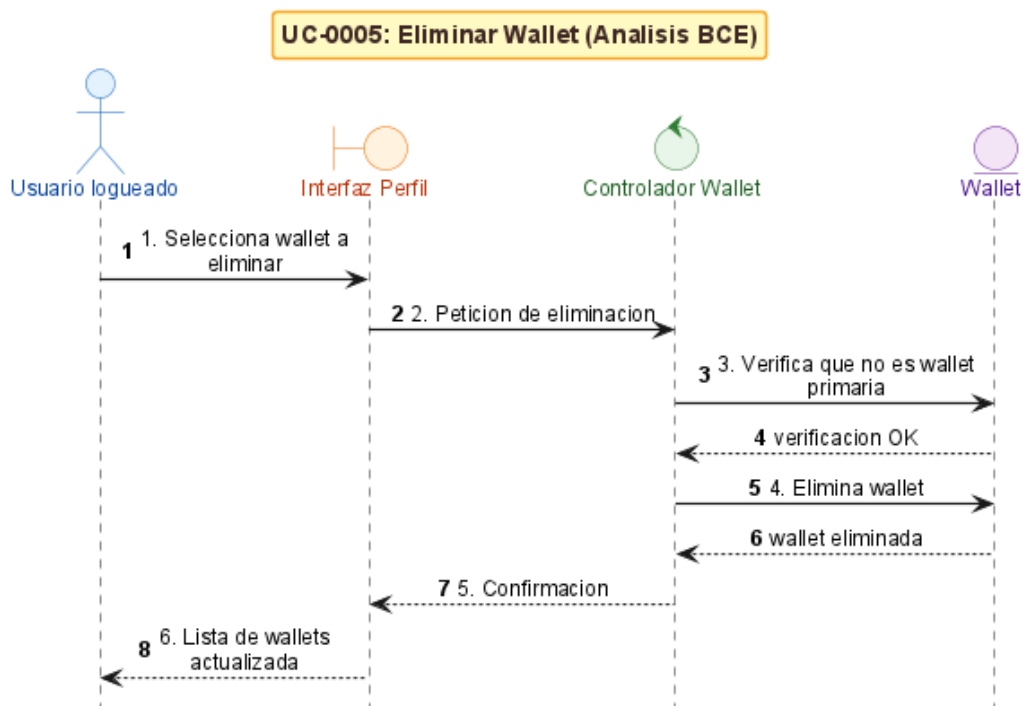


Figura 24: Diagrama de secuencia de análisis BCE – UC-0005: Eliminar Wallet

6.2.6. Análisis BCE: UC-0006 Establecer Wallet Principal

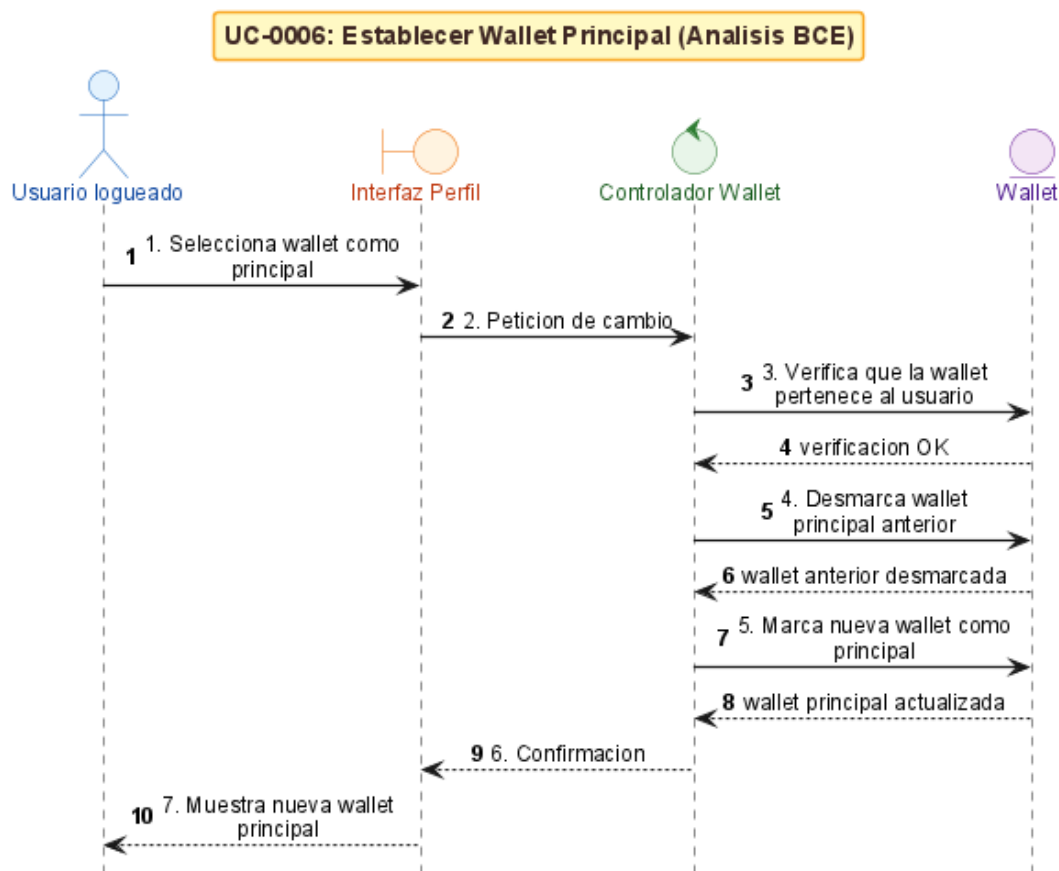


Figura 25: Diagrama de secuencia de análisis BCE – UC-0006: Establecer Wallet Principal

6.2.7. Análisis BCE: UC-0007 Renombrar Wallet

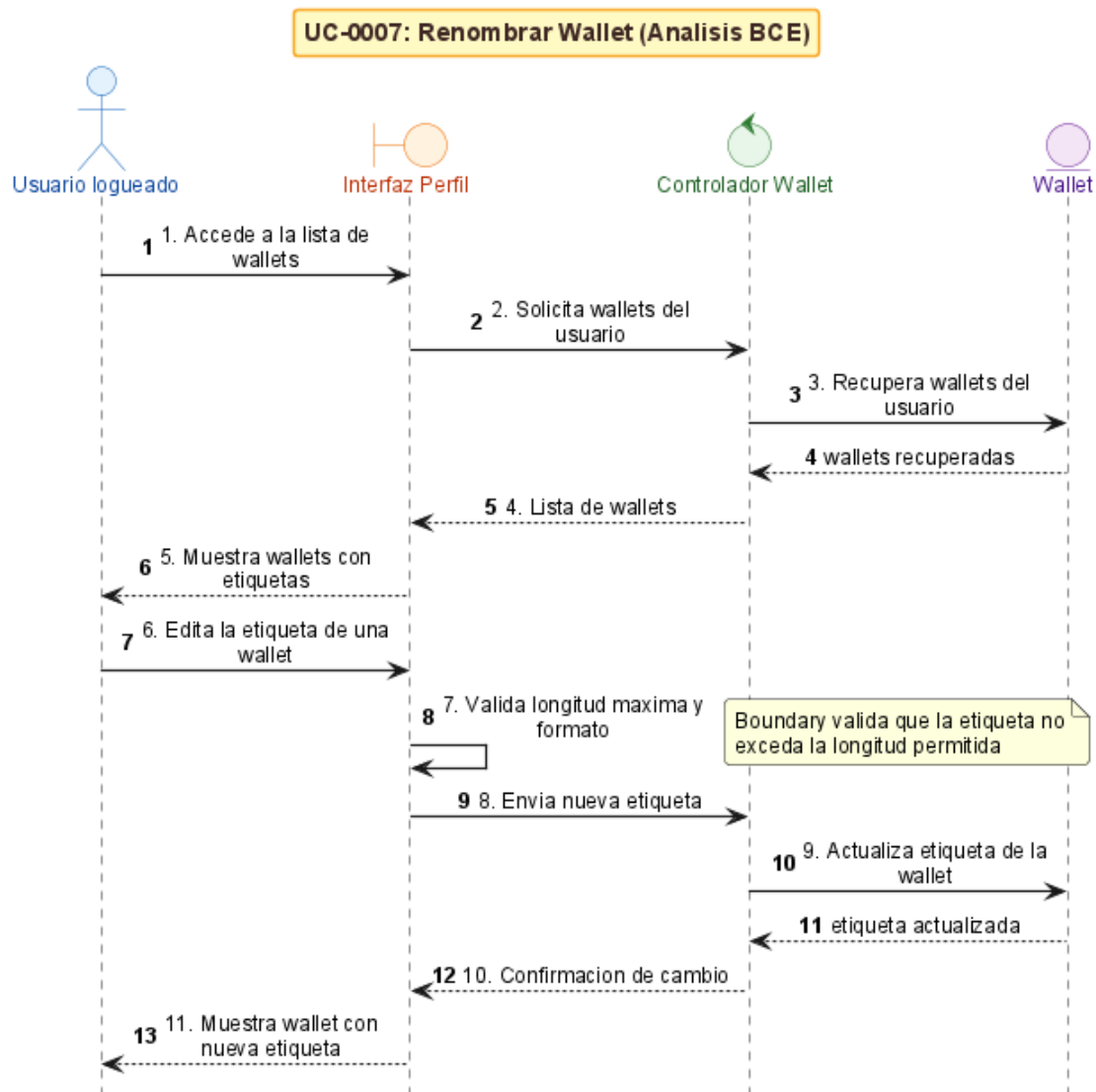


Figura 26: Diagrama de secuencia de análisis BCE – UC-0007: Renombrar Wallet

6.2.8. Análisis BCE: UC-0008 Gestionar Perfil

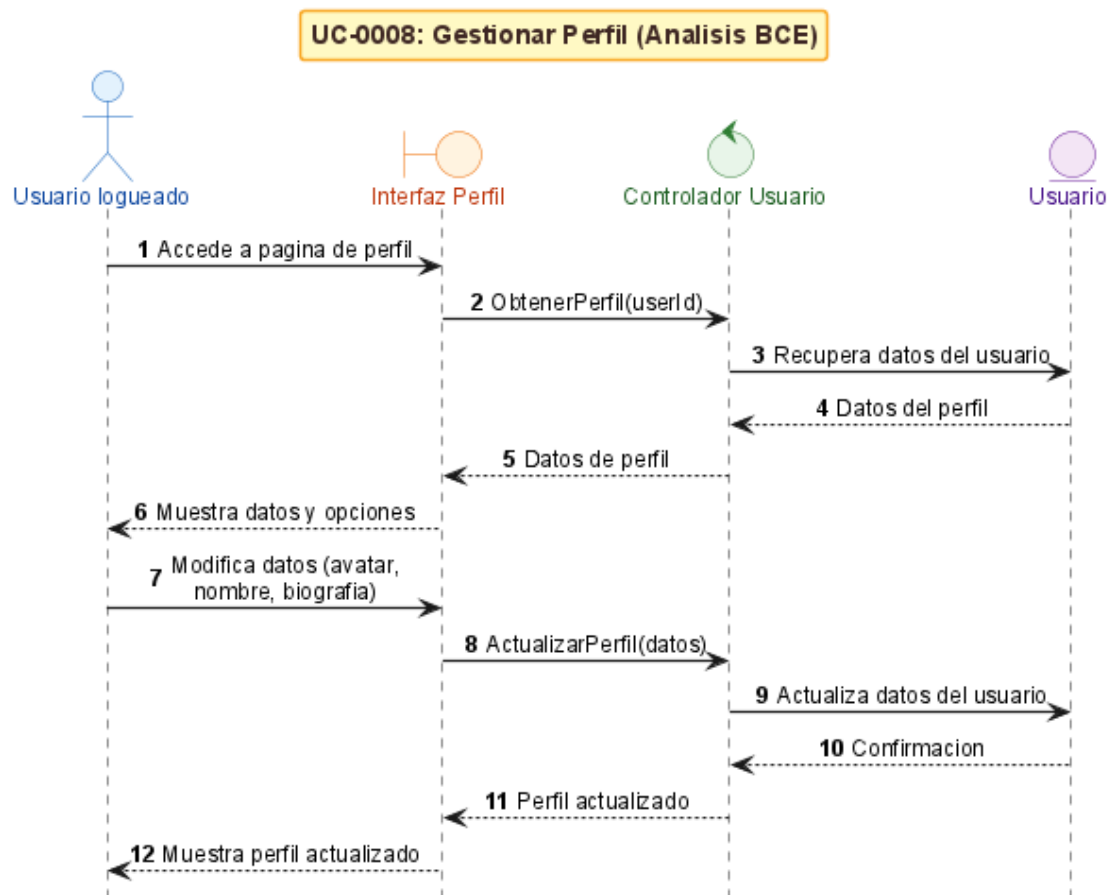


Figura 27: Diagrama de secuencia de análisis BCE – UC-0008: Gestionar Perfil

6.2.9. Análisis BCE: UC-0009 Eliminar Cuenta

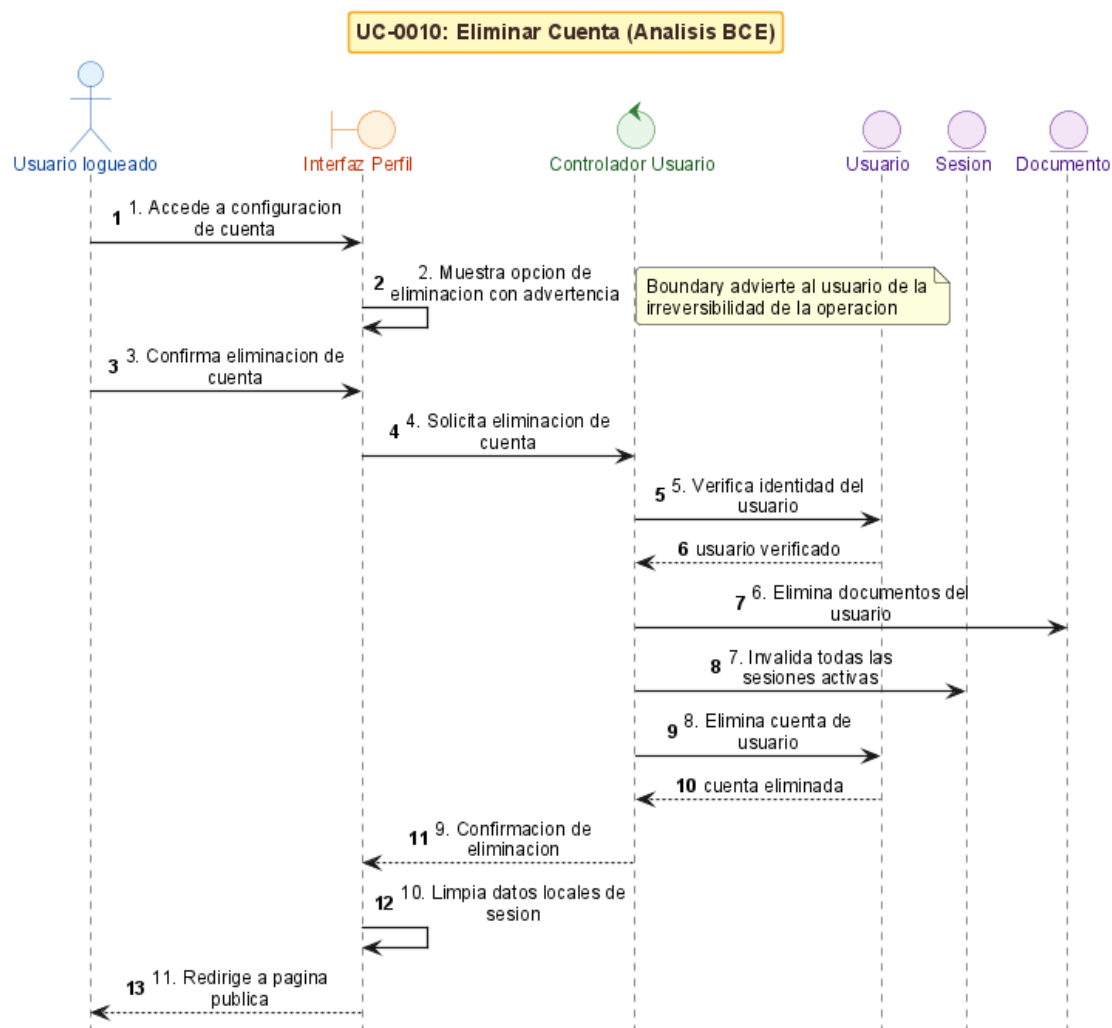


Figura 28: Diagrama de secuencia de análisis BCE – UC-0009: Eliminar Cuenta

6.2.10. Análisis BCE: UC-0010 Cambiar Contraseña

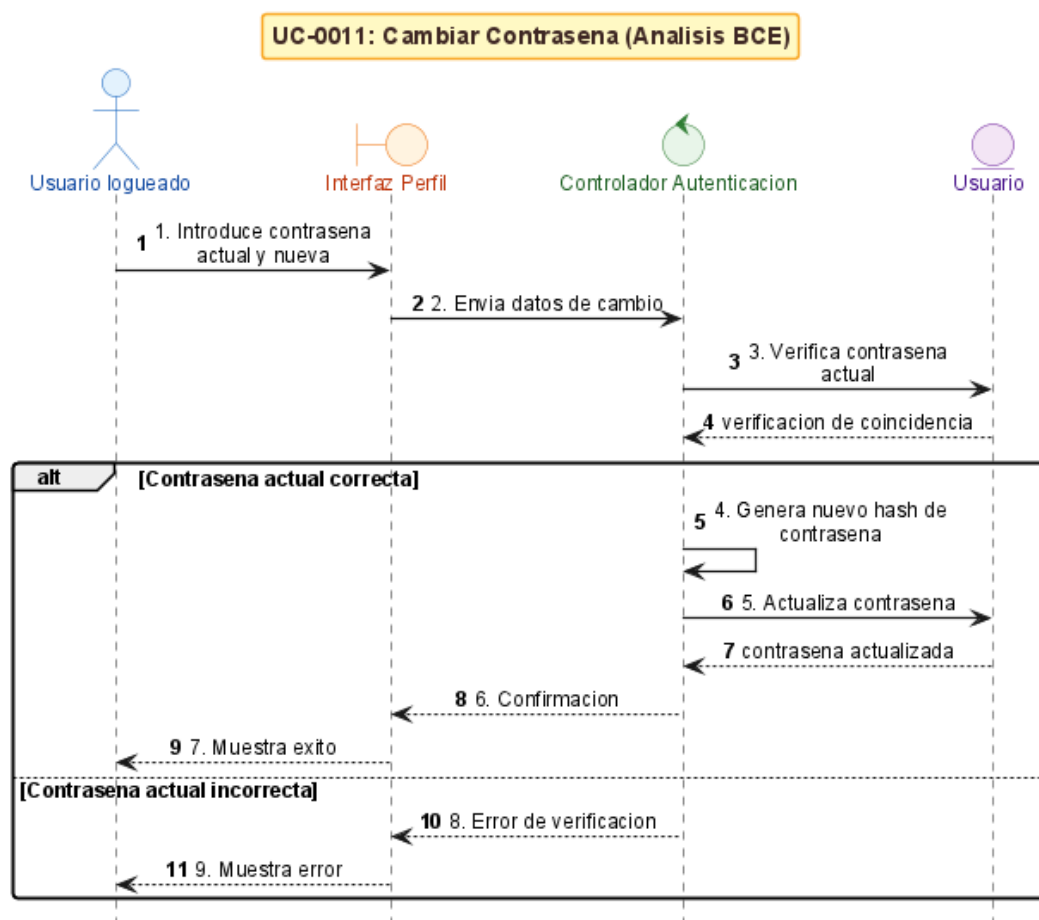


Figura 29: Diagrama de secuencia de análisis BCE – UC-0010: Cambiar Contraseña

6.2.11. Análisis BCE: UC-0011 Verificar Email

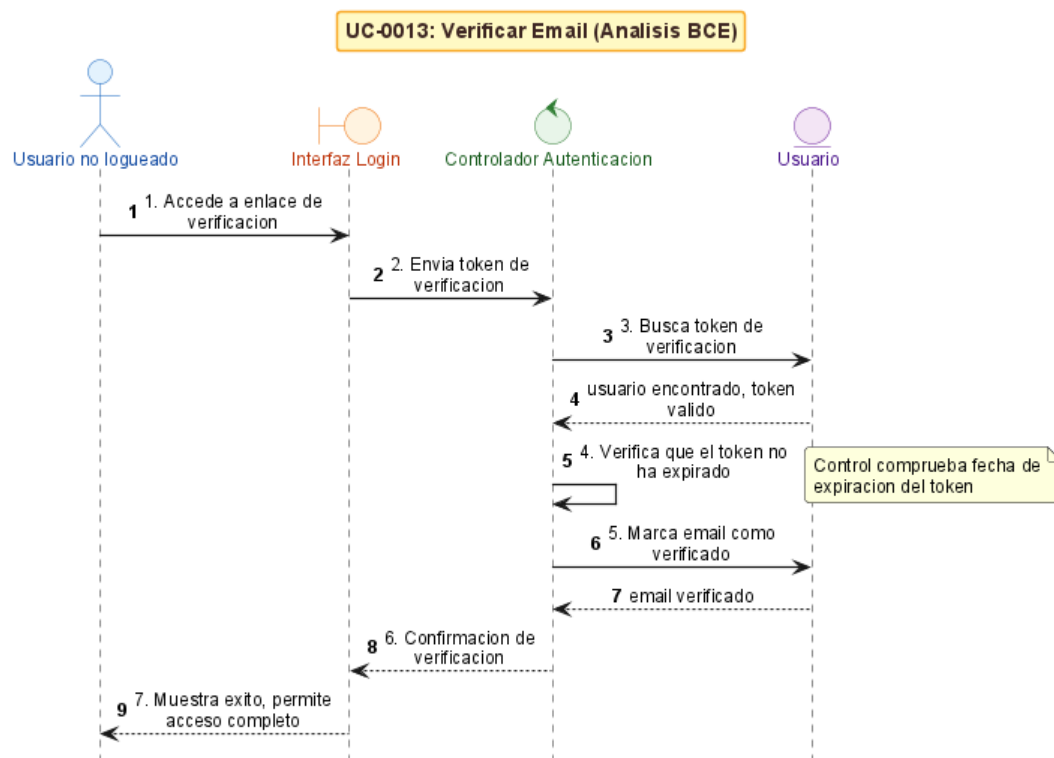


Figura 30: Diagrama de secuencia de análisis BCE – UC-0011: Verificar Email

6.2.12. Análisis BCE: UC-0012 Reenviar Verificación de Email

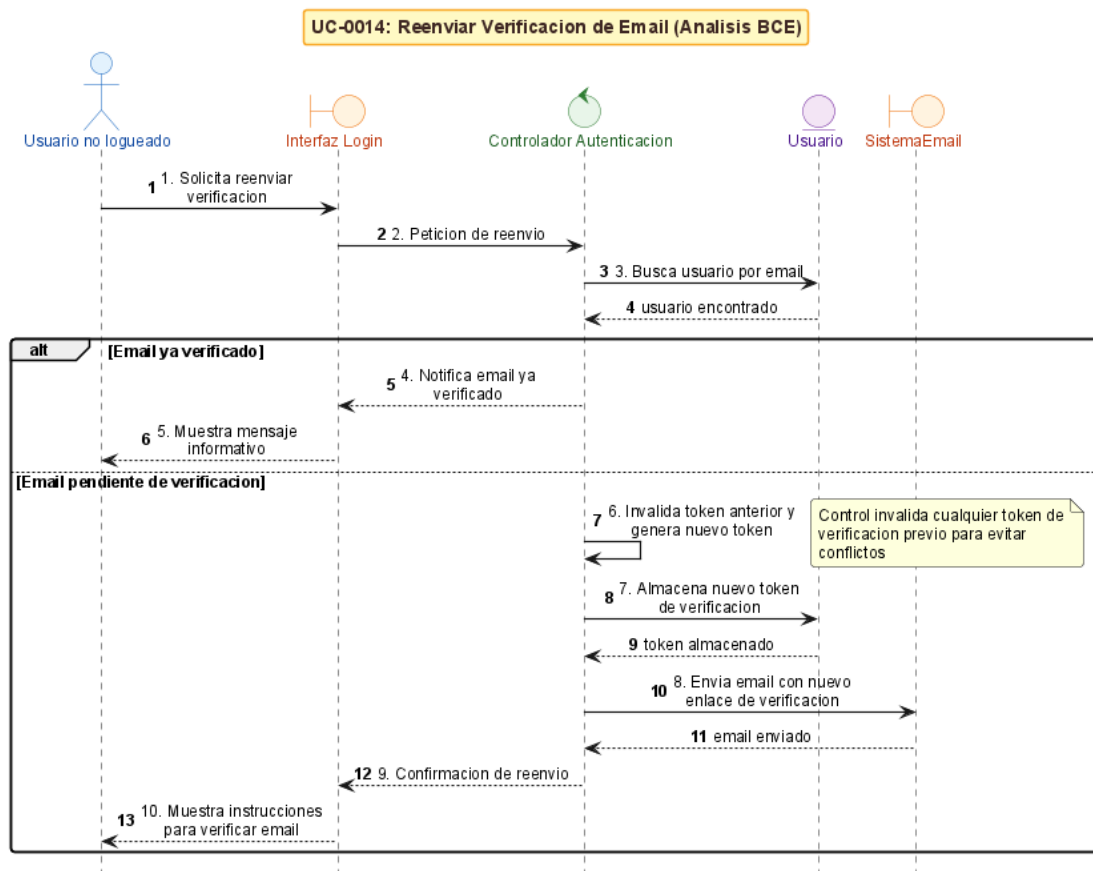


Figura 31: Diagrama de secuencia de análisis BCE – UC-0012: Reenviar Verificación de Email

6.2.13. Análisis BCE: UC-0013 Subir Documento

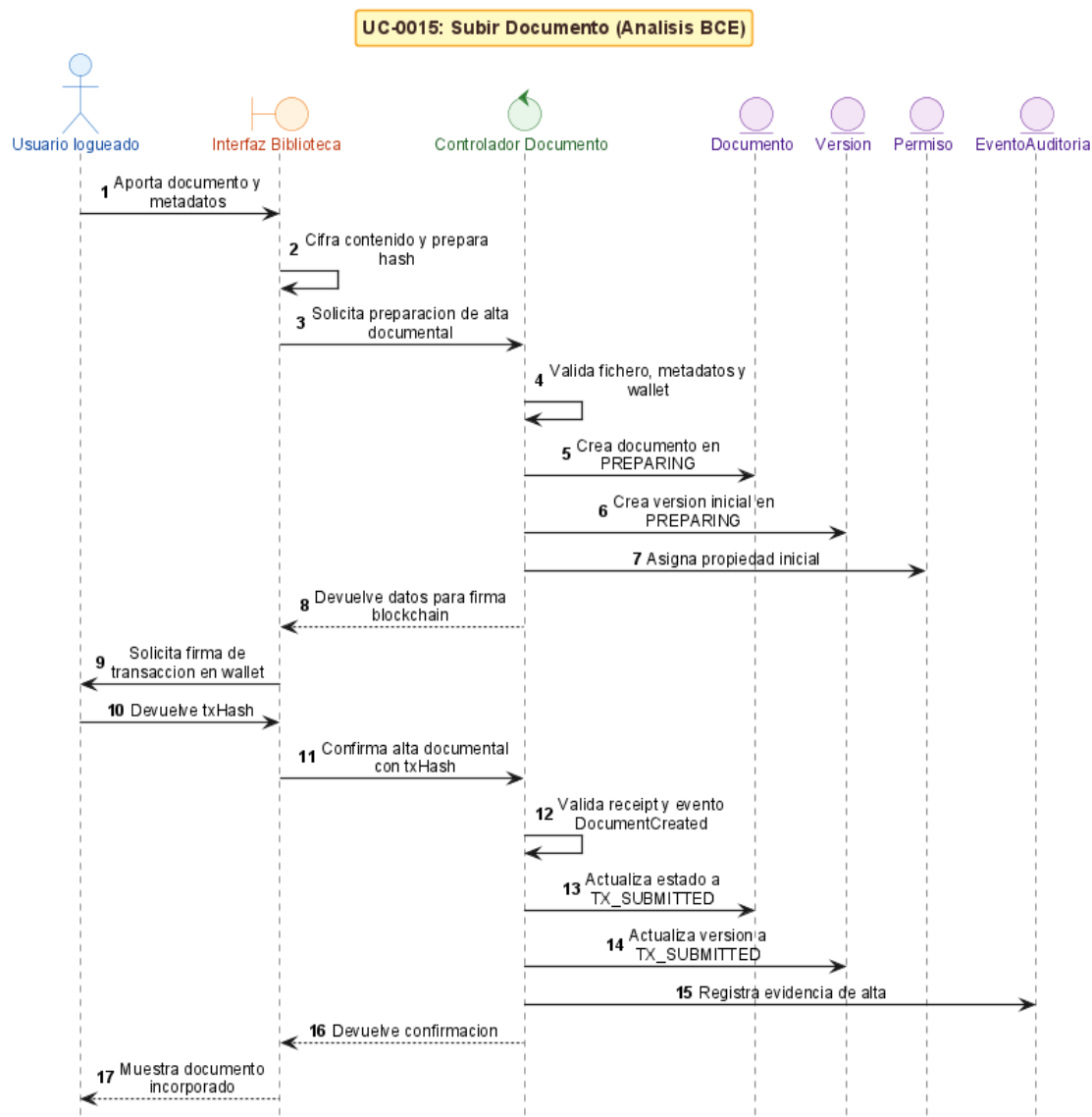


Figura 32: Diagrama de secuencia de análisis BCE – UC-0013: Subir Documento

6.2.14. Análisis BCE: UC-0014 Listar Documentos

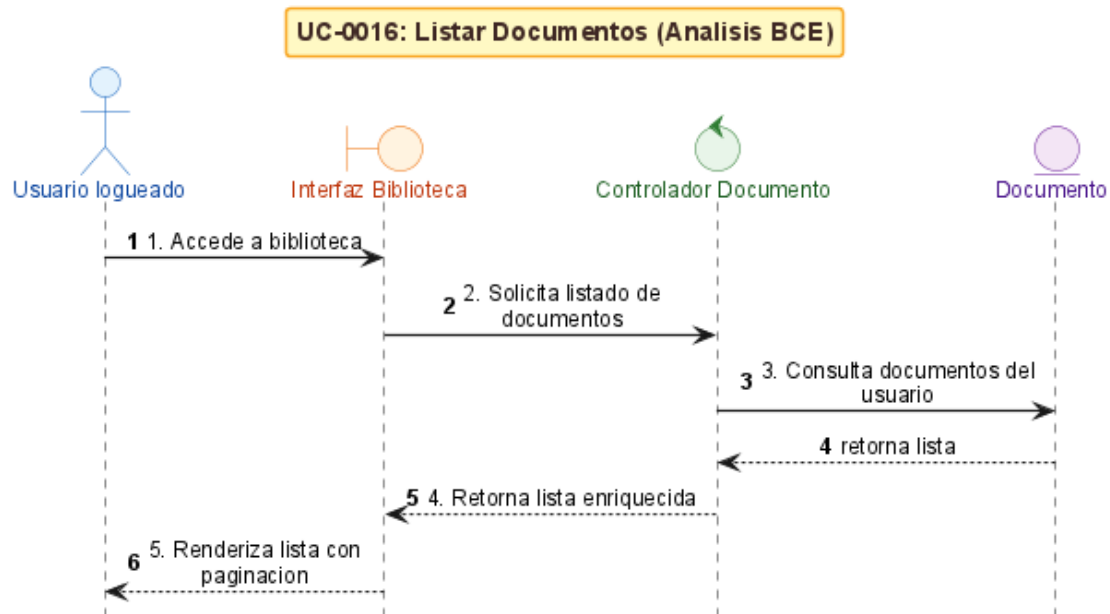


Figura 33: Diagrama de secuencia de análisis BCE – UC-0014: Listar Documentos

6.2.15. Análisis BCE: UC-0015 Ver Detalle de Documento

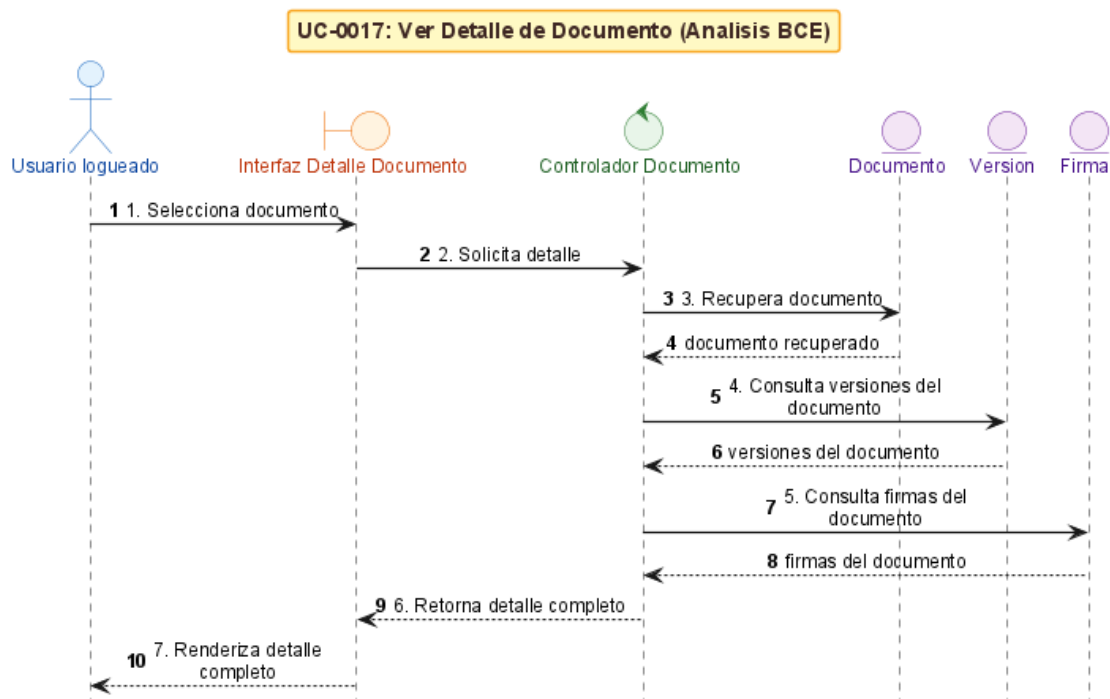


Figura 34: Diagrama de secuencia de análisis BCE – UC-0015: Ver Detalle de Documento

6.2.16. Análisis BCE: UC-0016 Descargar Documento

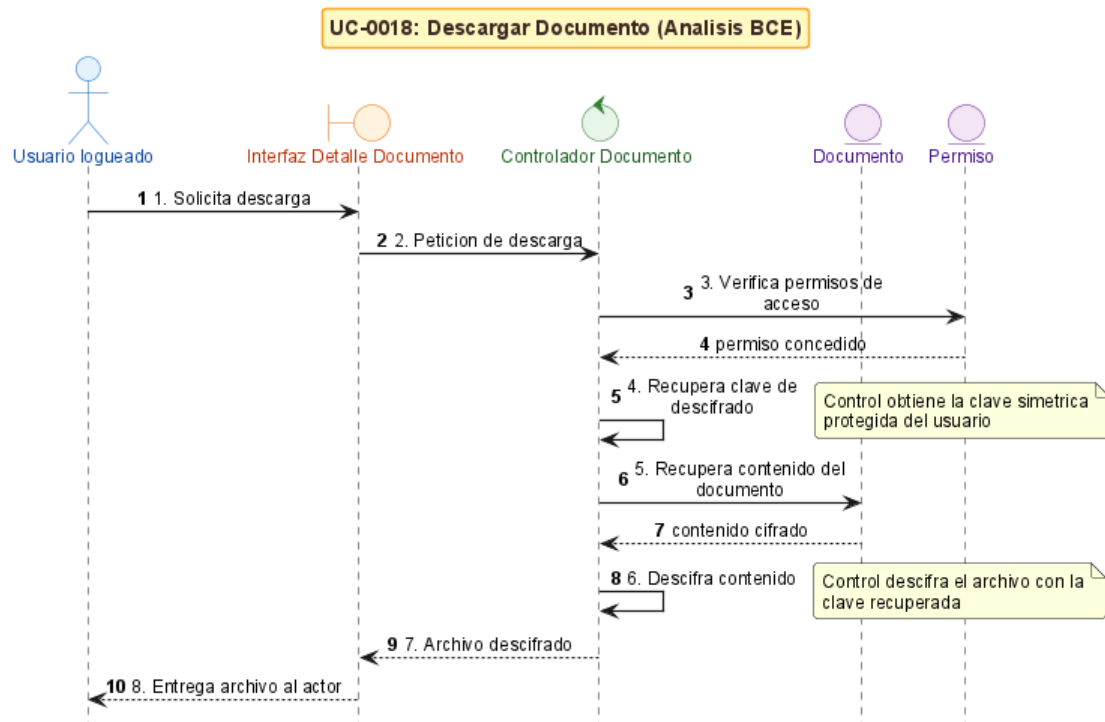


Figura 35: Diagrama de secuencia de análisis BCE – UC-0016: Descargar Documento

6.2.17. Análisis BCE: UC-0017 Archivar Documento

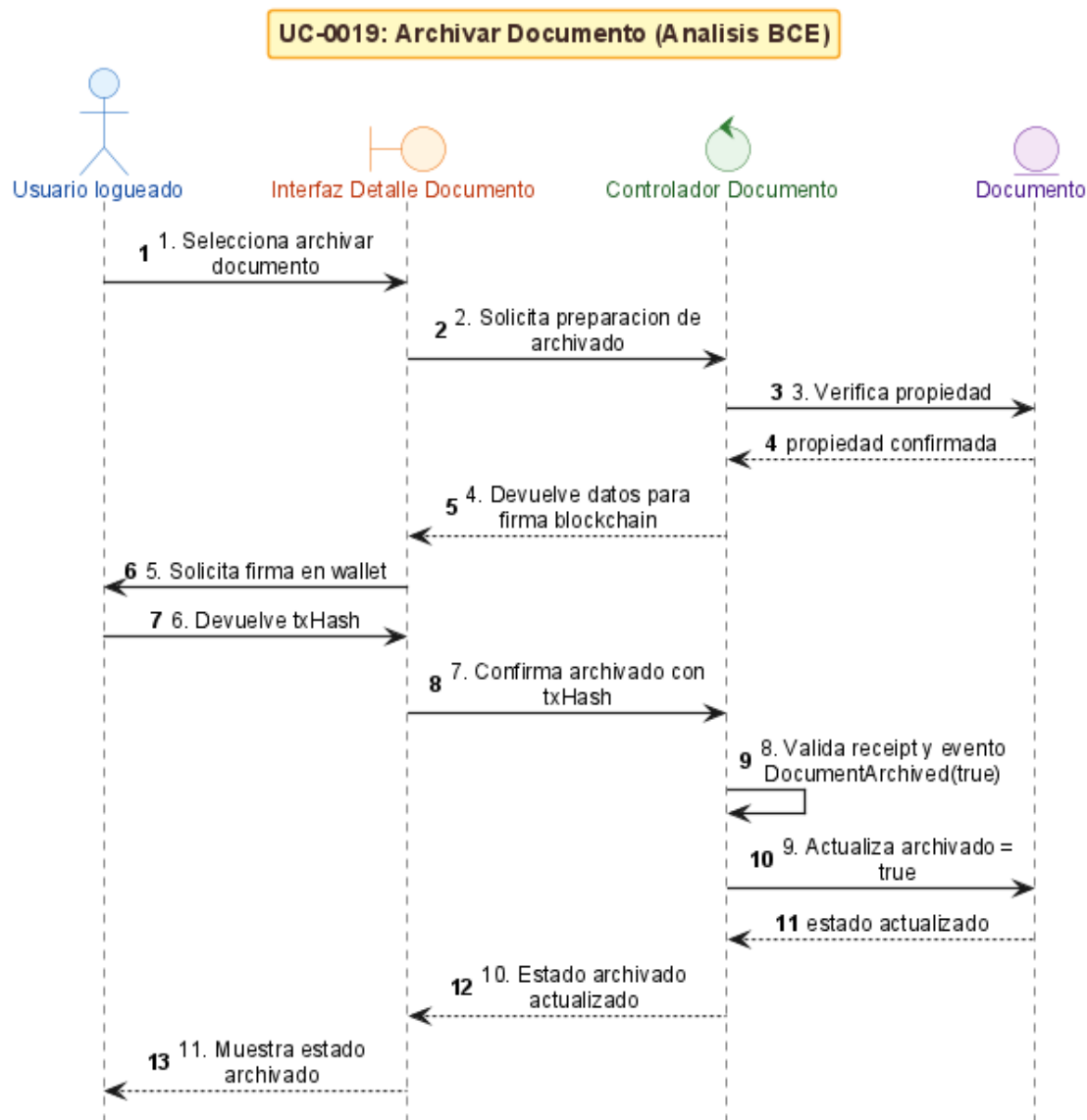


Figura 36: Diagrama de secuencia de análisis BCE – UC-0017: Archivar Documento

6.2.18. Análisis BCE: UC-0018 Desarchivar Documento

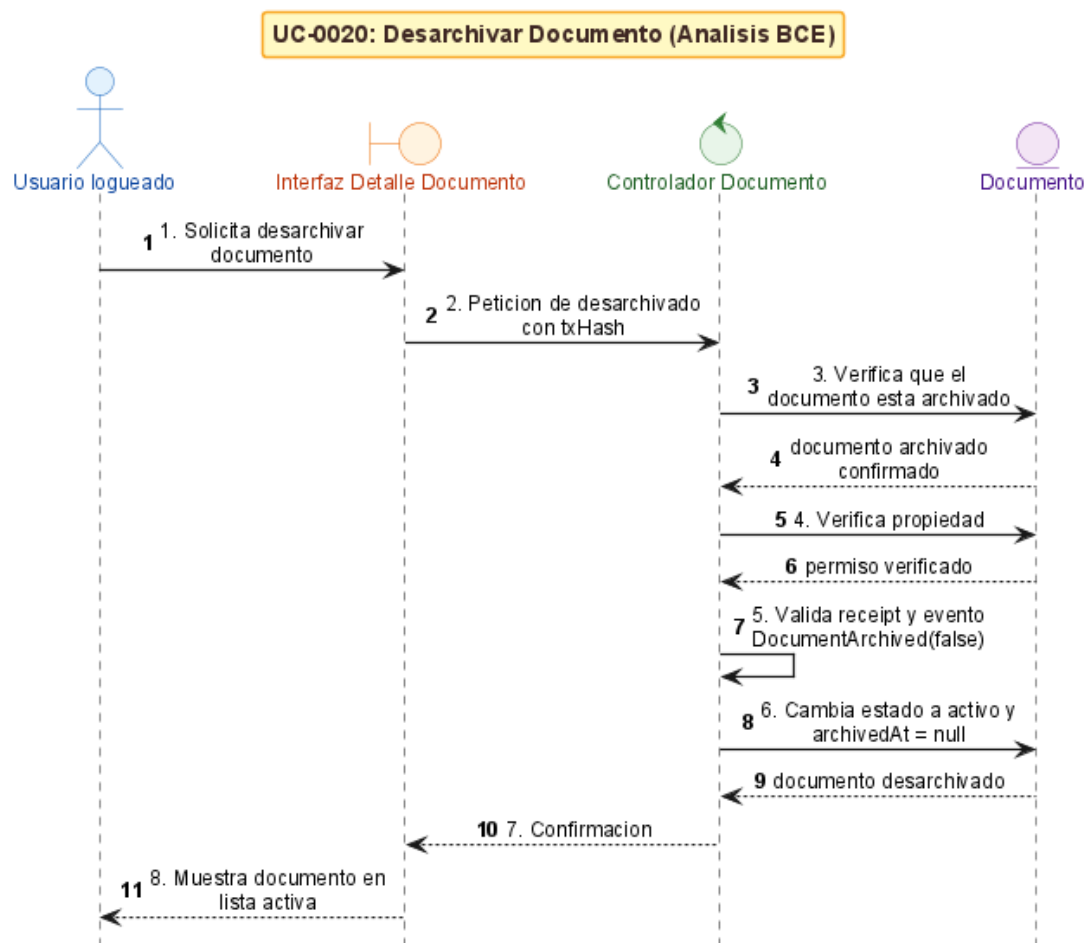


Figura 37: Diagrama de secuencia de análisis BCE – UC-0018: Desarchivar Documento

6.2.19. Análisis BCE: UC-0019 Eliminar Documento

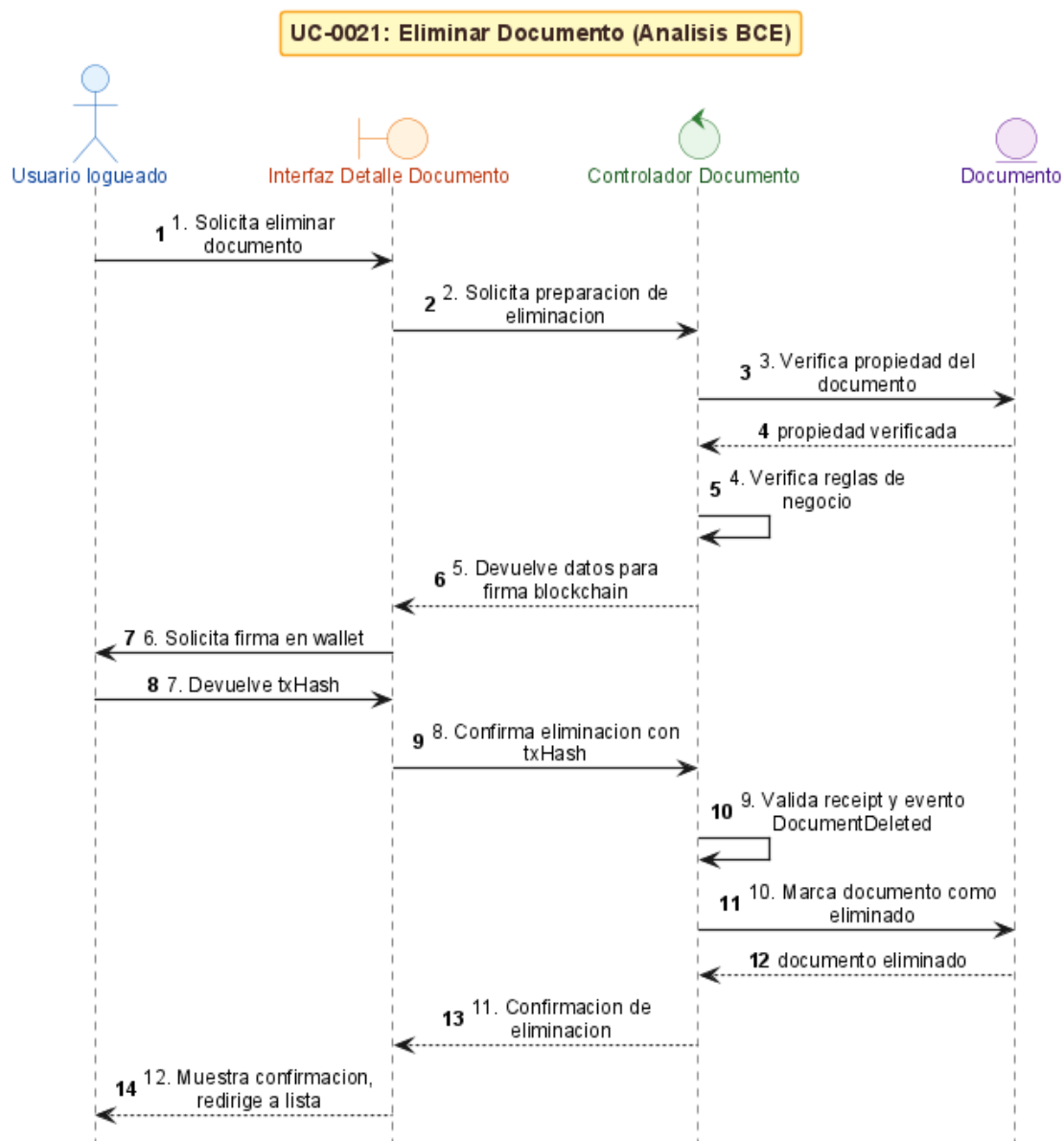


Figura 38: Diagrama de secuencia de análisis BCE – UC-0019: Eliminar Documento

6.2.20. Análisis BCE: UC-0020 Transferir Documento

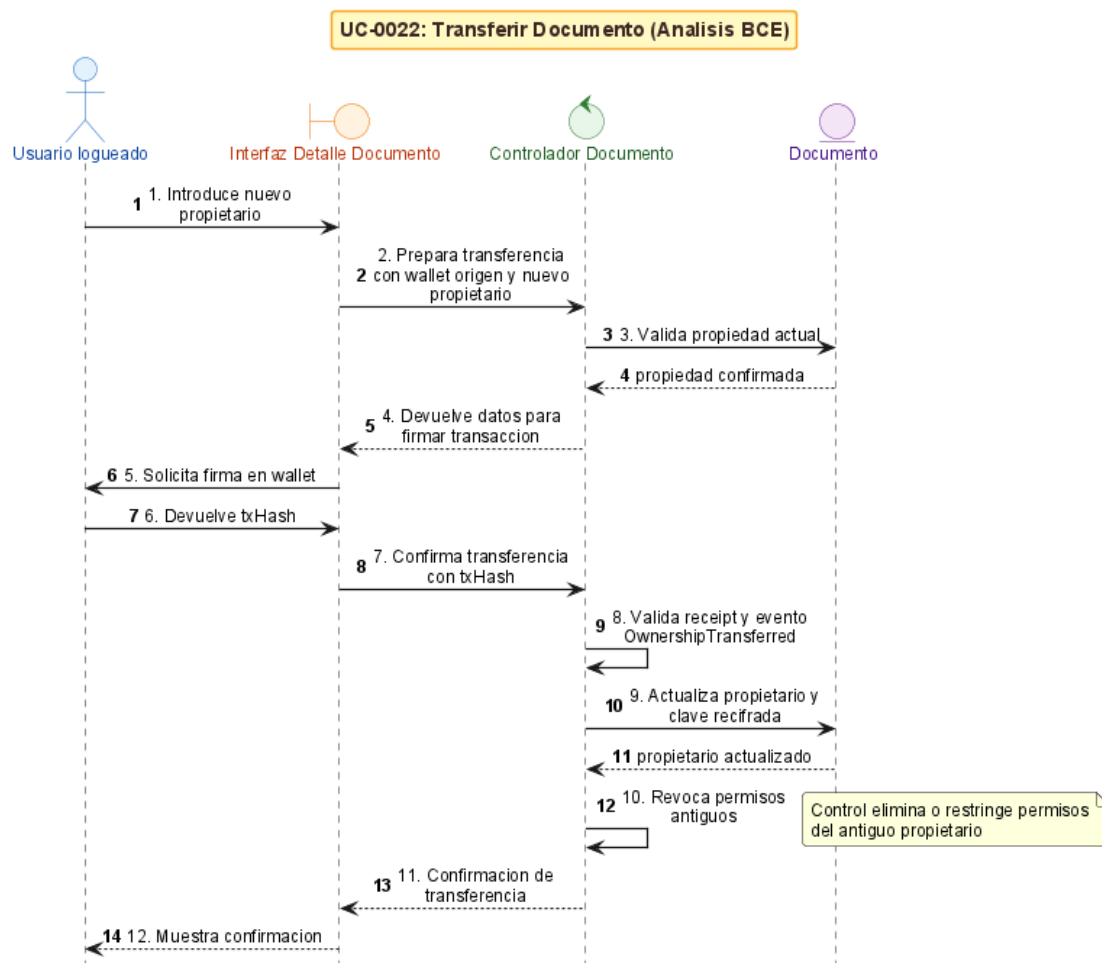


Figura 39: Diagrama de secuencia de análisis BCE – UC-0020: Transferir Documento

6.2.21. Análisis BCE: UC-0021 Buscar Documentos

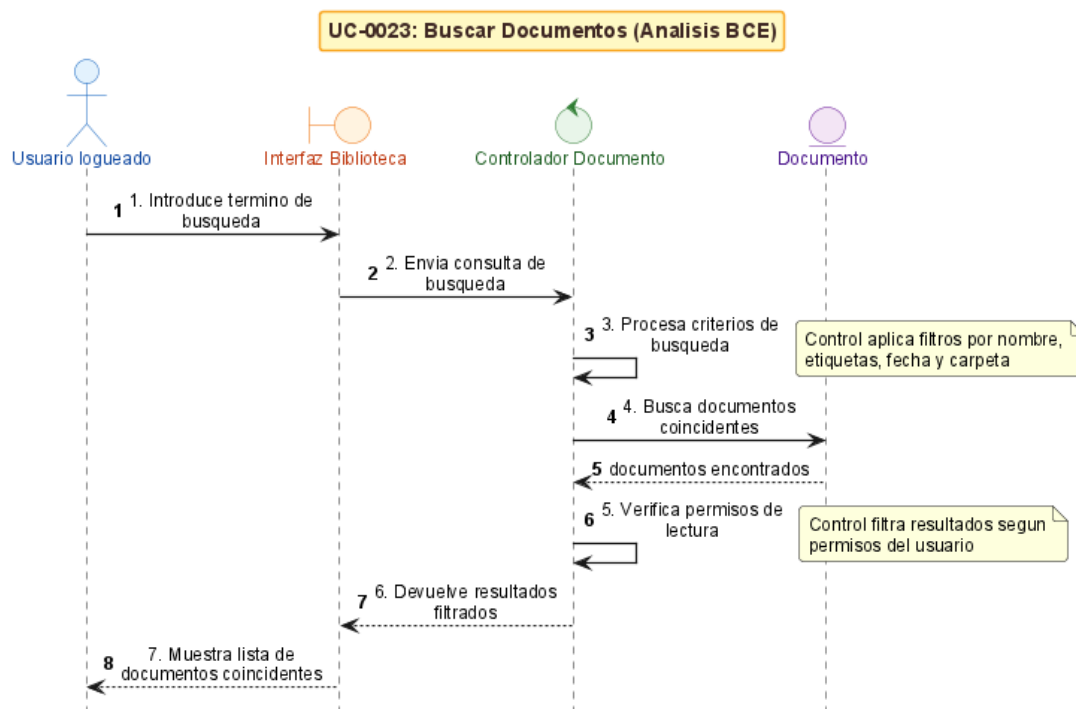


Figura 40: Diagrama de secuencia de análisis BCE – UC-0021: Buscar Documentos

6.2.22. Análisis BCE: UC-0022 Crear Nueva Versión

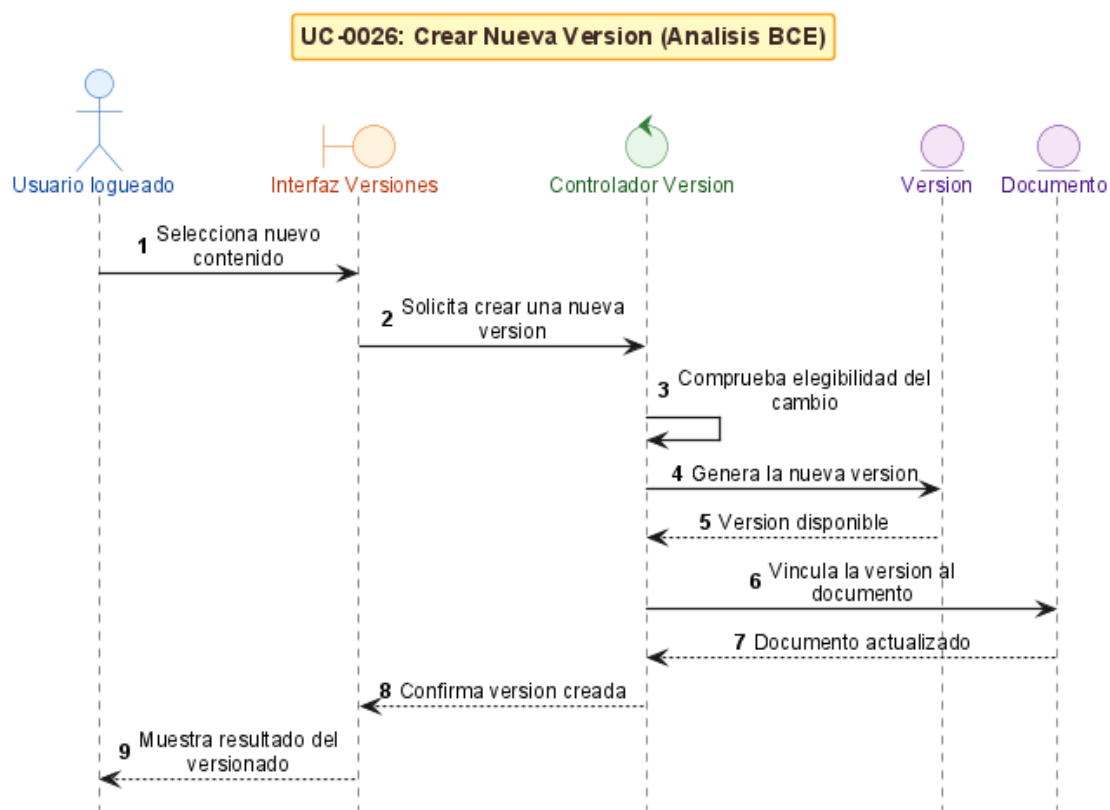


Figura 41: Diagrama de secuencia de análisis BCE – UC-0022: Crear Nueva Versión

6.2.23. Análisis BCE: UC-0023 Listar Versiones

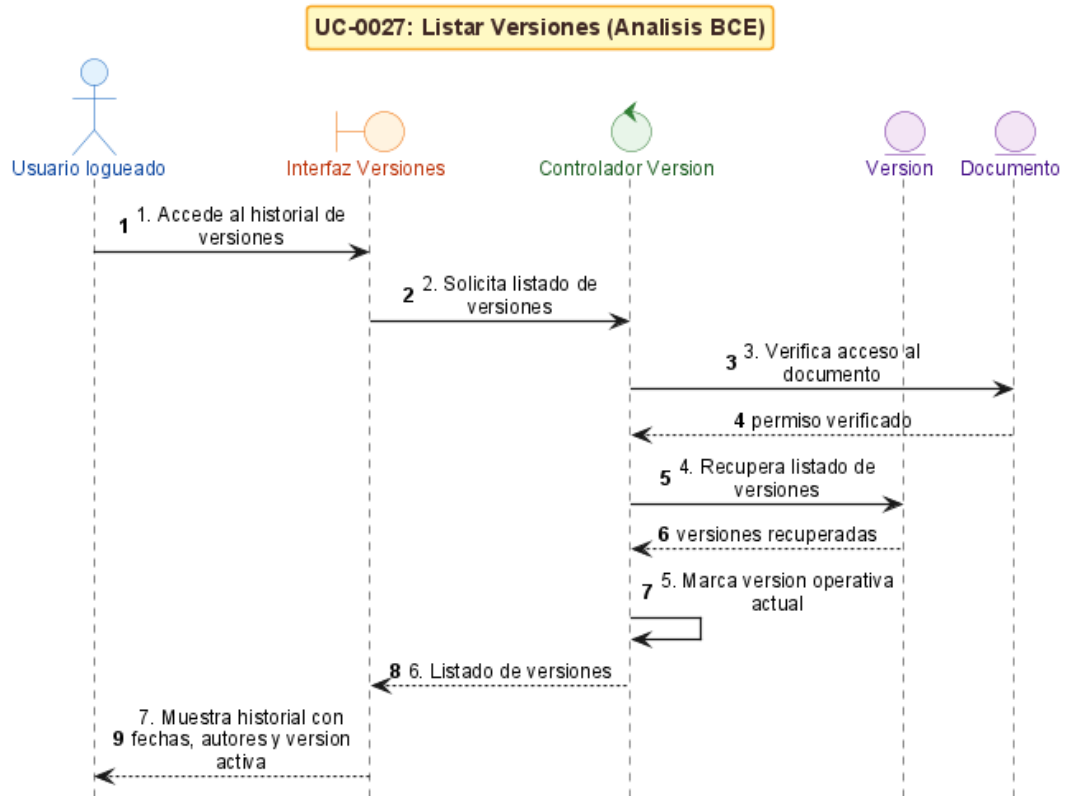


Figura 42: Diagrama de secuencia de análisis BCE – UC-0023: Listar Versiones

6.2.24. Análisis BCE: UC-0024 Establecer Versión Operativa

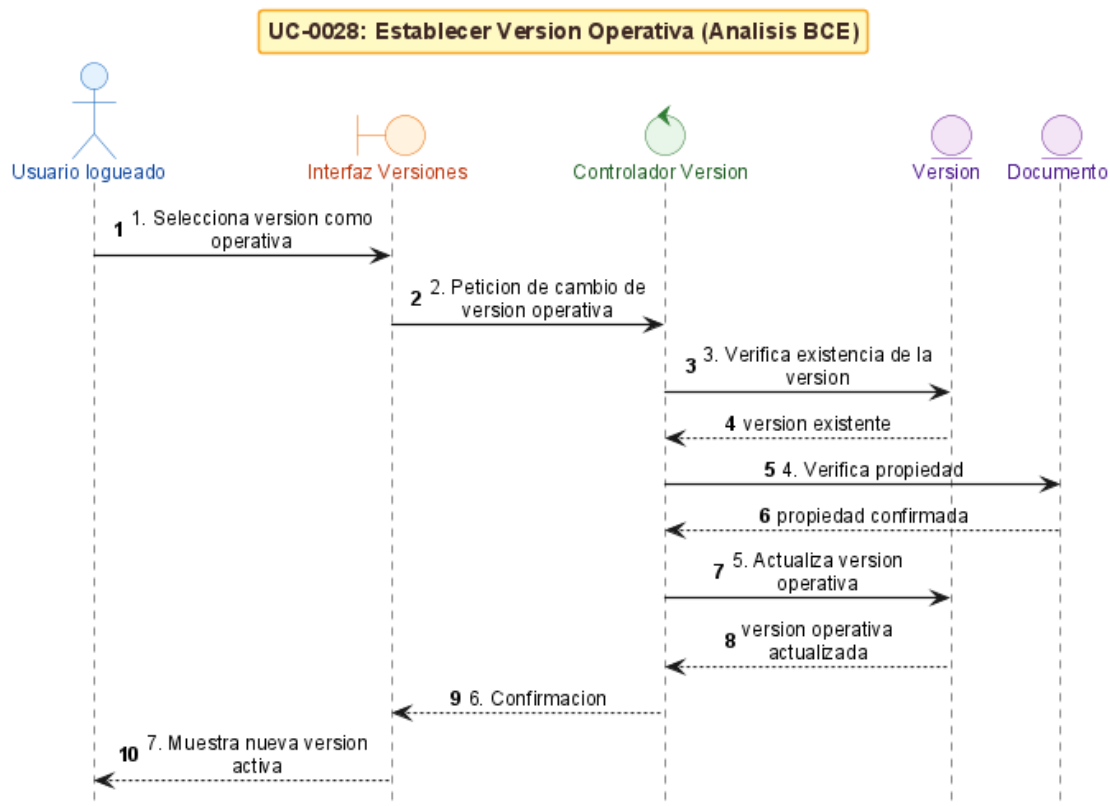


Figura 43: Diagrama de secuencia de análisis BCE – UC-0024: Establecer Versión Operativa

6.2.25. Análisis BCE: UC-0025 Restaurar Versión Anterior

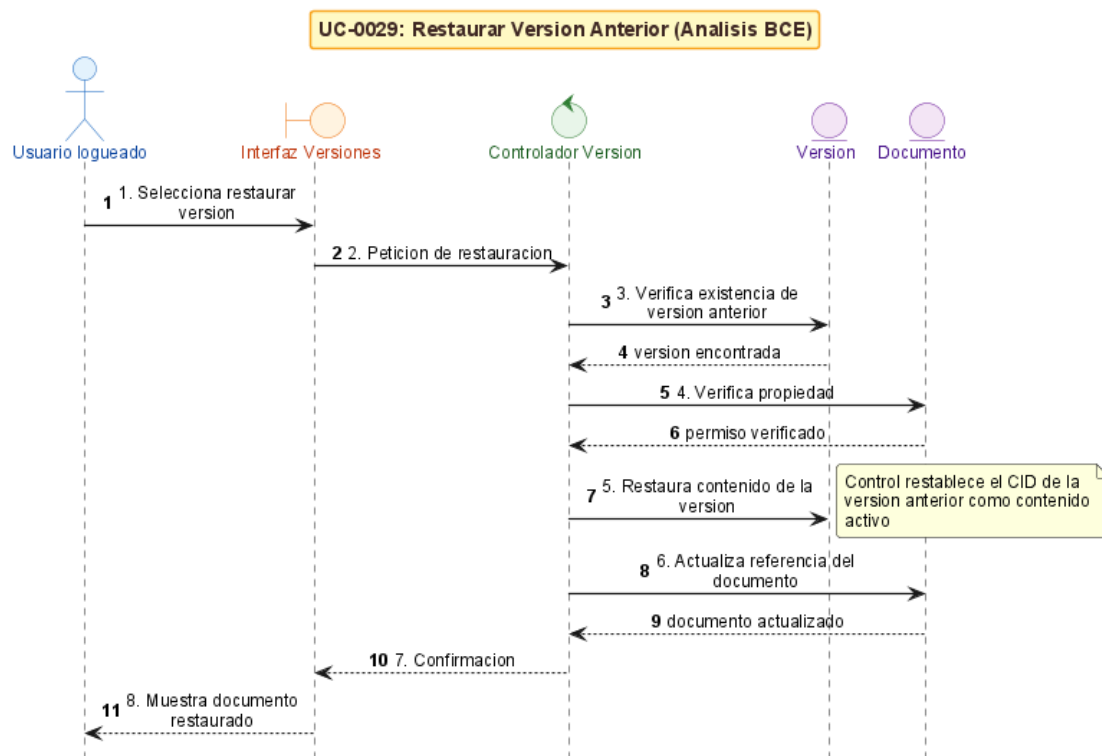


Figura 44: Diagrama de secuencia de análisis BCE – UC-0025: Restaurar Versión Anterior

6.2.26. Análisis BCE: UC-0026 Descargar Versión Específica

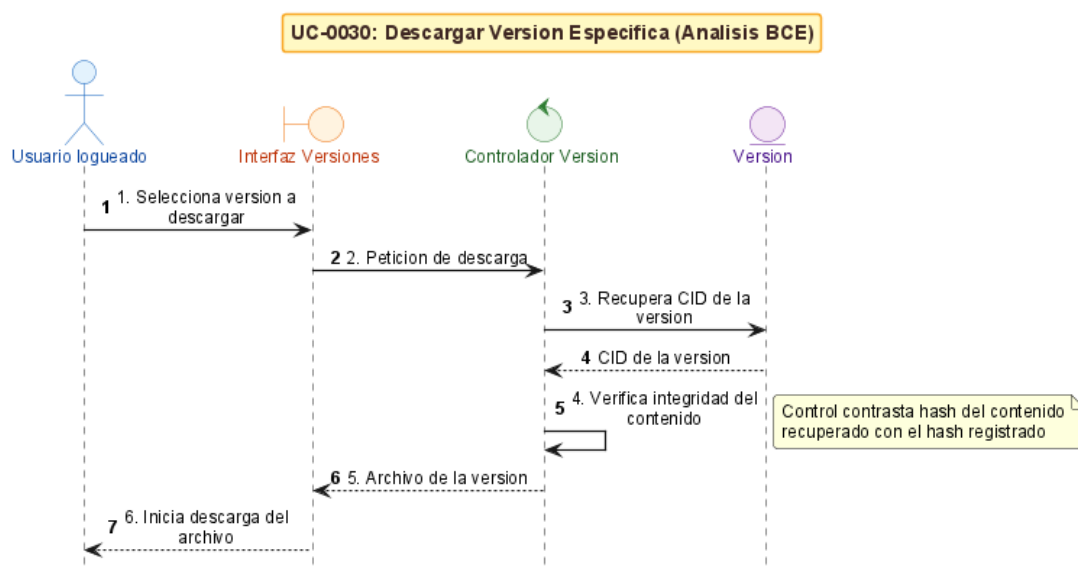


Figura 45: Diagrama de secuencia de análisis BCE – UC-0026: Descargar Versión Específica

6.2.27. Análisis BCE: UC-0027 Firmar Documento

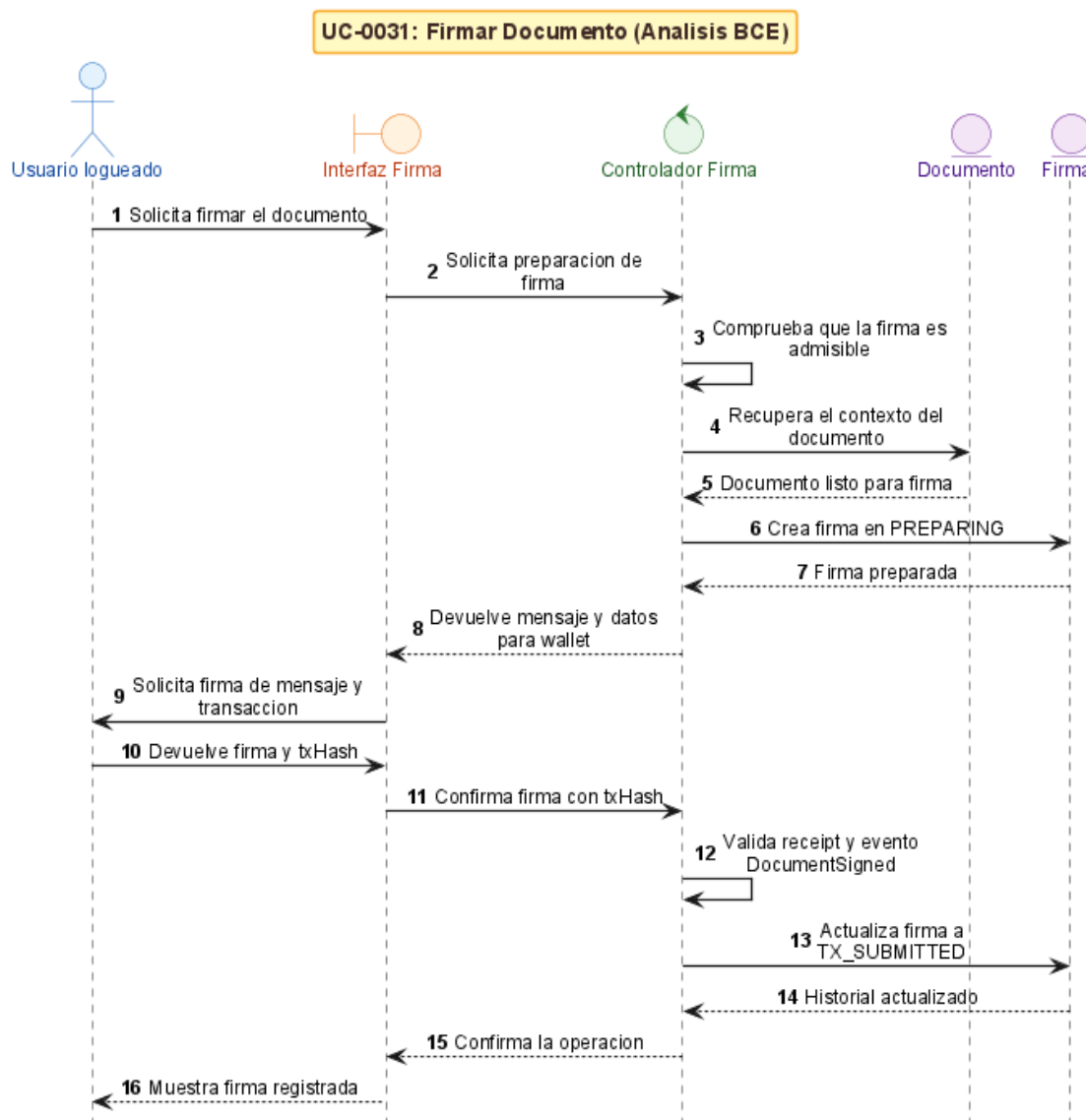


Figura 46: Diagrama de secuencia de análisis BCE – UC-0027: Firmar Documento

6.2.28. Análisis BCE: UC-0028 Ver Firmas de un Documento

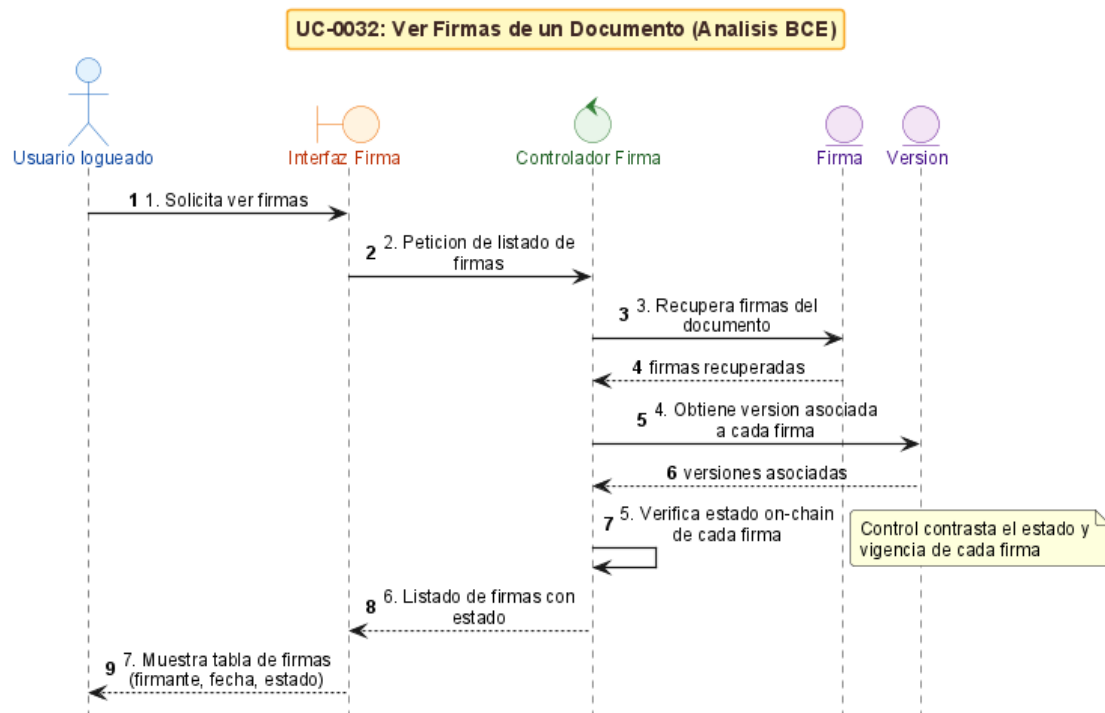


Figura 47: Diagrama de secuencia de análisis BCE – UC-0028: Ver Firmas de un Documento

6.2.29. Análisis BCE: UC-0029 Verificar Firma del Usuario

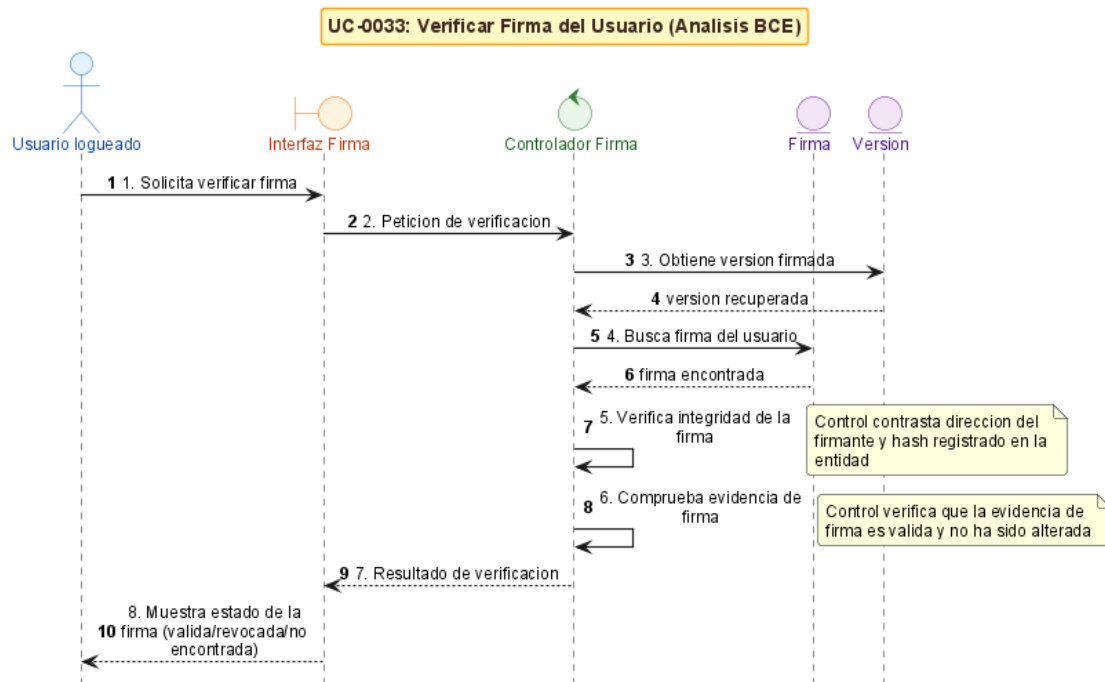


Figura 48: Diagrama de secuencia de análisis BCE – UC-0029: Verificar Firma del Usuario

6.2.30. Análisis BCE: UC-0030 Compartir Documento

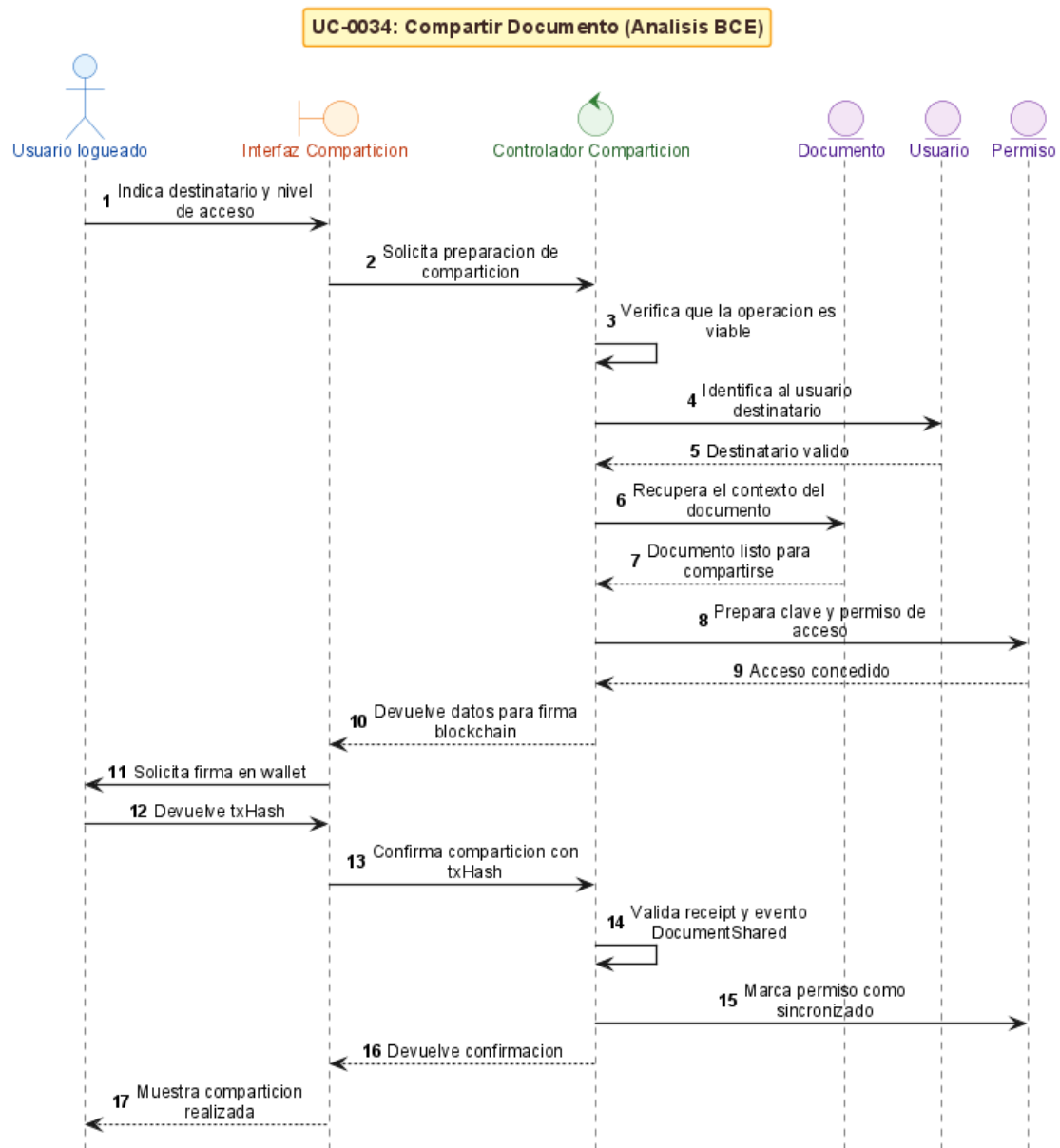


Figura 49: Diagrama de secuencia de análisis BCE – UC-0030: Compartir Documento

6.2.31. Análisis BCE: UC-0031 Revocar Acceso a Documento

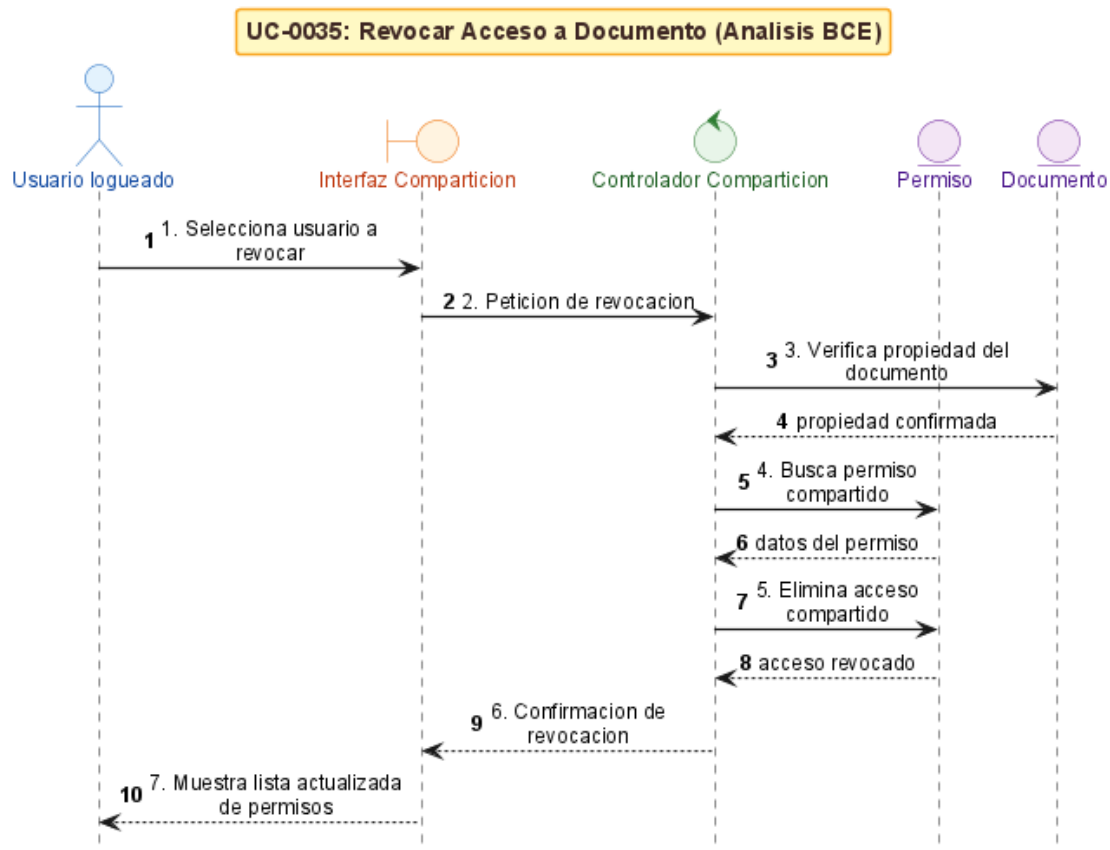


Figura 50: Diagrama de secuencia de análisis BCE – UC-0031: Revocar Acceso a Documento

6.2.32. Análisis BCE: UC-0032 Ver Compartidos Connigo

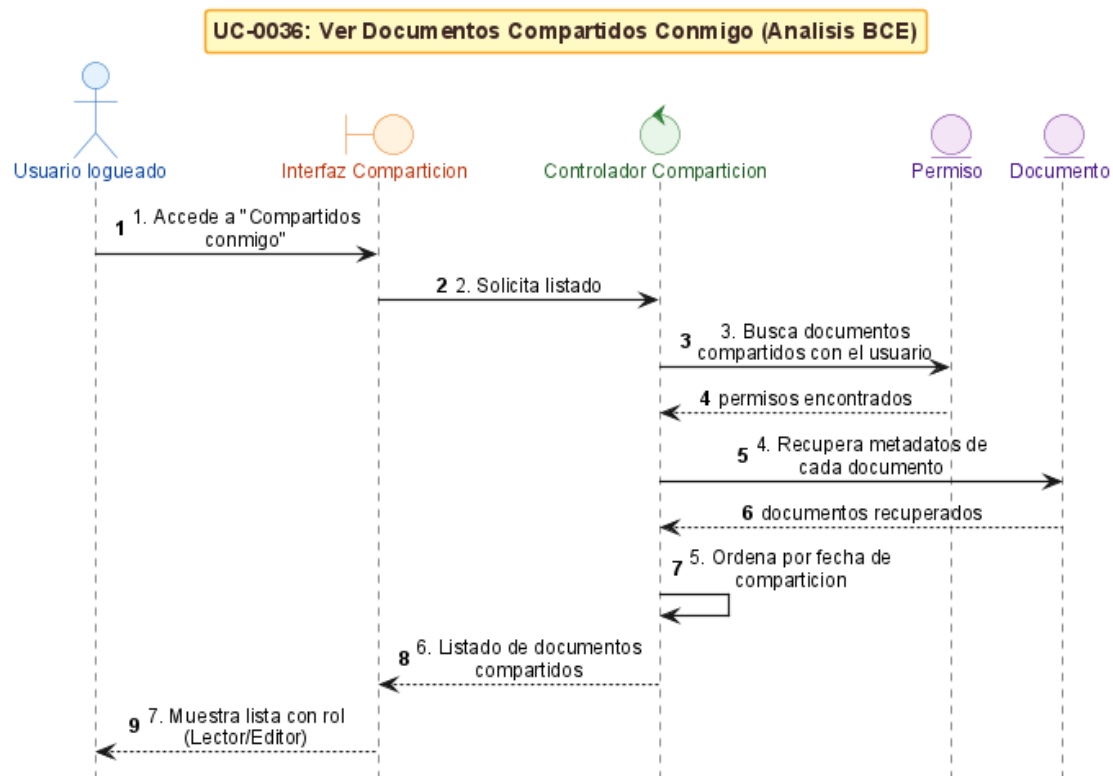


Figura 51: Diagrama de secuencia de análisis BCE – UC-0032: Ver Documentos Compartidos Connigo

6.2.33. Análisis BCE: UC-0033 Ver Timeline de Documento

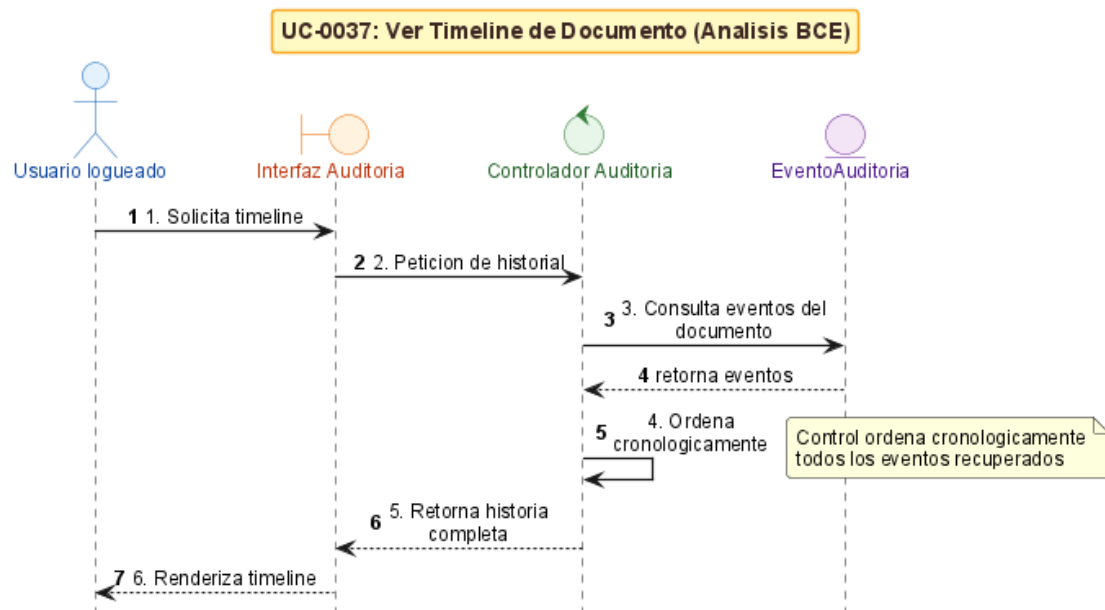


Figura 52: Diagrama de secuencia de análisis BCE – UC-0033: Ver Timeline de Documento

6.2.34. Análisis BCE: UC-0034 Ver Estadísticas Personales

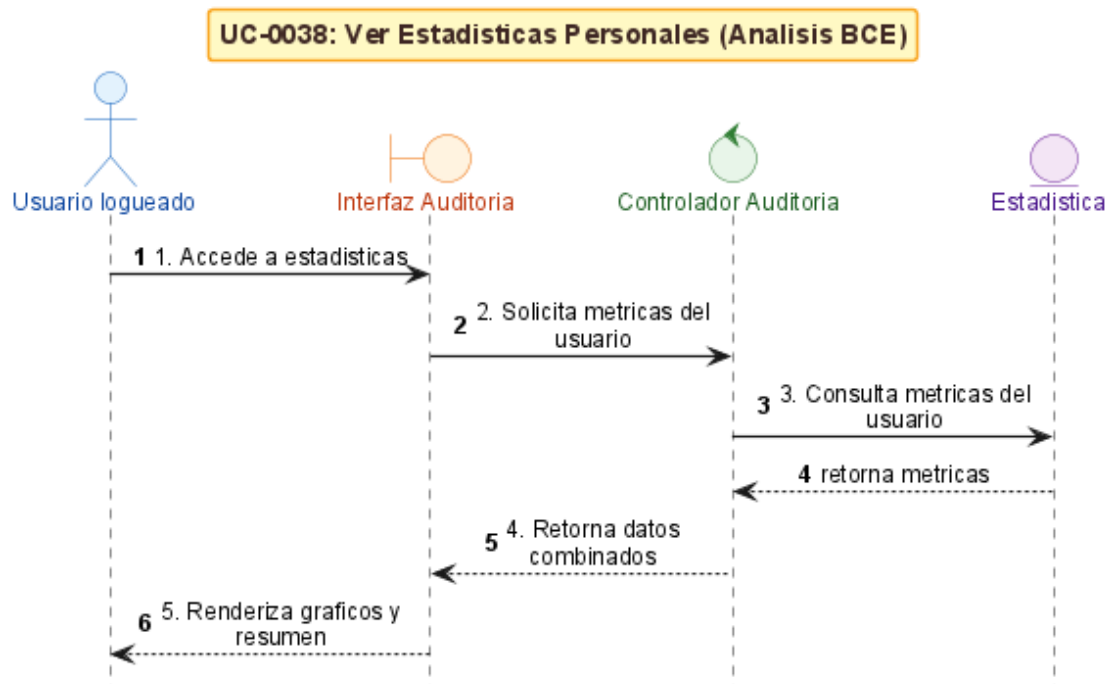


Figura 53: Diagrama de secuencia de análisis BCE – UC-0034: Ver Estadísticas Personales

6.2.35. Análisis BCE: UC-0035 Ver Estadísticas Globales del Sistema

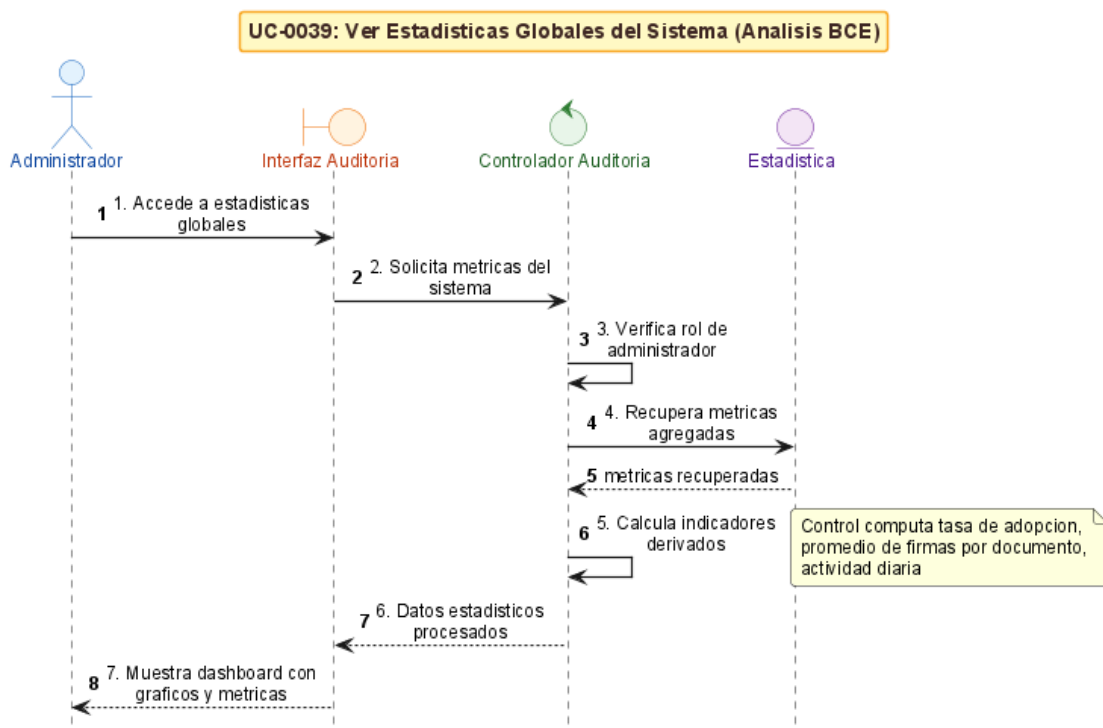


Figura 54: Diagrama de secuencia de análisis BCE – UC-0035: Ver Estadísticas Globales del Sistema

6.2.36. Análisis BCE: UC-0036 Verificar Autenticidad de Documento

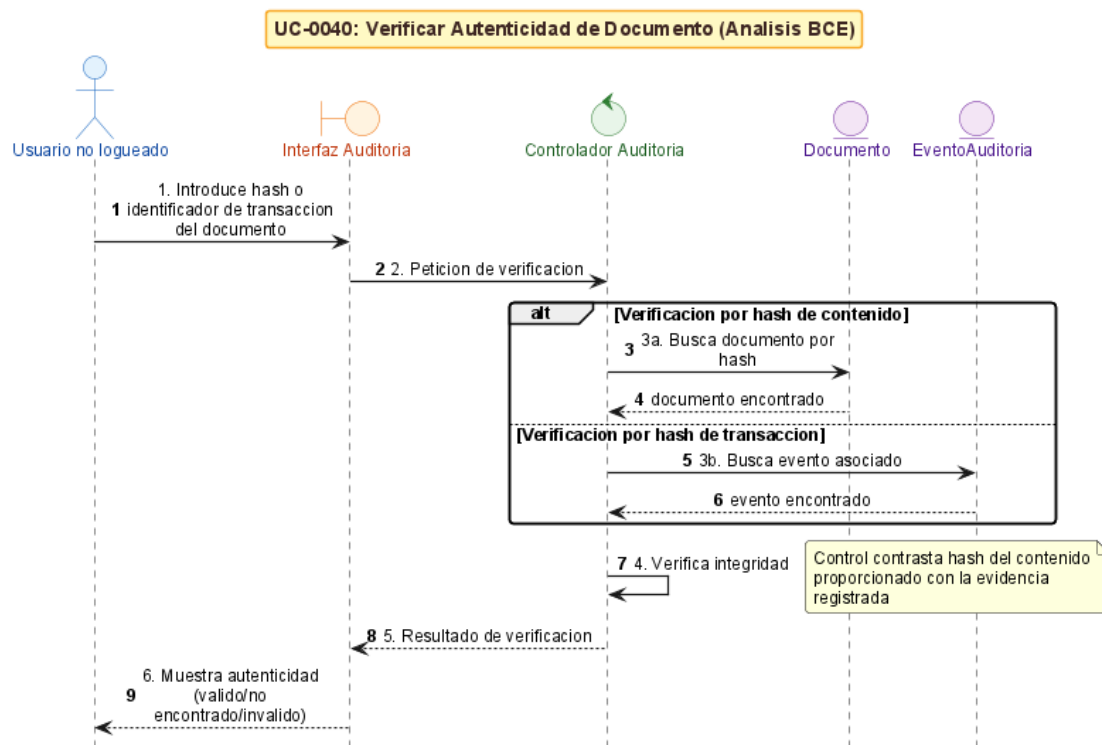


Figura 55: Diagrama de secuencia de análisis BCE – UC-0036: Verificar Autenticidad de Documento

6.2.37. Análisis BCE: UC-0037 Auditoría Blockchain

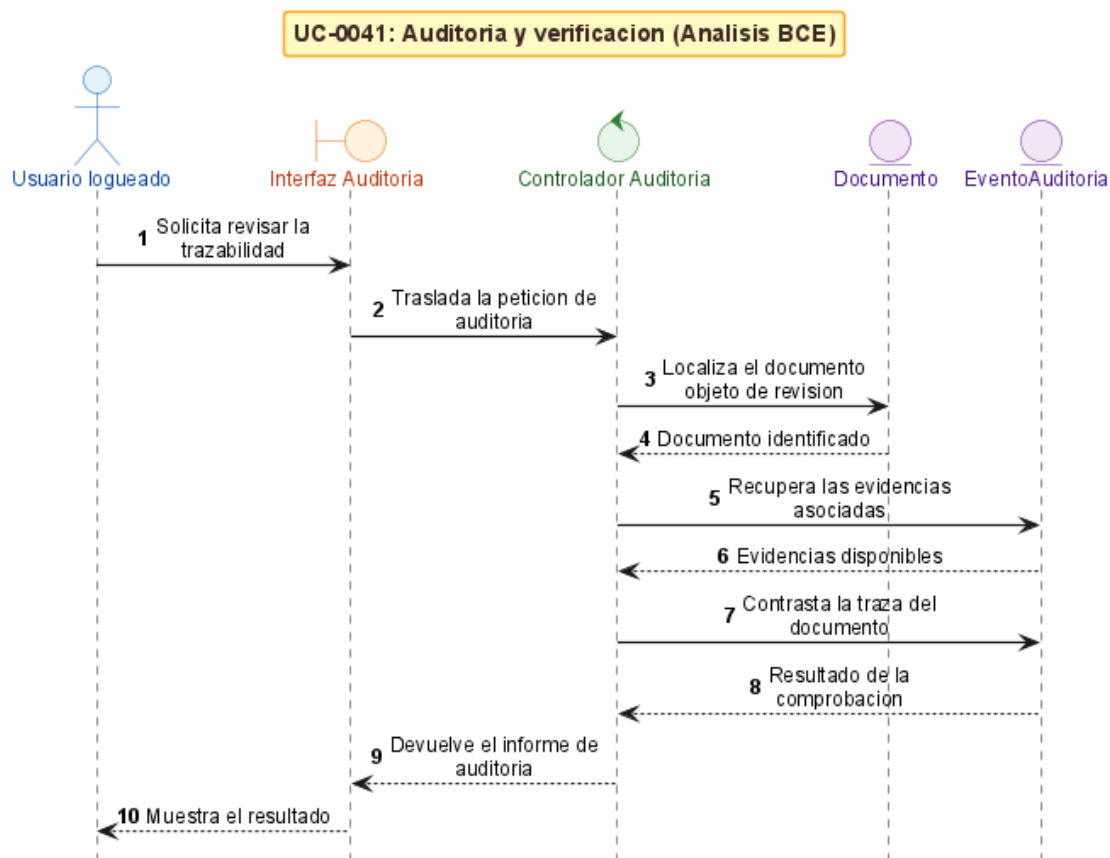


Figura 56: Diagrama de secuencia de análisis BCE – UC-0037: Auditoría Blockchain

6.2.38. Análisis BCE: UC-0038 Listar Usuarios

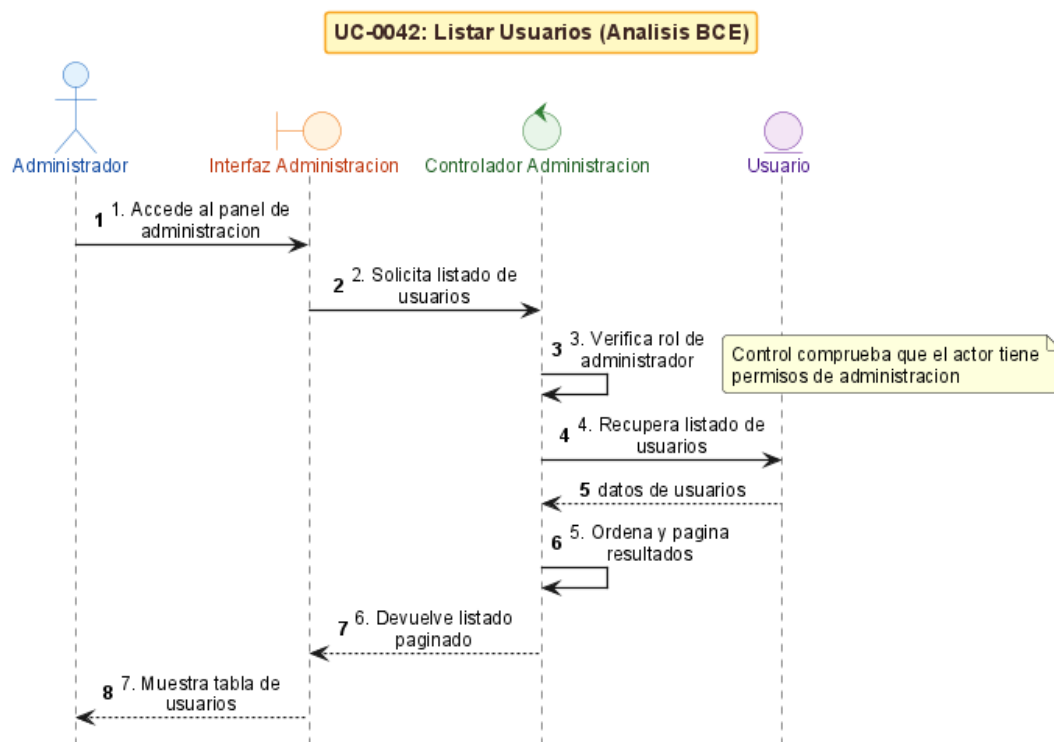


Figura 57: Diagrama de secuencia de análisis BCE – UC-0038: Listar Usuarios

6.2.39. Análisis BCE: UC-0039 Gestionar Roles de Usuario

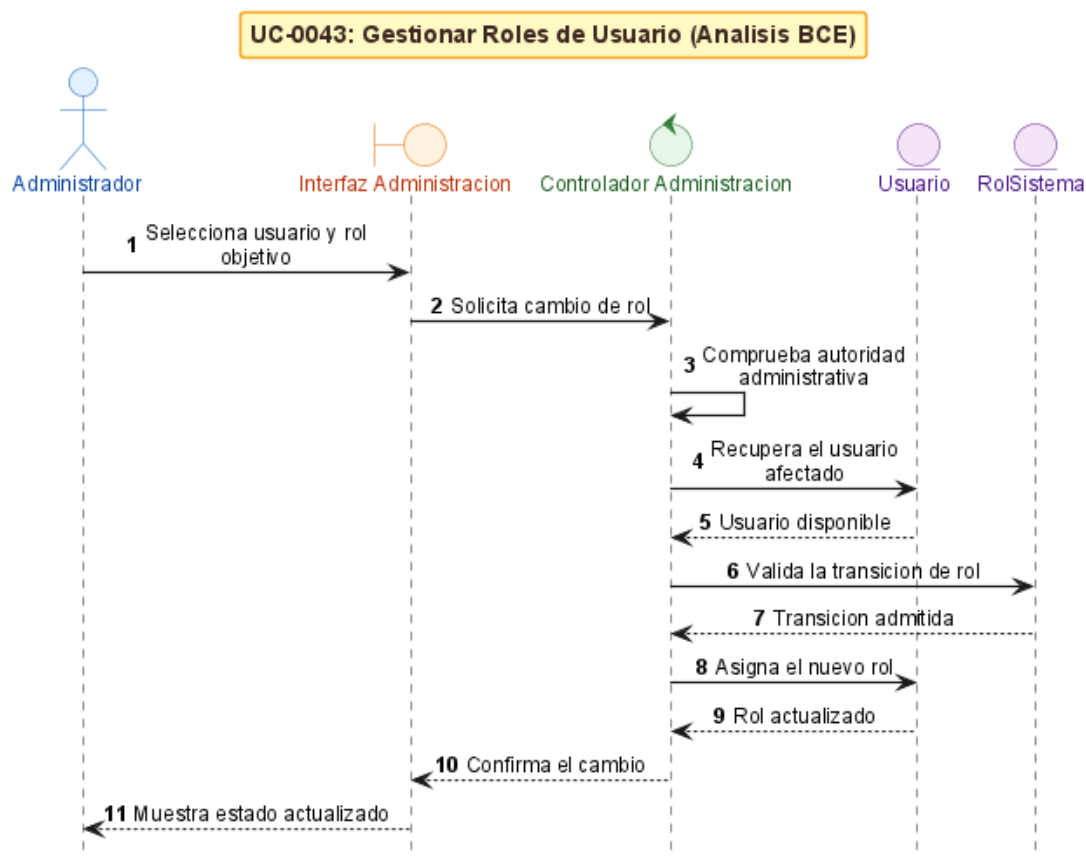


Figura 58: Diagrama de secuencia de análisis BCE – UC-0039: Gestionar Roles de Usuario

6.3. Diagramas de secuencia de diseño

Los diagramas de secuencia de diseño ya muestran los componentes reales del sistema: páginas React, endpoints REST, servicios del backend, capa de acceso a base de datos sobre PostgreSQL, adaptador IPFS y el contrato DocumentRegistry. A diferencia de los diagramas BCE de análisis, aquí cada objeto equivale a una clase o servicio concreto del código.

6.3.1. Secuencia: Registro de usuario

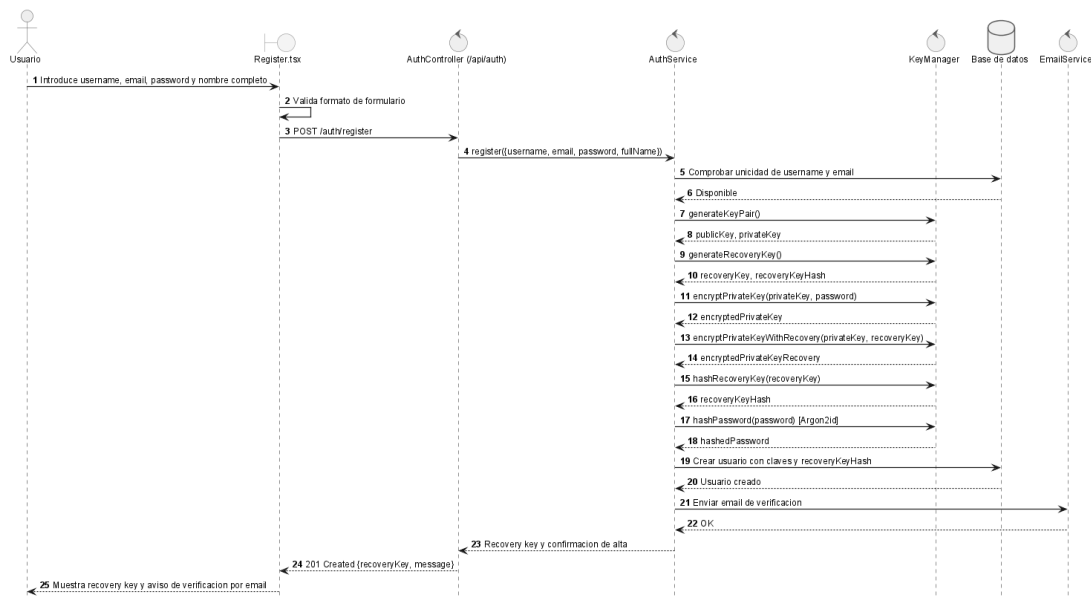


Figura 59: Diagrama de secuencia UC-0001: Registrar Usuario

6.3.2. Secuencia: Iniciar Sesión

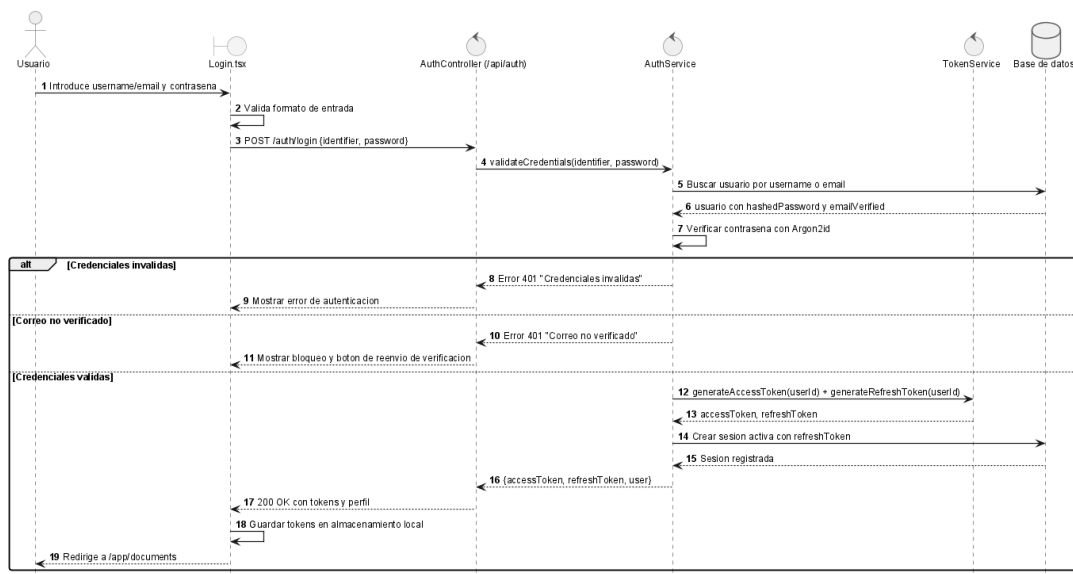


Figura 60: Diagrama de secuencia UC-0002: Iniciar Sesión

6.3.3. Secuencia: Consultar Notificaciones

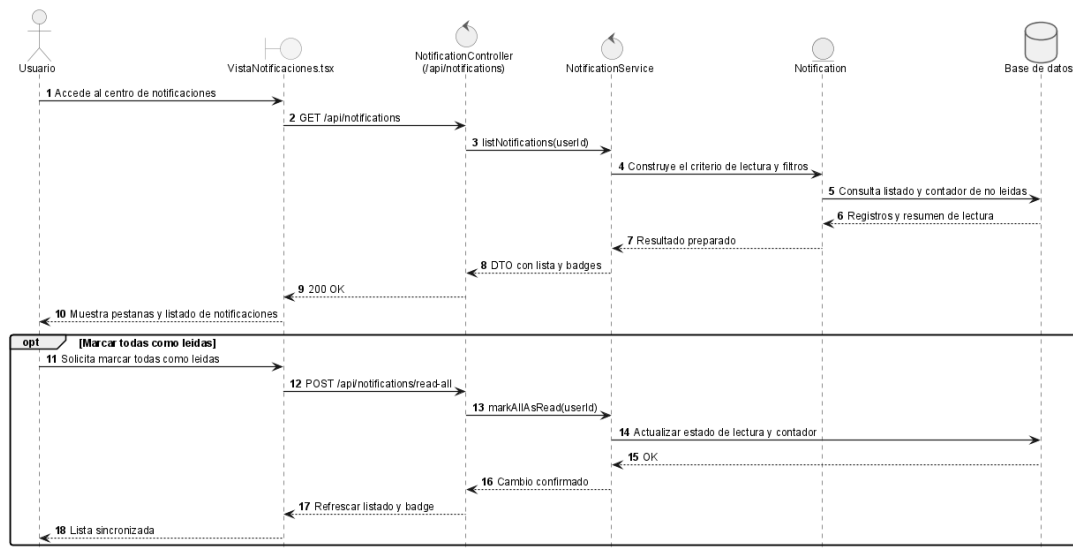


Figura 61: Diagrama de secuencia UC-0003: Consultar Notificaciones

6.3.4. Secuencia: Cerrar Sesión

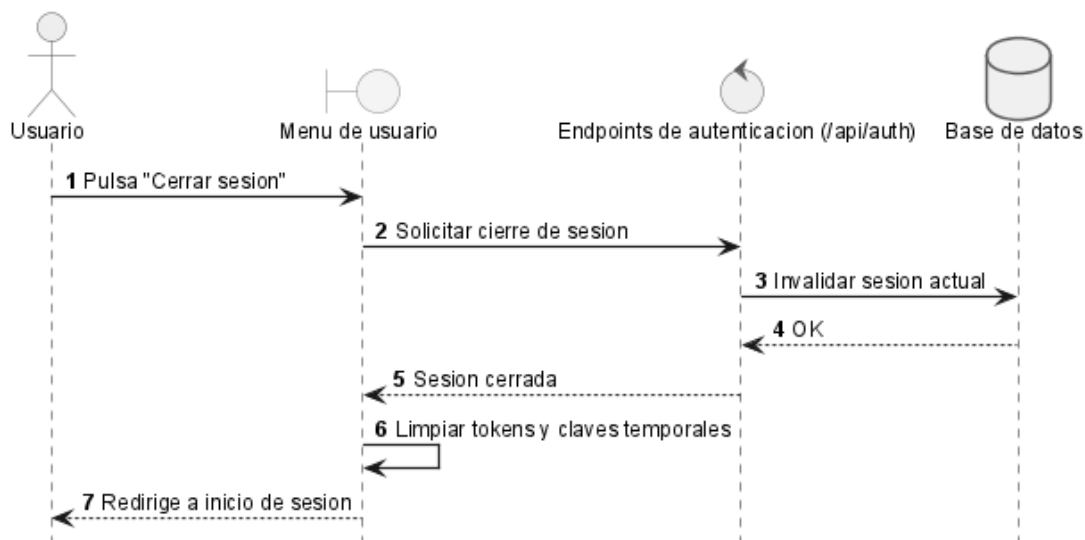


Figura 62: Diagrama de secuencia UC-0004: Cerrar Sesión

6.3.5. Secuencia: Conectar Wallet

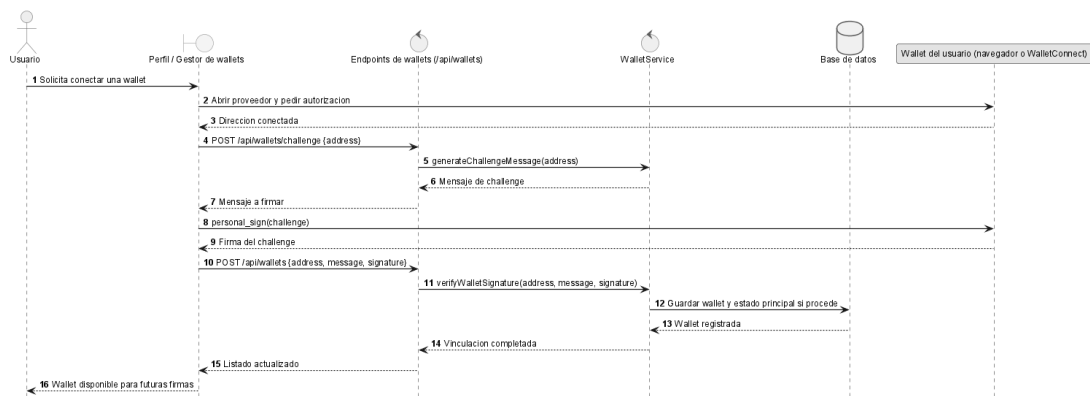


Figura 63: Diagrama de secuencia UC-0005: Conectar Wallet

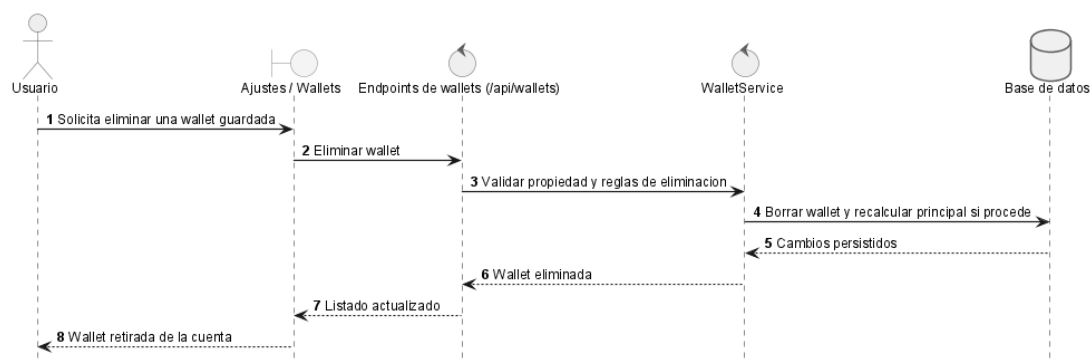


Figura 64: Diagrama de secuencia UC-0006: Eliminar Wallet

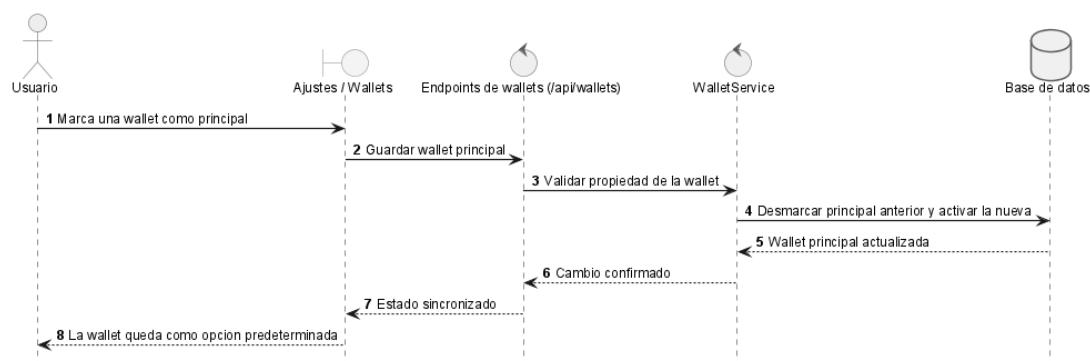


Figura 65: Diagrama de secuencia UC-0007: Establecer Wallet Principal

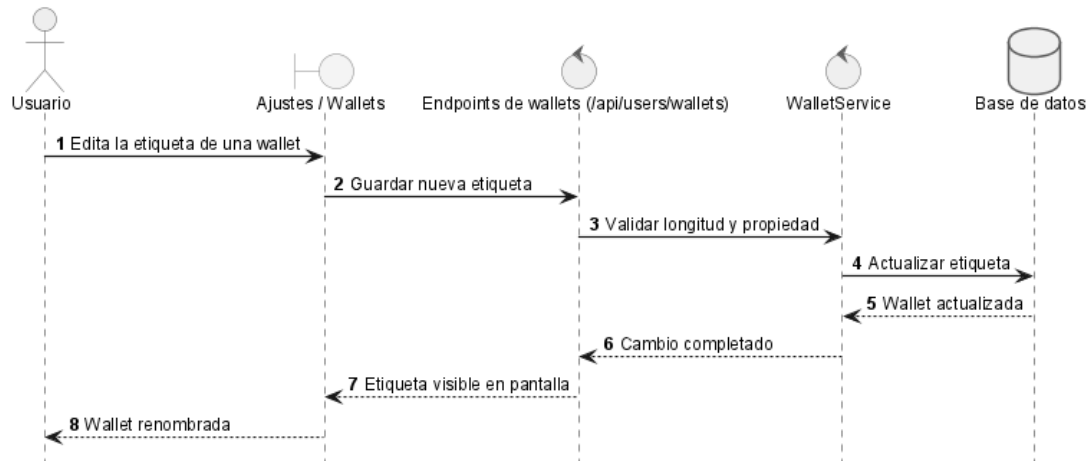


Figura 66: Diagrama de secuencia UC-0008: Renombrar/Etiquetar Wallet

6.3.6. Secuencia: Gestionar Perfil

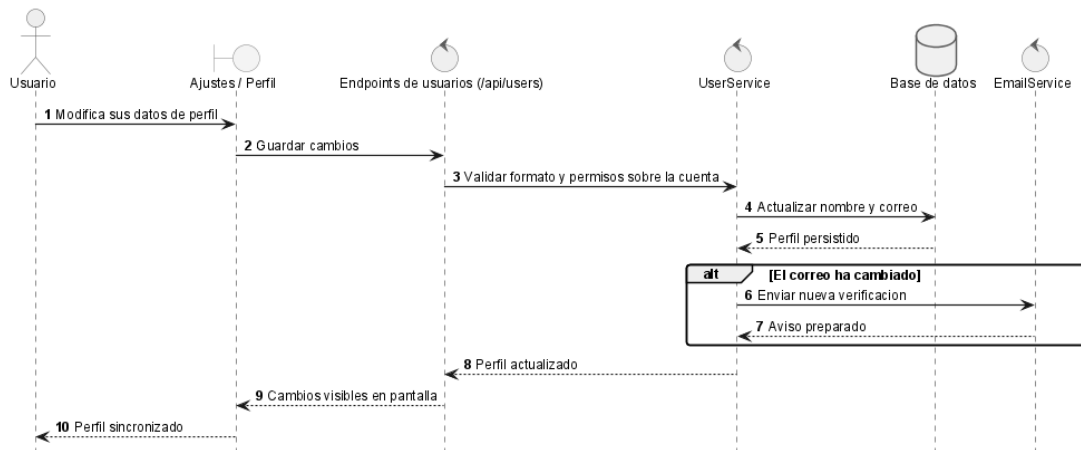


Figura 67: Diagrama de secuencia UC-0009: Gestionar Perfil

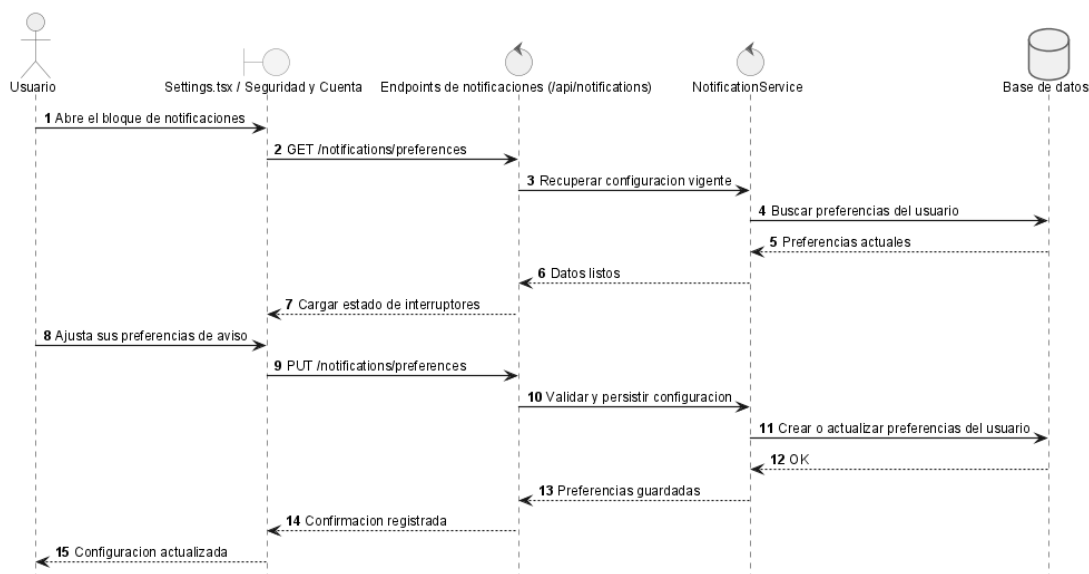


Figura 68: Diagrama de secuencia UC-0010: Configurar Preferencias de Notificación

6.3.7. Secuencia: Cambiar Contraseña

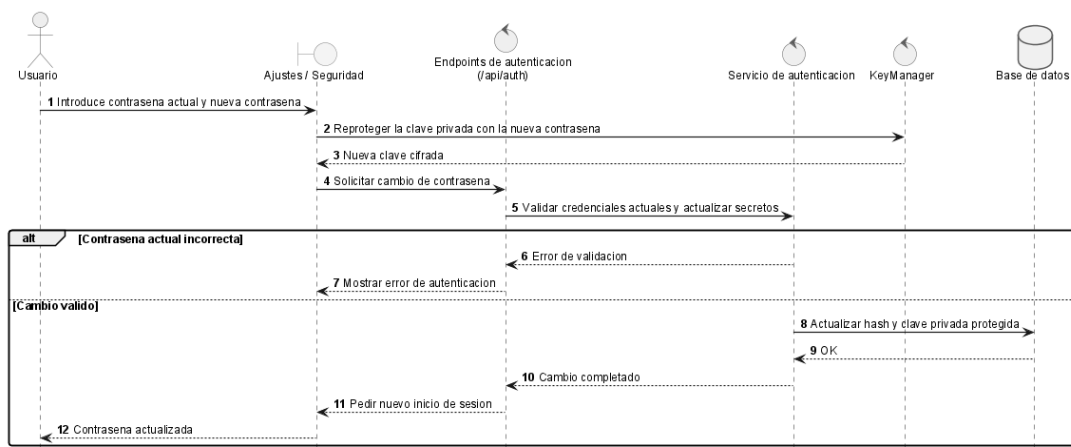


Figura 69: Diagrama de secuencia UC-0010: Cambiar Contraseña

6.3.8. Secuencia: Verificar Email

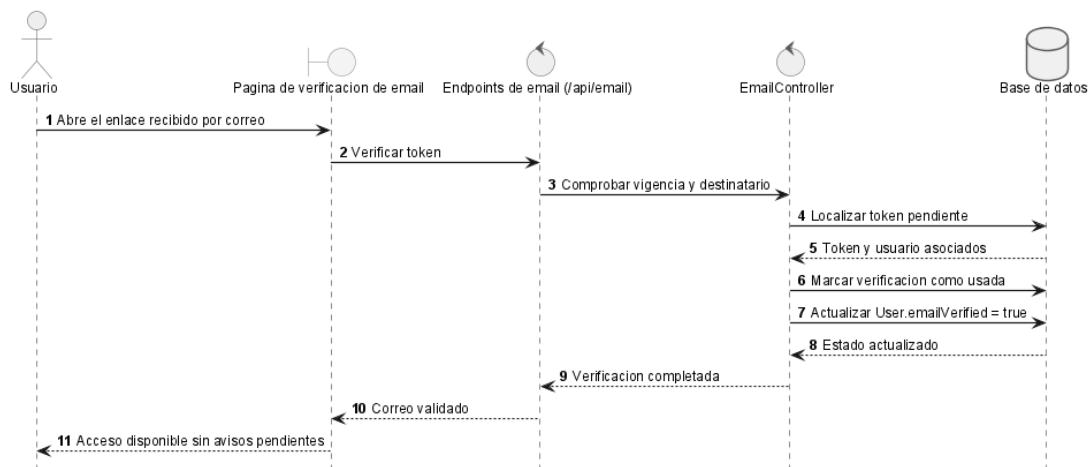


Figura 70: Diagrama de secuencia UC-0011: Verificar Email

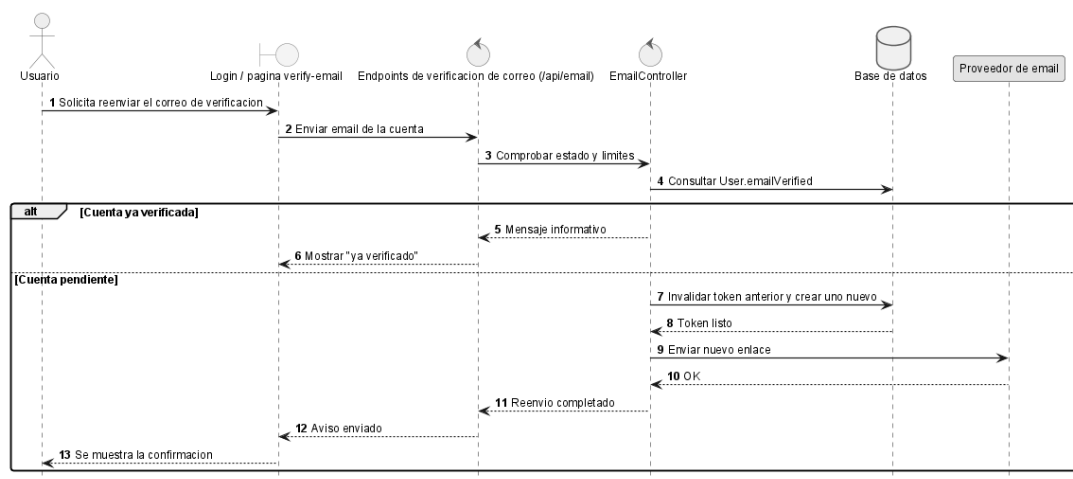


Figura 71: Diagrama de secuencia UC-0015: Reenviar Email de Verificación

6.3.9. Secuencia: Subir documento

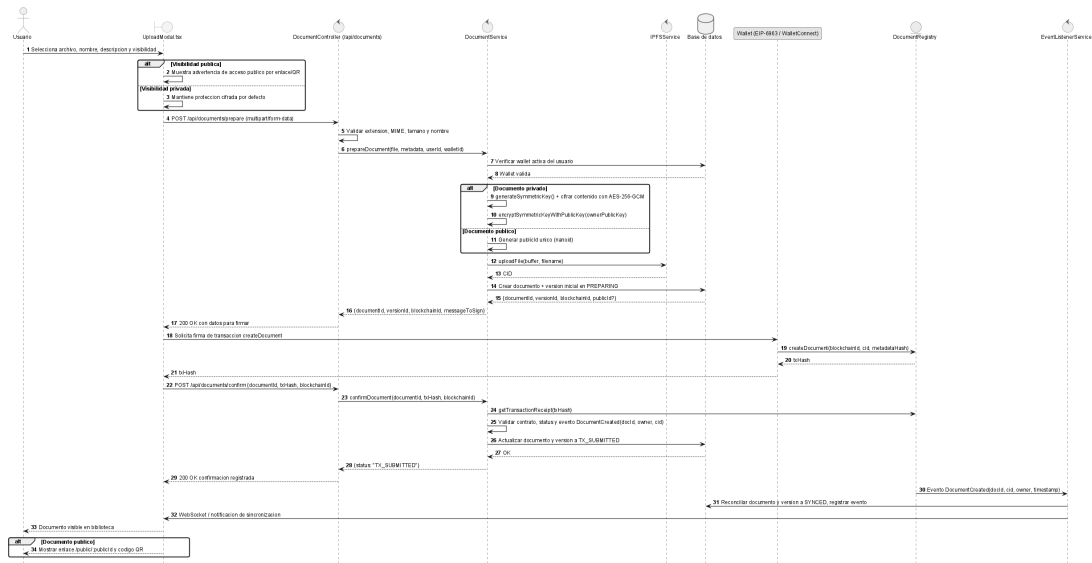


Figura 72: Diagrama de secuencia UC-0013: Subir Documento

6.3.10. Secuencia: Listar Documentos

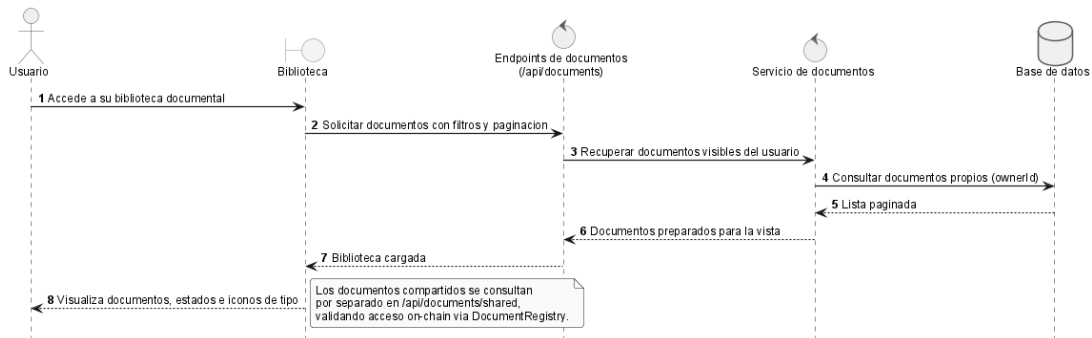


Figura 73: Diagrama de secuencia UC-0014: Listar Documentos

6.3.11. Secuencia: Ver Detalle de Documento

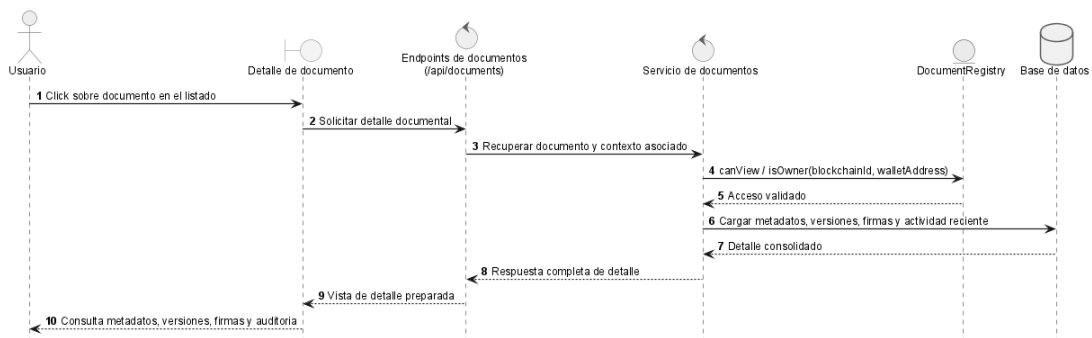


Figura 74: Diagrama de secuencia UC-0015: Ver Detalle de Documento

6.3.12. Secuencia: Descargar documento

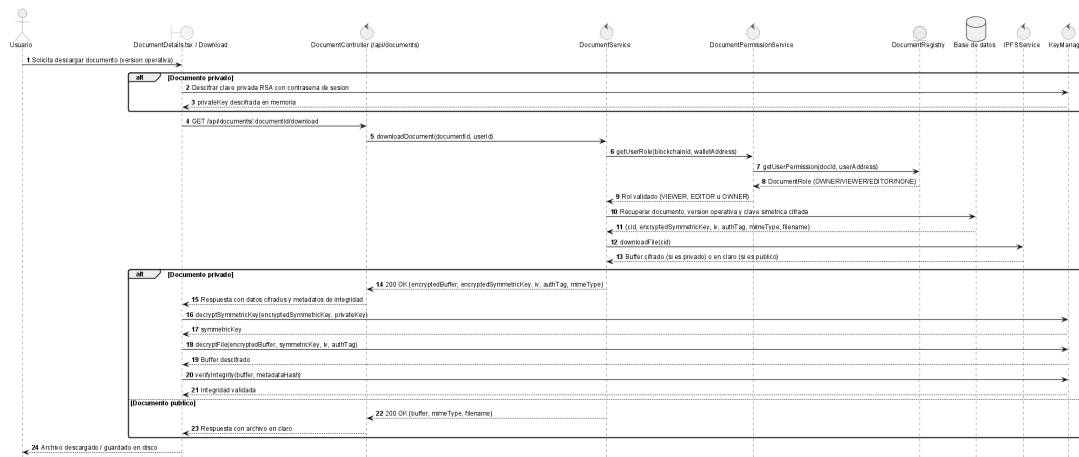


Figura 75: Diagrama de secuencia UC-0016: Descargar Documento

6.3.13. Secuencia: Archivar Documento

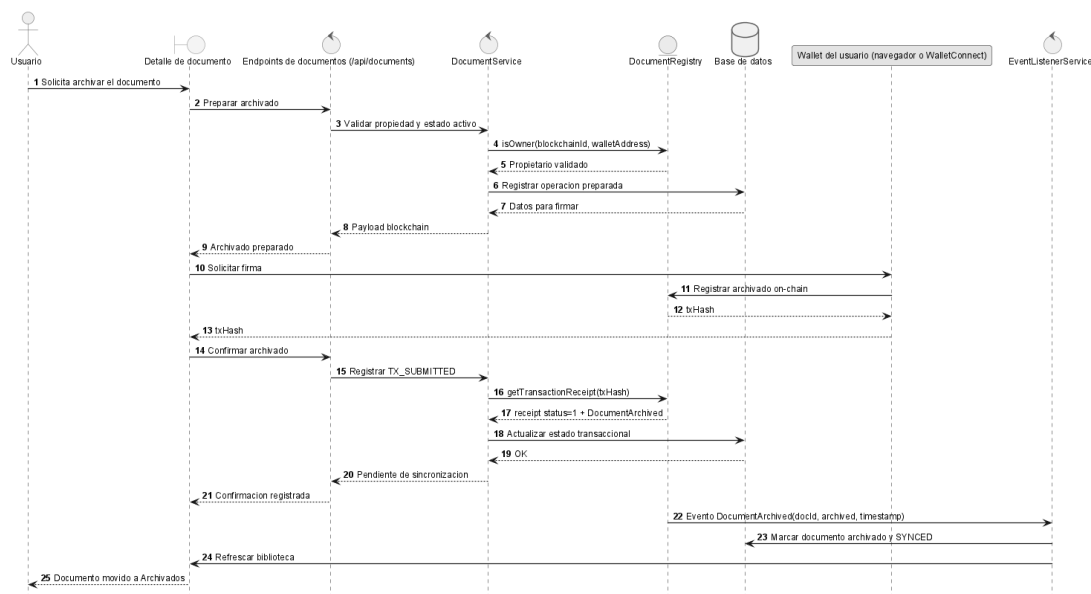


Figura 76: Diagrama de secuencia UC-0017: Archivar Documento

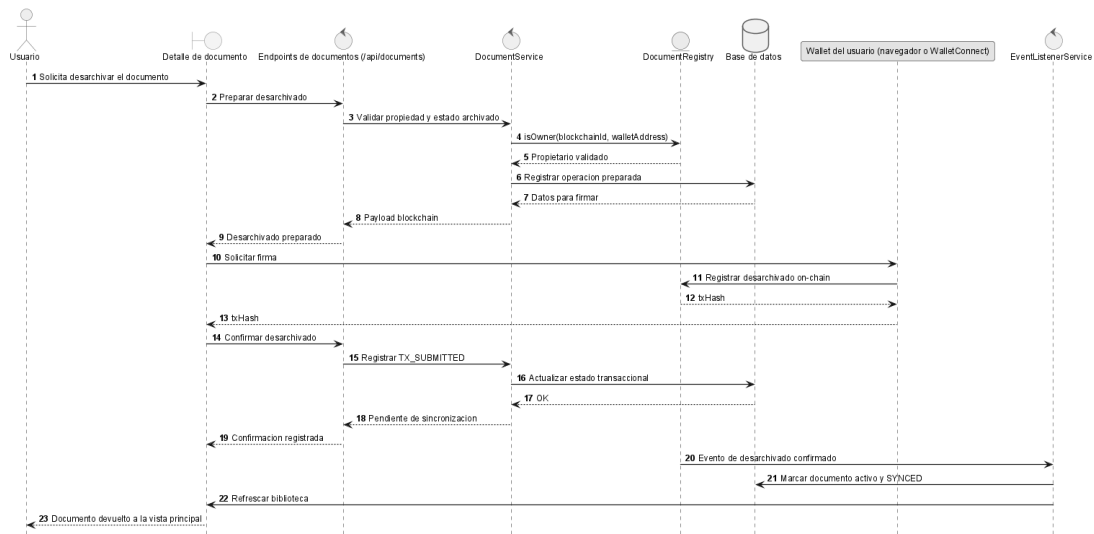


Figura 77: Diagrama de secuencia UC-0018: Desarchivar Documento

6.3.14. Secuencia: Eliminar Documento

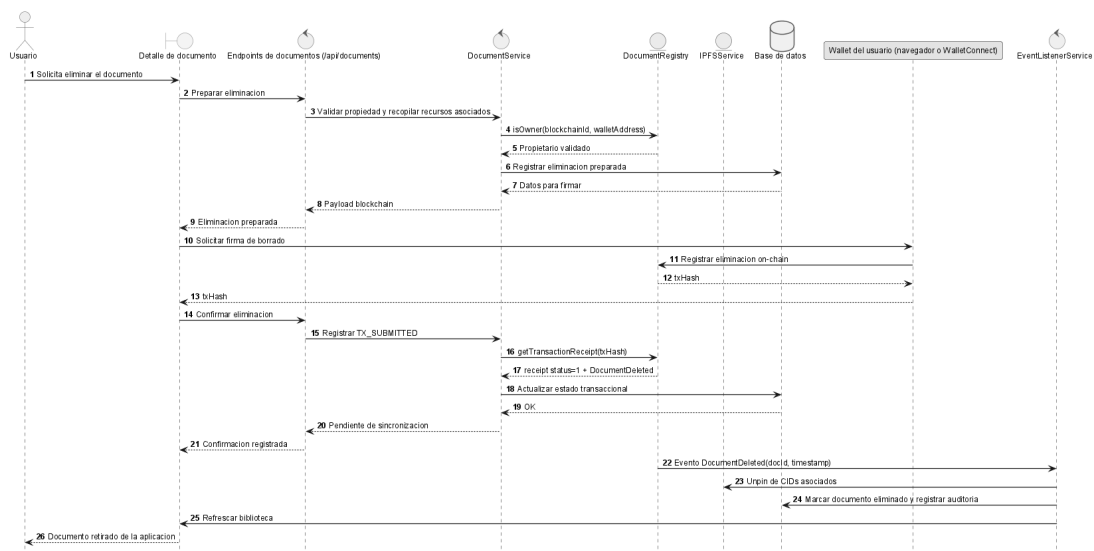


Figura 78: Diagrama de secuencia UC-0019: Eliminar Documento

6.3.15. Secuencia: Transferir Documento

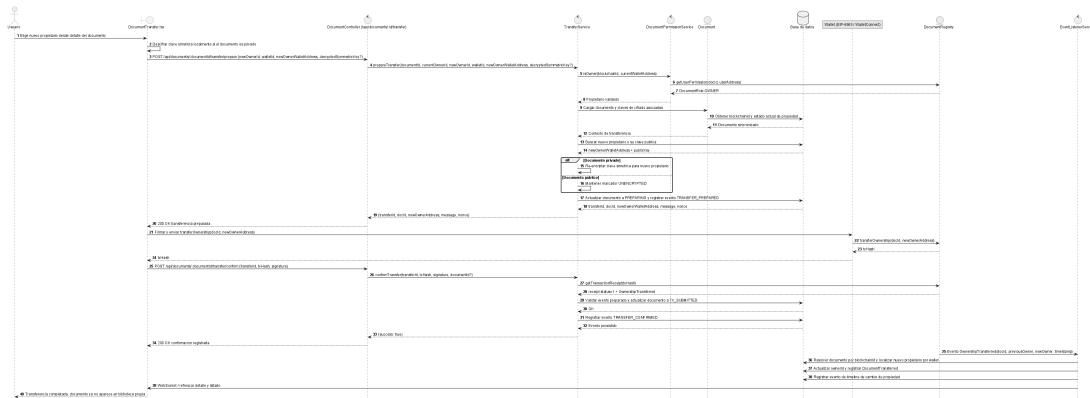


Figura 79: Diagrama de secuencia UC-0020: Transferir Documento

6.3.16. Secuencia: Buscar Documentos

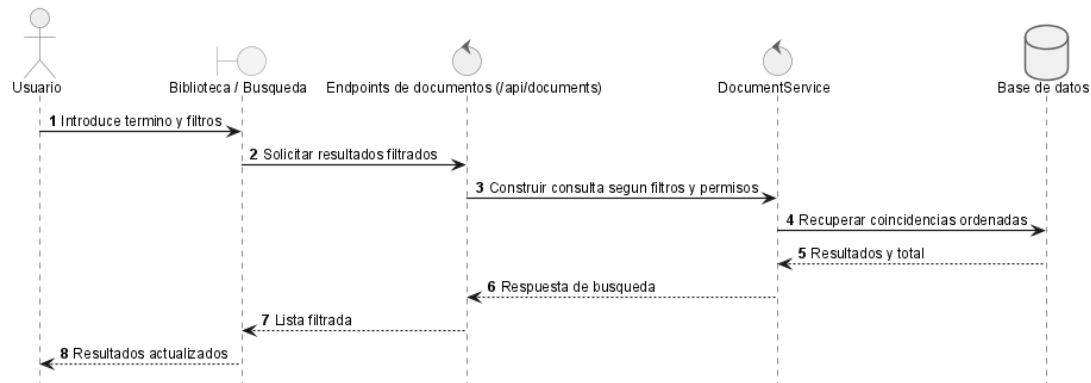


Figura 80: Diagrama de secuencia UC-0021: Buscar Documentos

6.3.17. Secuencia: Crear nueva versión

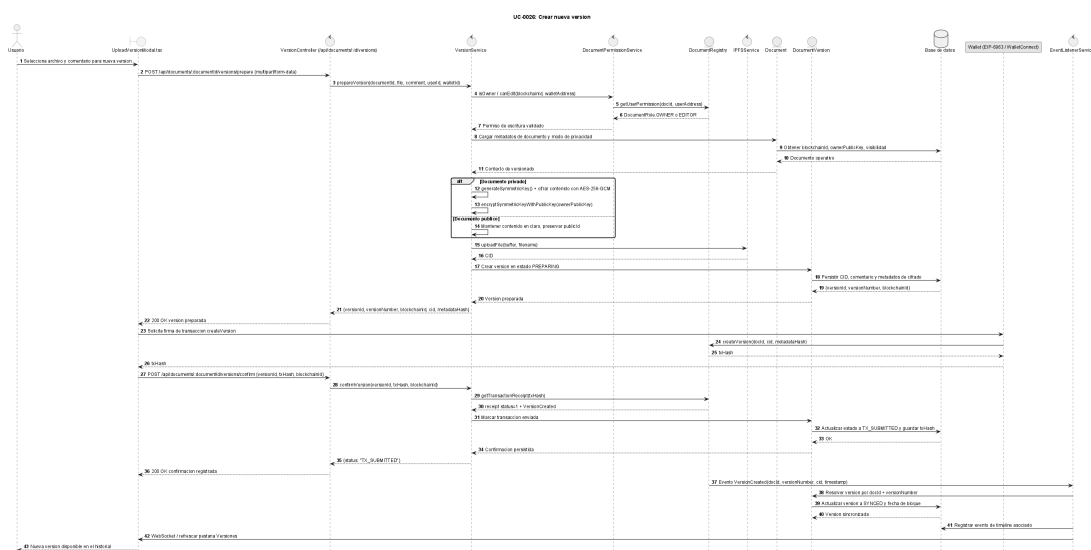


Figura 81: Diagrama de secuencia UC-0022: Crear Nueva Versión

6.3.18. Secuencia: Listar Versiones

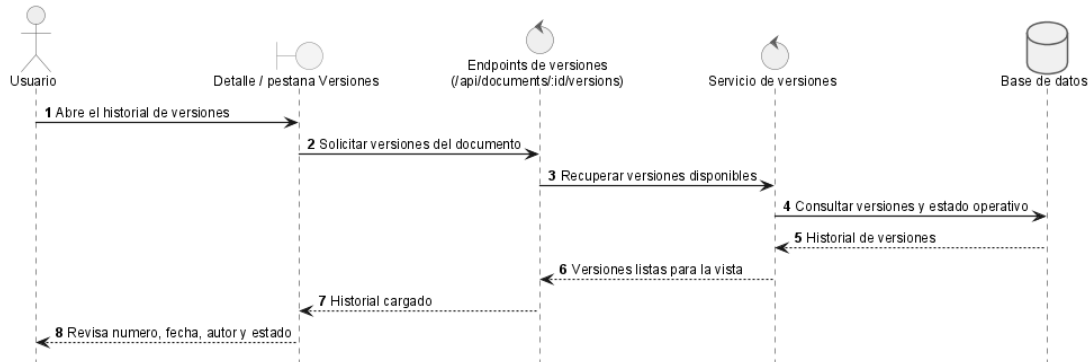


Figura 82: Diagrama de secuencia UC-0023: Listar Versiones

6.3.19. Secuencia: Establecer Versión Operativa

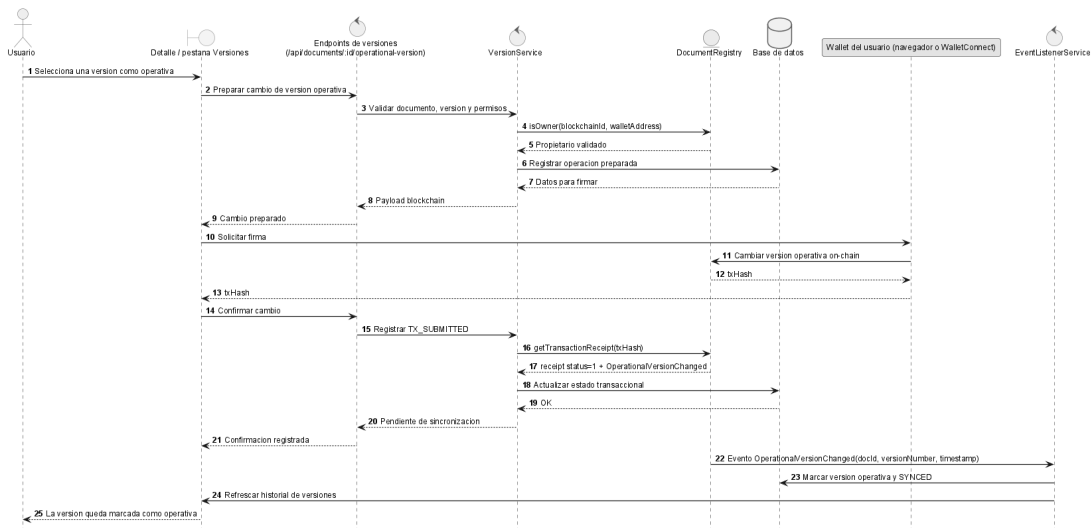


Figura 83: Diagrama de secuencia UC-0024: Establecer Versión Operativa

6.3.20. Secuencia: Restaurar Versión Anterior

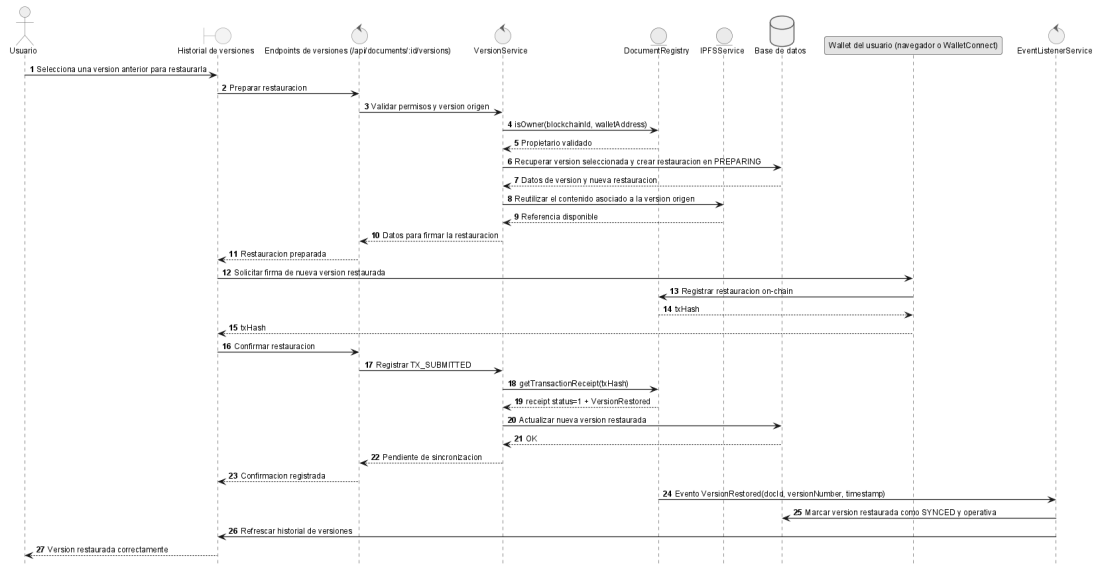


Figura 84: Diagrama de secuencia UC-0025: Restaurar Versión Anterior

6.3.21. Secuencia: Descargar Versión Específica

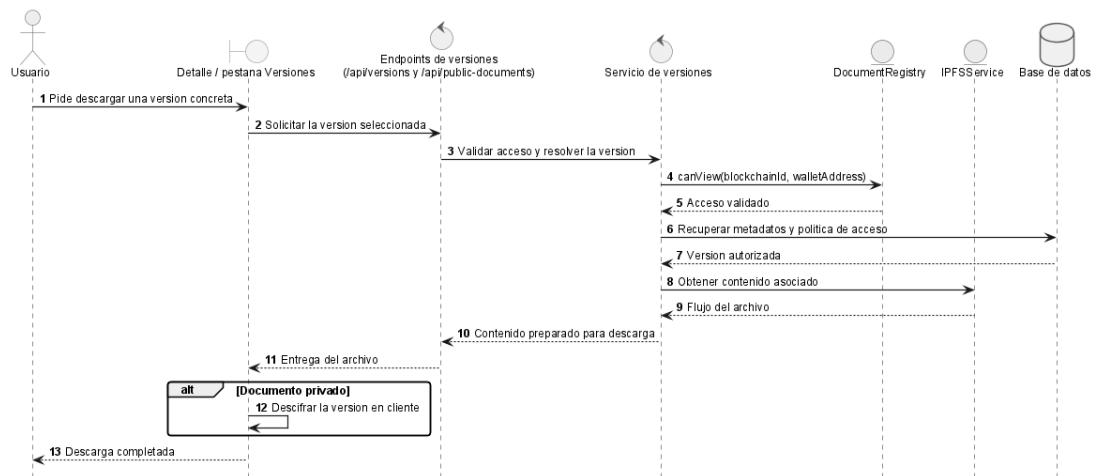


Figura 85: Diagrama de secuencia UC-0026: Descargar Versión Específica

6.3.22. Secuencia: Firmar Documento

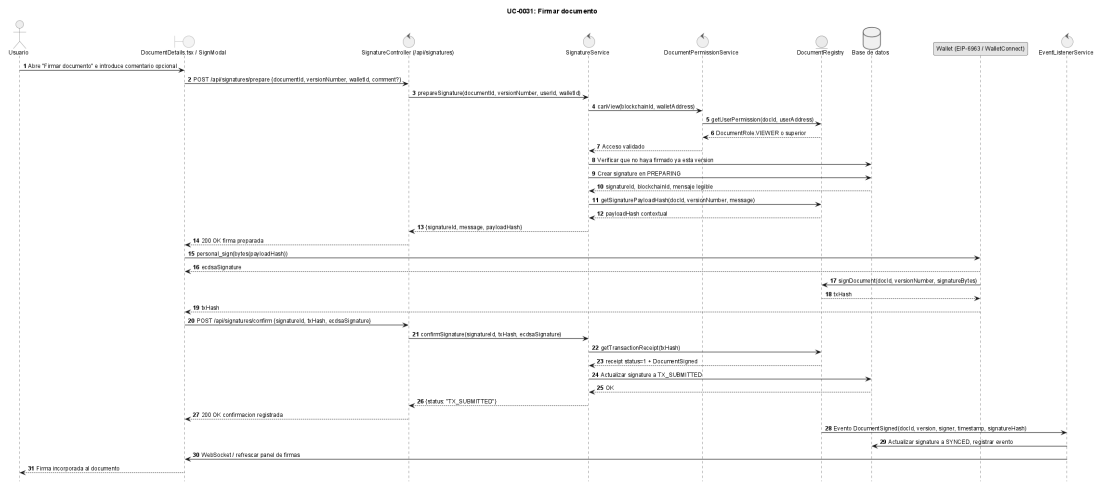


Figura 86: Diagrama de secuencia UC-0027: Firmar Documento

6.3.23. Secuencia: Ver Firmas de un Documento

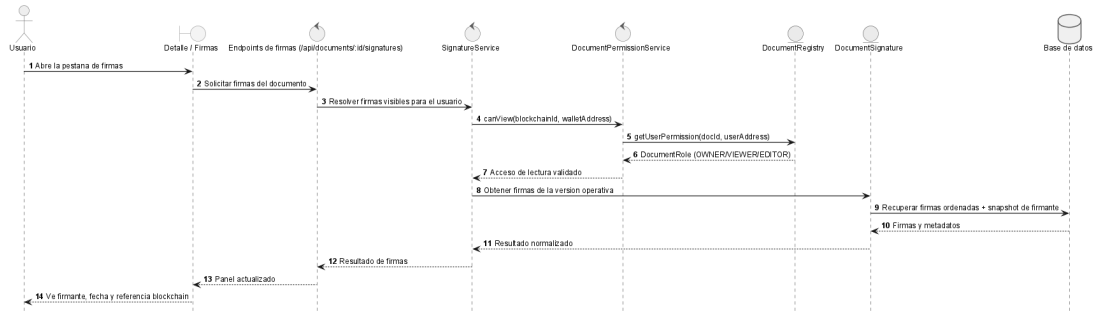


Figura 87: Diagrama de secuencia UC-0028: Ver Firmas de un Documento

6.3.24. Secuencia: Verificar Firma del Usuario

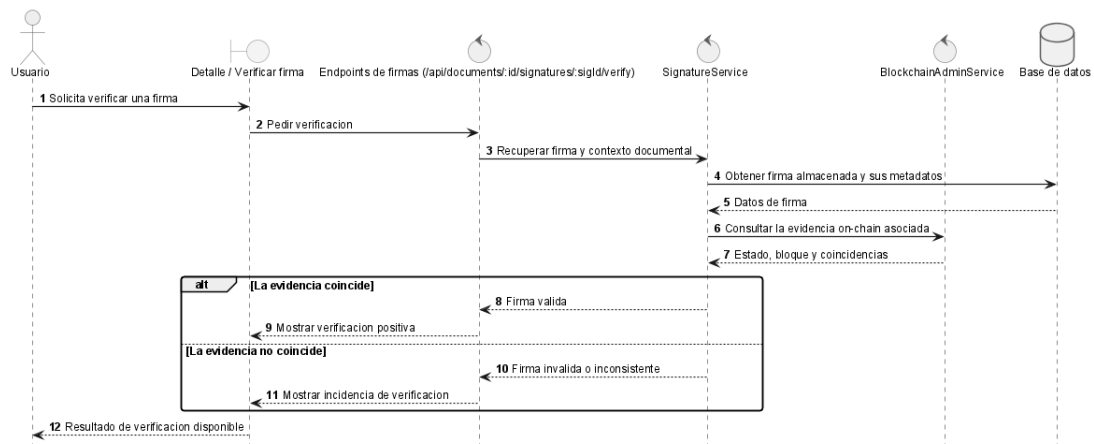


Figura 88: Diagrama de secuencia UC-0029: Verificar Firma del Usuario

6.3.25. Secuencia: Compartir Documento

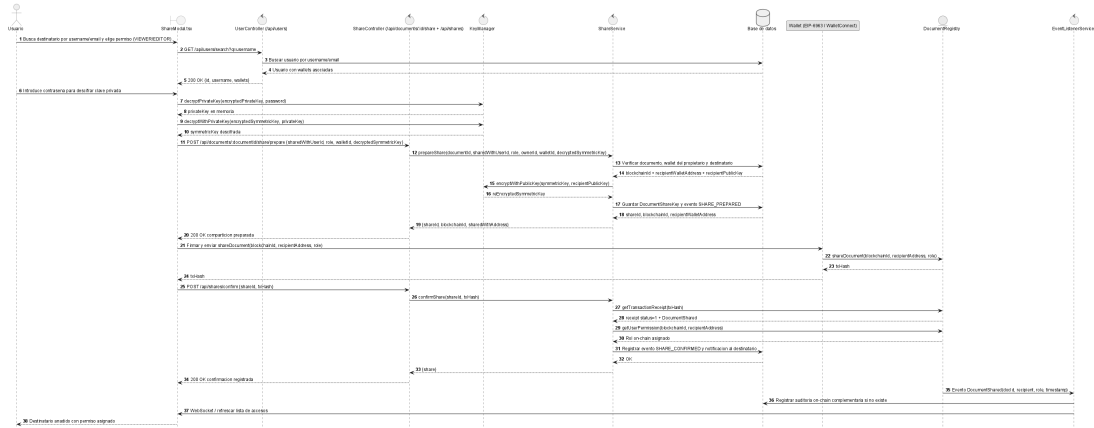


Figura 89: Diagrama de secuencia UC-0030: Compartir Documento

6.3.26. Secuencia: Revocar Acceso a Documento

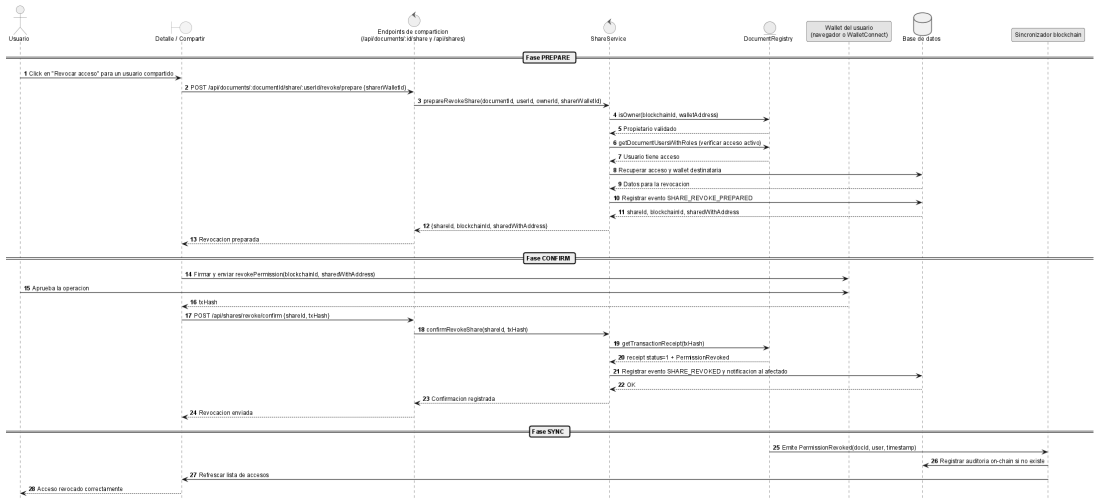


Figura 90: Diagrama de secuencia UC-0031: Revocar Acceso a Documento

6.3.27. Secuencia: Ver Documentos Compartidos Connigo

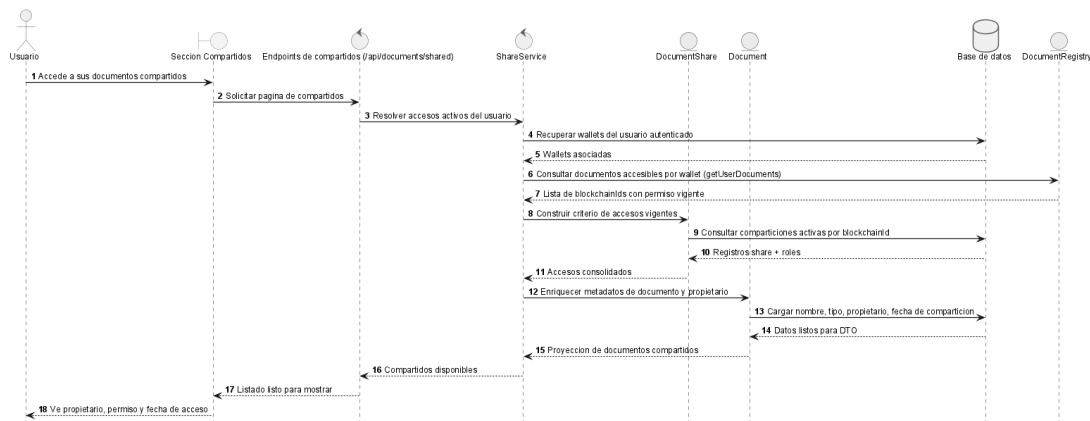


Figura 91: Diagrama de secuencia UC-0032: Ver Documentos Compartidos Connigo

6.3.28. Secuencia: Ver Timeline de Documento

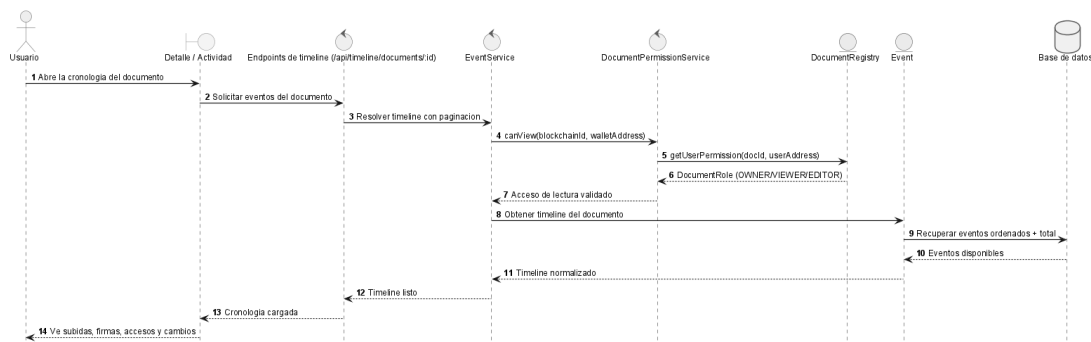


Figura 92: Diagrama de secuencia UC-0033: Ver Timeline de Documento

6.3.29. Secuencia: Ver Estadísticas Personales

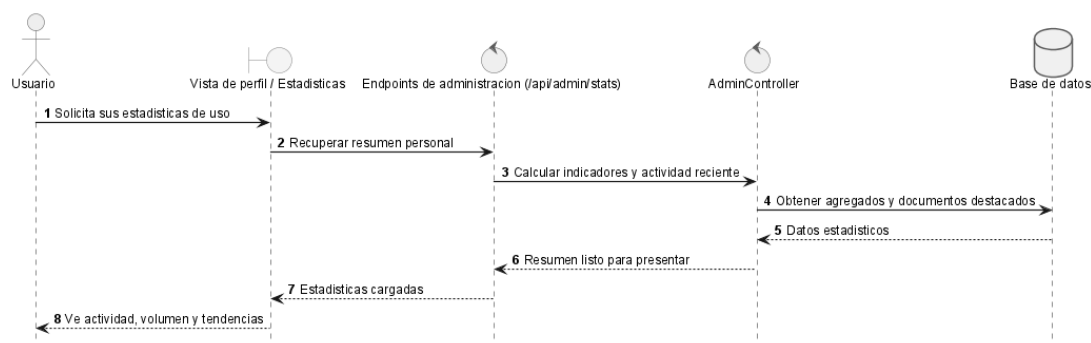


Figura 93: Diagrama de secuencia UC-0034: Ver Estadísticas Personales

6.3.30. Secuencia: Ver Estadísticas Globales del Sistema

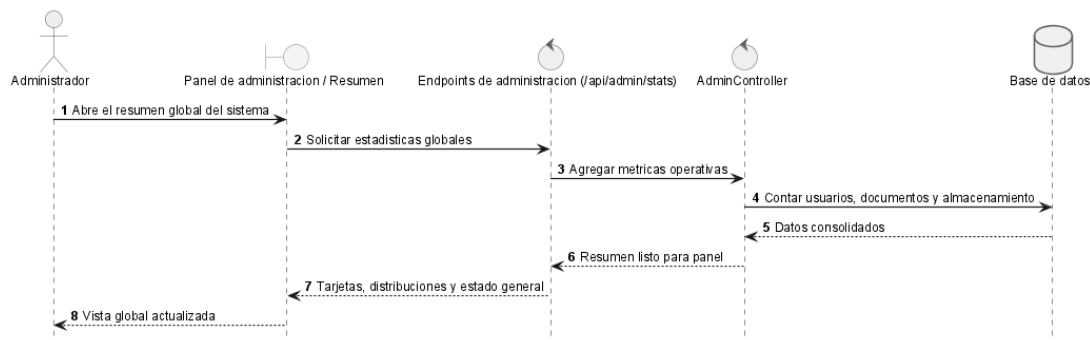


Figura 94: Diagrama de secuencia UC-0035: Ver Estadísticas Globales del Sistema

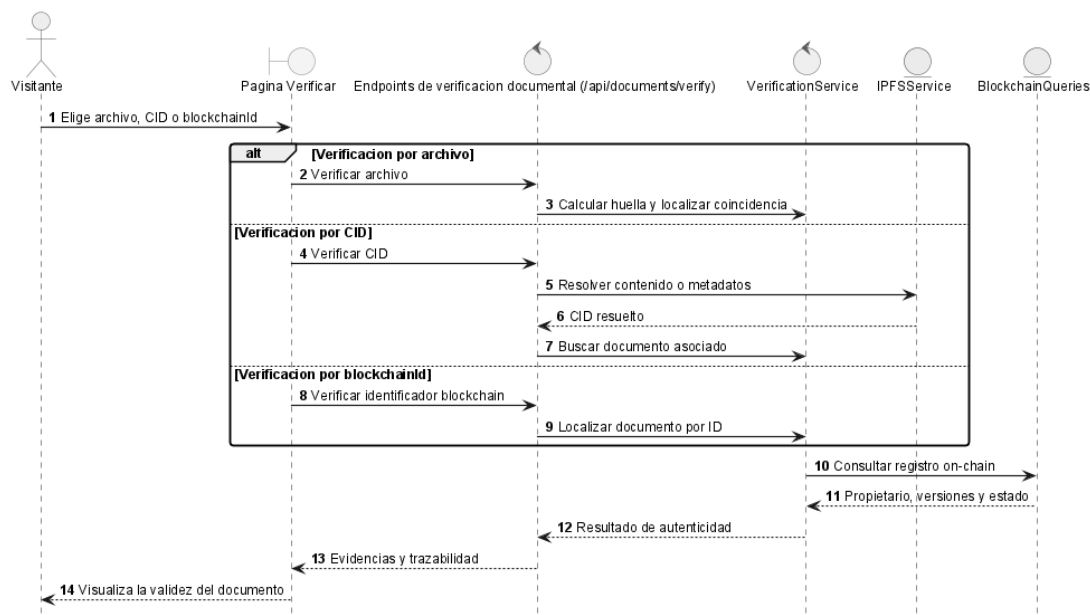


Figura 95: Diagrama de secuencia UC-0036: Verificar Autenticidad de Documento

6.3.31. Secuencia: Auditoría Blockchain

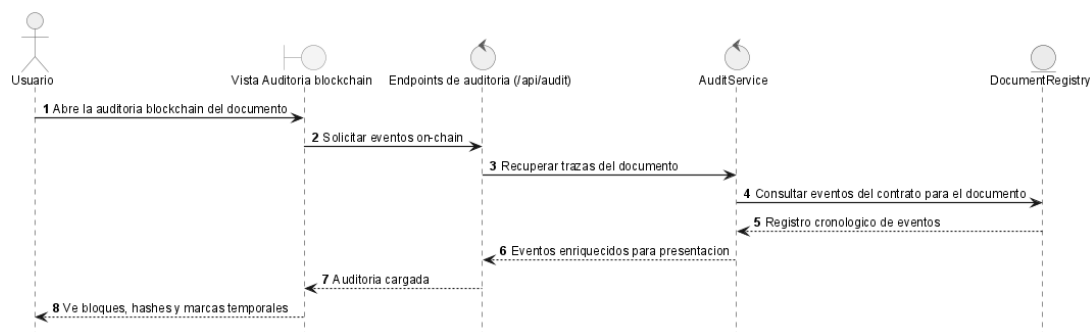


Figura 96: Diagrama de secuencia UC-0037: Auditoría Blockchain

6.3.32. Secuencia: Listar Usuarios (Admin)

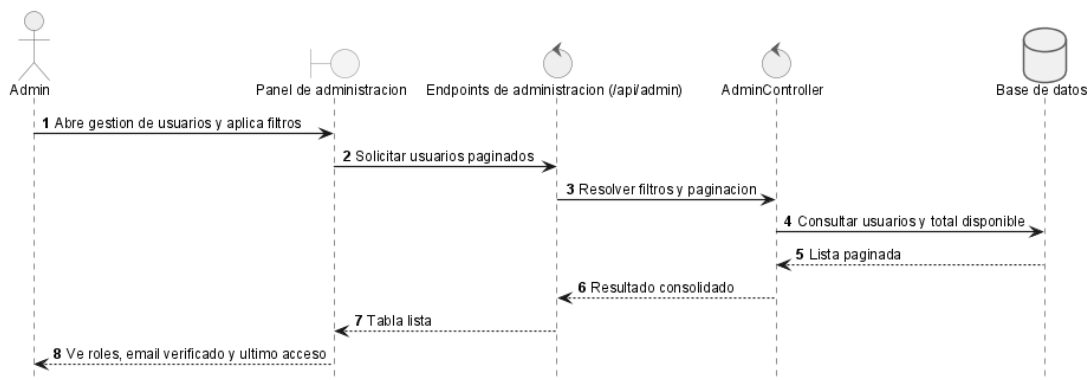


Figura 97: Diagrama de secuencia UC-0038: Listar Usuarios

6.3.33. Secuencia: Gestionar Roles de Usuario

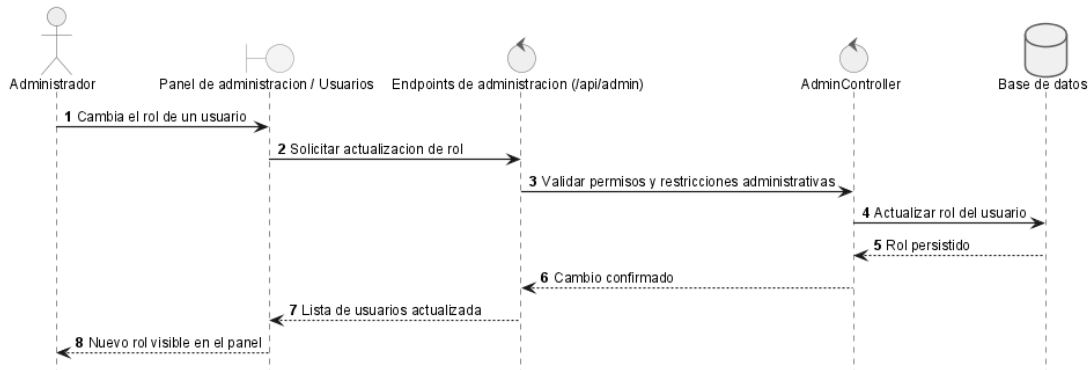


Figura 98: Diagrama de secuencia UC-0039: Gestionar Roles de Usuario

6.4. Subflujos y extensiones de los casos de uso principales

Los diagramas que siguen documentan funcionalidades que, según el criterio del Anexo I, se integran como subflujos o extensiones dentro de los 43 casos de uso principales.

6.5. Diagramas de caso de uso-diseño

La numeración de casos de uso llega hasta UC-0043. Lo que viene a continuación no añade nuevos casos de uso, sino variantes y flujos complementarios que detallan escenarios especialmente relevantes dentro de UC ya definidos. En estos diagramas se ve cómo participan a la vez la interfaz, los servicios, la persistencia y la confirmación en blockchain.

6.5.1. Colaboración: UC-0013 Subir Documento

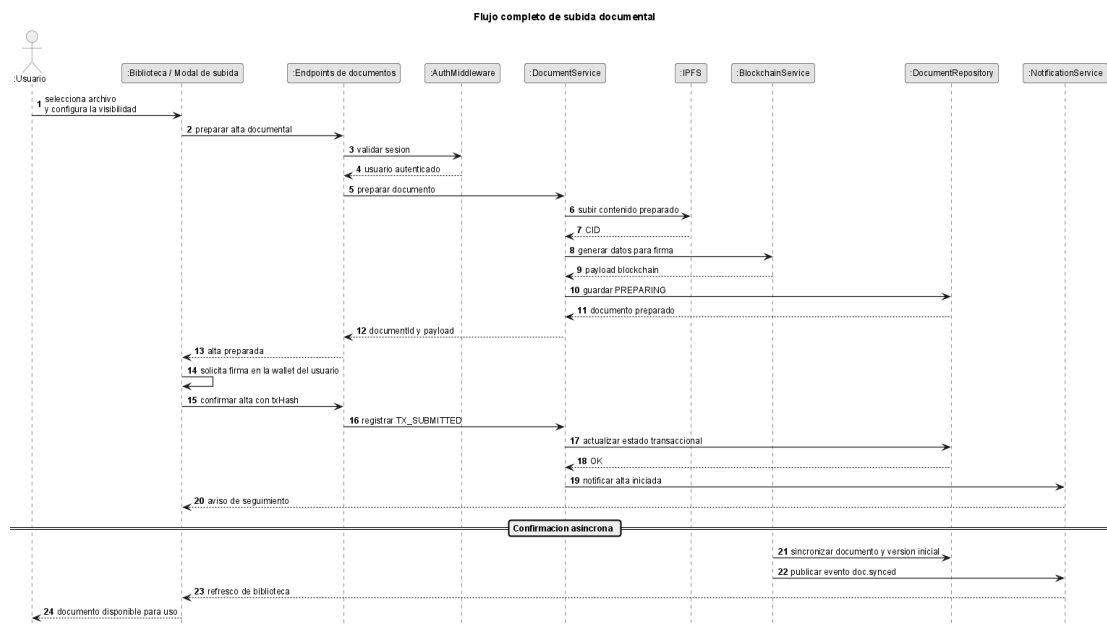


Figura 99: Diagrama de colaboración para UC-0013: Subir Documento

Este diagrama resume la coordinación entre la interfaz de carga, el backend de documentos, IPFS, PostgreSQL y el contrato inteligente durante el alta inicial de un documento.

6.5.2. Colaboración: UC-0027 Firmar Documento

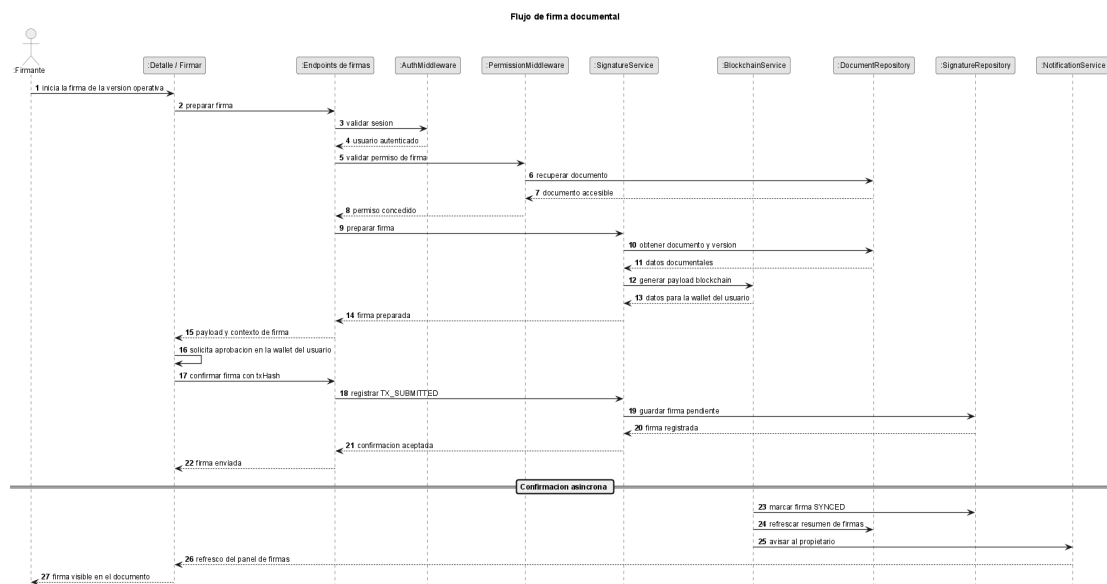


Figura 100: Diagrama de colaboración para UC-0027: Firmar Documento

El mismo patrón de colaboración se reutiliza en operaciones críticas con firma de wallet, como compartir, transferir o crear nuevas versiones, variando únicamente el servicio de dominio y el evento on-chain confirmado.

6.5.3. Flujo complementario: Publicación pública durante la subida

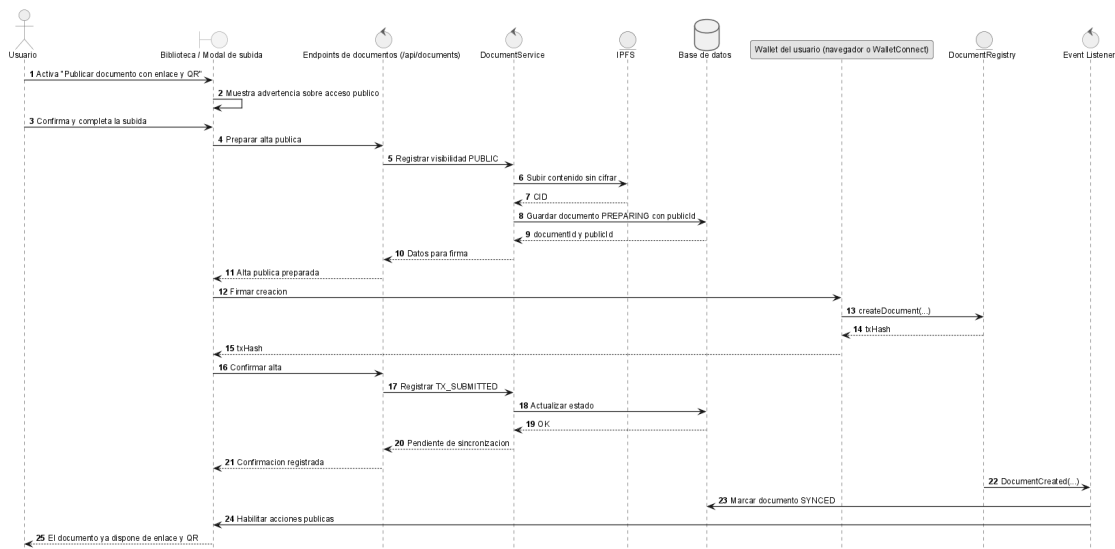


Figura 101: Flujo complementario de publicación pública integrado en UC-0013

Este flujo desarrolla la variante pública del alta documental: el contenido se publica sin cifrado, se genera un publicId y el acceso compartible queda disponible tras la confirmación de la transacción.

6.5.4. Flujo complementario: Compartición mediante enlace y código QR

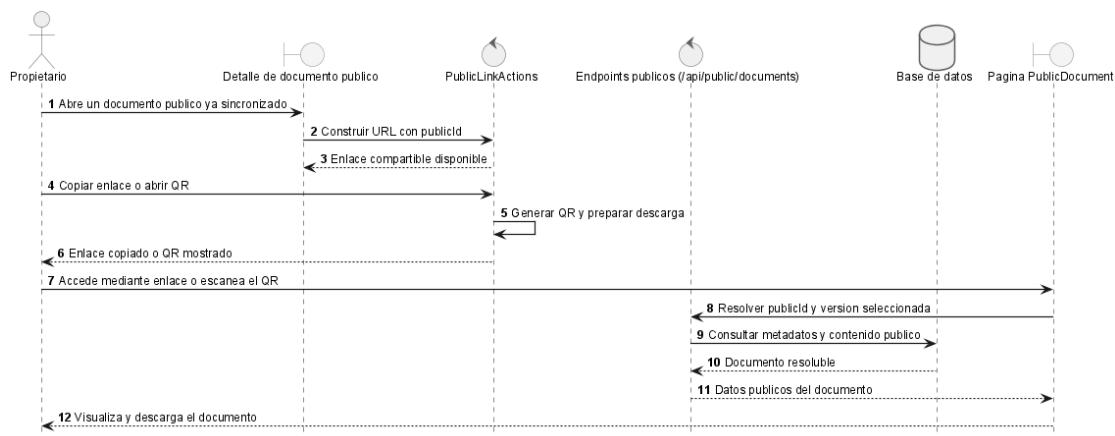


Figura 102: Flujo complementario de distribución por enlace y código QR integrado en UC-0015

Este segundo flujo recoge la distribución pública desde la vista de detalle y desde el selector de versiones operativas, donde se generan el enlace resoluble y el código QR asociados al publicId del documento.

6.6. Diagrama de estados: Ciclo de vida del documento

La Figura 103 muestra el ciclo de vida de un documento mediante un diagrama de estados UML. El documento pasa por DRAFT, PREPARING, TX_PENDING, SYNCED, ARCHIVED,

TRANSFERRED y DELETED según se van ejecutando las operaciones de los casos de uso UC-0013 a UC-0020.

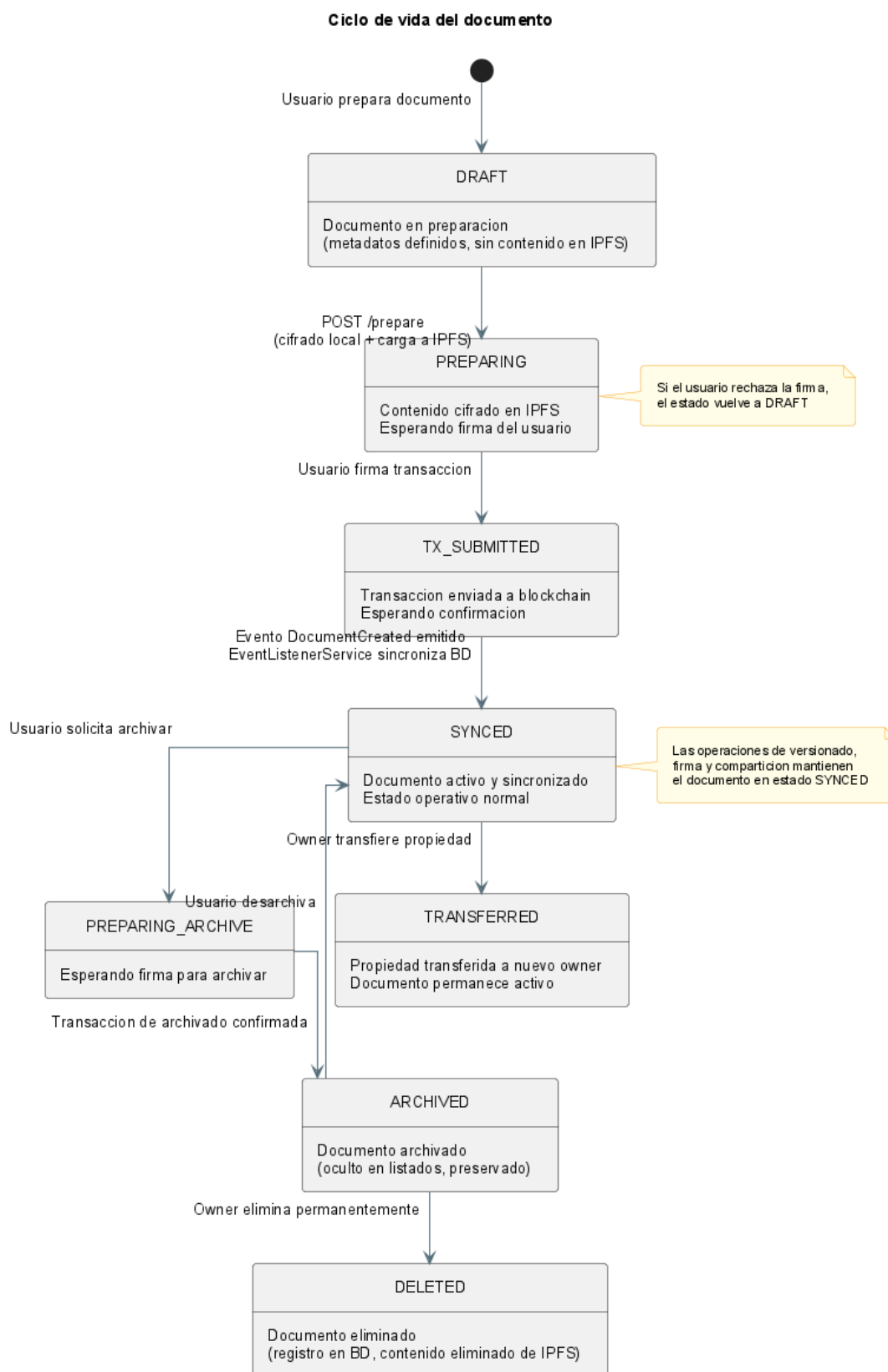


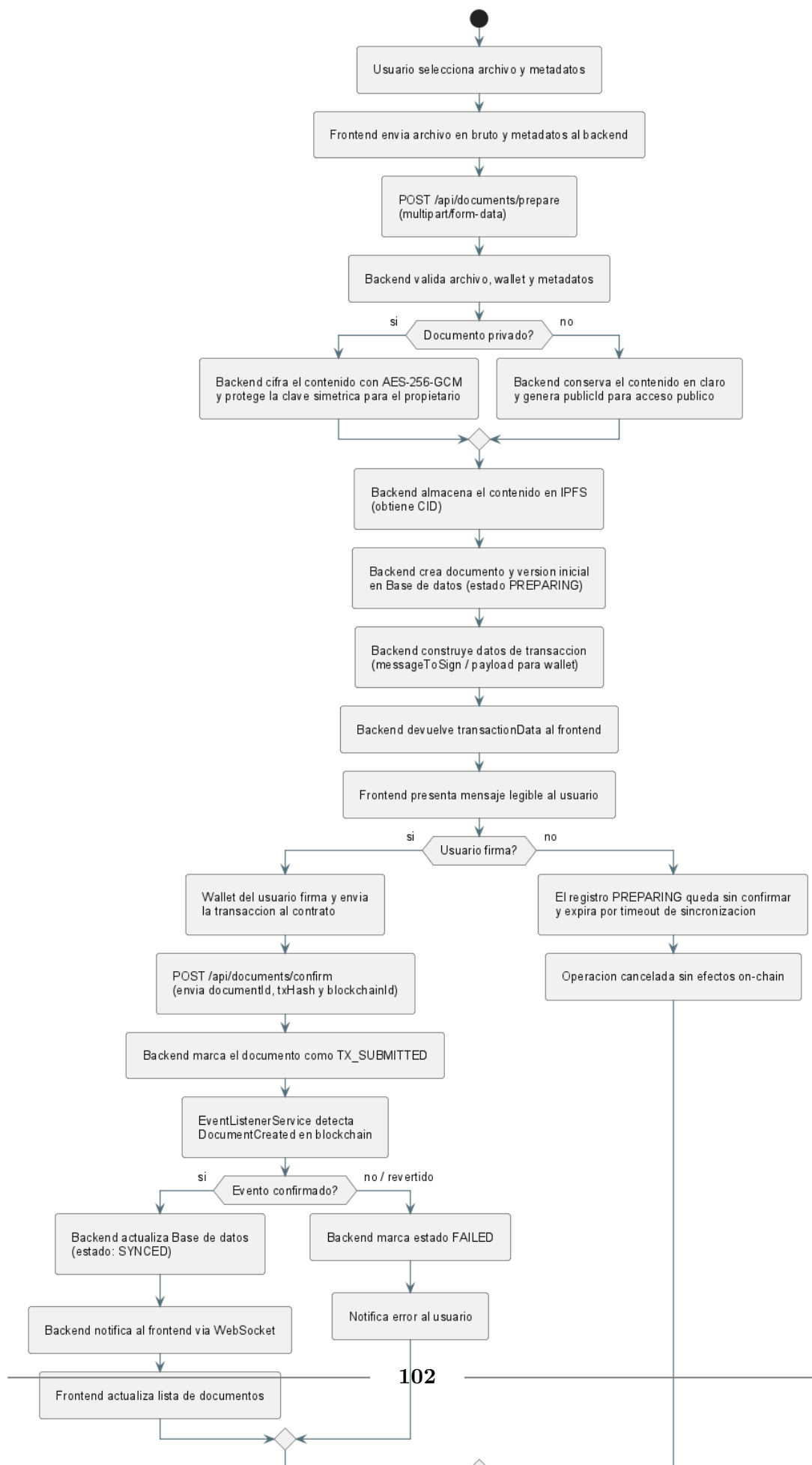
Figura 103: Diagrama de estados: ciclo de vida del documento

El patrón prepare/confirm se aprecia en las transiciones entre DRAFT y SYNCED: si el usuario rechaza la firma estando en PREPARING, el documento vuelve a DRAFT sin coste ni efectos laterales. Las operaciones de versionado (UC-0022), firma (UC-0027) y compartición (UC-0030) se ejecutan sobre documentos en SYNCED sin cambiar su estado, así que el ciclo principal no se ve interrumpido por operaciones secundarias.

6.7. Diagrama de actividad: Subida de documento con firma

La Figura 104 presenta el diagrama de actividad para el flujo de subida de un documento (UC-0013), incluyendo el cifrado en cliente, la carga a IPFS, la preparacion de la transaccion blockchain y la sincronizacion final mediante el EventListenerService.

Diagrama de Actividad: Subida de Documento con Firma



7. Referencia cruzada a los requisitos

Las siguientes matrices verifican que todos los requisitos del Anexo I tienen cobertura en el diseño de este anexo.

7.1. Matriz IRQ – Componente de diseño

Requisito	Entidades clave	Tablas BD	Componentes de diseño
IRQ-0001	Usuario, Wallet, Sesión	users, wallets, sessions	AuthService, WalletService, SessionRepository
IRQ-0002	Documento, Versión	documents, versions	DocumentService, VersionService, IPFS, Smart Contract
IRQ-0003	Carpeta, Etiquetas	folders	FolderService
IRQ-0004	Firma digital	document_signatures	SignatureService, Smart Contract
IRQ-0005	Evento, Auditoría	events	DocumentTimelineService, EventListenerService
IRQ-0006	Estadísticas	user_stats, document_stats, system_stats	StatsService
IRQ-0007	Notificación, Email	notifications, email_verifications, password_resets	NotificationService, EmailService

Cuadro 16: Matriz de trazabilidad IRQ – componentes de diseño

7.2. Matriz UC – Clases de análisis

Caso de uso	Usuario	Wallet	Documento	Versión	Firma
UC-0001 a UC-0012 (Acceso)	X	X			
UC-0013 a UC-0021 (Documentos)	X		X	X	
UC-0022 a UC-0026 (Versionado)	X		X	X	
UC-0027 a UC-0029 (Firmas)	X	X	X	X	X
UC-0030 a UC-0032 (Compartición)	X		X	X	
UC-0033 a UC-0037 (Auditoría)	X		X	X	
UC-0038 a UC-0039 (Admin)	X	X	X		

Cuadro 17: Matriz de trazabilidad UC – clases de análisis

7.3. Cobertura de requisitos no funcionales

Requisito	Elementos de diseño que lo satisfacen
NFR-0001 (Seguridad)	Cifrado AES-256-GCM en IPFS, Argon2id para passwords, HTTPS forzoso, JWT con rotación, Rate Limiting, validación de entrada en todos los endpoints
NFR-0002 (Rendimiento)	Índices en BD (userId, documentId, eventType), backend IPFS configurable con almacenamiento direccionado por contenido, pool de conexiones a base de datos, streaming de archivos grandes
NFR-0003 (Usabilidad)	Interfaz React responsive con Tailwind, feedback inmediato en operaciones async, prototipos validados por flujos de usuario
NFR-0004 (Disponibilidad)	Patrón Prepare/Confirm con reintentos, backend IPFS configurable (Kubo autoalojado o Pinata) y supervisión operativa
NFR-0005 (Mantenibilidad)	TypeScript estricto, ESLint, arquitectura en capas con responsabilidades claras, cobertura de tests
NFR-0006 (Portabilidad)	Docker Compose para todos los servicios, variables de entorno para configuración, backend stateless

Cuadro 18: Cobertura de requisitos no funcionales

8. Plan de desarrollo e implementación

En este apartado se documentan los endpoints de la API REST que materializan el diseño previamente descrito.

8.1. API REST del sistema

La Tabla 19 recoge los endpoints principales de la API REST que implementa el backend, organizados por funcionalidad. Se indica el metodo HTTP, la ruta y una breve descripción de la operación.

Grupo	Endpoint	Descripción
Autenticación y Cuenta		
Auth	POST /auth/register	Registrar nuevo usuario con verificación de email
Auth	POST /auth/login	Iniciar sesión con email y contraseña
Auth	POST /auth/logout	Cerrar sesión activa
Auth	POST /auth/refresh	Renovar token JWT de acceso
Auth	GET /auth/me	Recuperar el perfil autenticado vigente
Auth	POST /auth/change-password	Cambiar contraseña del usuario autenticado
Users	DELETE /users/me	Eliminar la cuenta del usuario autenticado
Auth	POST /auth/wallet-login	Iniciar sesión mediante firma de wallet
Gestión de Documentos		
Documentos	POST /documents/prepare	Preparar subida de documento (cifrado + IPFS)
Documentos	POST /documents/confirm	Confirmar registro de documento en blockchain
Documentos	GET /documents	Listar documentos del usuario con paginación
Documentos	GET /documents/:id	Obtener detalle de un documento
Documentos	GET /documents/:id/download	Descargar contenido del documento desde IPFS
Documentos	PUT /documents/:id	Actualizar metadatos del documento

Grupo	Endpoint	Descripcion
Documentos	POST /documents/:id/archive/prepare	Preparar archivado de documento
Documentos	POST /documents/:id/unarchive/prepare	Preparar desarchivado de documento
Documentos	POST /documents/:id/delete/prepare	Preparar eliminacion de documento
Documentos	POST /documents/:id/transfer/prepare	Preparar transferencia de propiedad
Versionado		
Versiones	POST /documents/:id/versions/prepare	Preparar nueva version (carga a IPFS)
Versiones	POST /documents/:id/versions/confirm	Confirmar version en blockchain
Versiones	GET /documents/:id/versions	Listar versiones de un documento
Versiones	GET /versions/:id/download	Descargar una version especifica
Versiones	POST /documents/:id/versions/restore/prepare	Preparar restauracion de version
Versiones	POST /documents/:id/operational-version/prepare	Preparar cambio de version operativa
Firmas Digitales		
Firmas	POST /signatures/prepare	Preparar firma digital de documento
Firmas	POST /signatures/confirm	Confirmar firma en blockchain
Firmas	GET /documents/:id/signatures	Ver firmas de un documento
Firmas	GET /versions/:id/signatures/check	Verificar firma del usuario en version
Comparticion		
Comparticion	POST /documents/:id/share/prepare	Preparar comparticion con otro usuario
Comparticion	POST /documents/:id/share/confirm	Confirmar comparticion en blockchain
Comparticion	POST /documents/:id/share/:uid/revoke/prepare	Preparar revocacion de acceso
Comparticion	GET /shares/with-me	Ver documentos compartidos con el usuario
Administracion		
Admin	GET /admin/users	Listar todos los usuarios del sistema
Admin	PUT /admin/users/:userId/role	Cambiar rol de un usuario
Admin	POST /admin/users	Crear una cuenta administradora
Admin	DELETE /admin/users/:userId	Eliminar usuario del sistema
Admin	GET /admin/stats	Ver estadisticas globales del sistema
Auditoria y Verificacion		
Auditoria	GET /audit/transaction/:txHash	Auditar transaccion blockchain por hash
Auditoria	POST /verification/file	Verificar autenticidad mediante archivo
Auditoria	POST /verification/blockchain	Verificar autenticidad mediante datos on-chain
Auditoria	GET /timeline/documents/:id	Ver timeline de eventos de un documento
Auditoria	GET /audit/stats	Consultar metricas publicas del sistema
Wallets y Notificaciones		
Wallets	GET /wallets	Listar wallets del usuario
Wallets	POST /wallets	Vincular nueva wallet
Wallets	PUT /wallets/:id/primary	Establecer wallet principal
Wallets	DELETE /wallets/:id	Eliminar wallet
Notificaciones	GET /notifications	Listar notificaciones del usuario
Notificaciones	POST /notifications/mark-all-read	Marcar todas las notificaciones como leidas
Notificaciones	POST /notifications/:id/read	Marcar notificacion como leida
Notificaciones	GET /notifications/preferences	Obtener preferencias de notificacion
Notificaciones	PUT /notifications/preferences	Actualizar preferencias de notificacion

Cuadro 19: Endpoints principales de la API REST de DocumentChain

8.2. Diagrama de despliegue

El diagrama de despliegue muestra la distribución física de los componentes del sistema en los distintos nodos de infraestructura: el servidor backend con sus listeners de eventos, la

red Ethereum, una capa de almacenamiento IPFS compatible representada mediante despliegue autoalojado de referencia, la base de datos PostgreSQL y el proveedor de email.

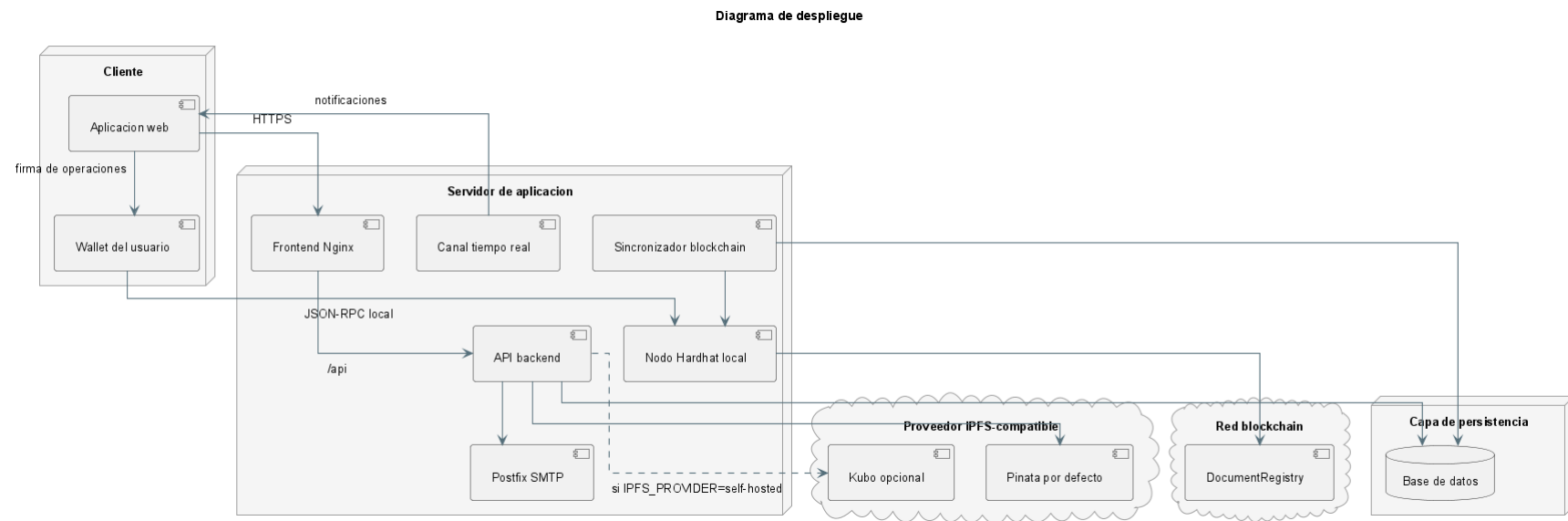


Figura 105: Diagrama de despliegue completo

8.3. Tecnologías de cada componente

Frontend:

- React 18 + TypeScript 5
- Vite para build
- Tailwind CSS 3
- Ethers.js v6 para blockchain
- Axios para HTTP
- React Router v6
- Web Crypto API para descifrado local, hash y gestión de claves en cliente

Backend:

- Node.js 20 LTS
- Express 4
- TypeScript 5
- Capa de acceso a base de datos (Node.js)
- Ethers.js v6
- Argon2 para hashing
- Nodemailer para emails
- WebSocket (ws)

Base de Datos:

- PostgreSQL 15+
- Sistema de migraciones para esquema relacional

Blockchain:

- Solidity 0.8.20
- OpenZeppelin Contracts 5
- Hardhat para desarrollo
- Ethers.js para interacción

IPFS:

- IPFS Kubo (go-ipfs)
- Docker Compose para despliegues autoalojados

8.4. Descripción de nodos de despliegue

El diagrama de despliegue representa cinco nodos principales que interactúan dentro de una misma red Docker Compose:

Nodo Frontend / Nginx. Sirve la aplicación React como SPA estática y actúa como proxy inverso hacia la API. Expone el puerto 80 internamente y se mapea al anfitrión según la configuración del contenedor. Resuelve el enrutado de la SPA devolviendo `index.html` para rutas internas.

Nodo Backend Node.js. Ejecuta la API REST construida con Express. Gestiona la lógica de negocio, la autenticación JWT, la comunicación con PostgreSQL, IPFS y Ethereum, y los listeners de eventos blockchain. Se comunica con PostgreSQL por TCP/5432, con IPFS por la API HTTP interna y con la red Ethereum mediante JSON-RPC.

Nodo PostgreSQL. Almacena el estado operativo del sistema: usuarios, documentos, versiones, firmas, permisos, sesiones, notificaciones y configuración. Utiliza volúmenes Docker para persistencia entre reinicios. Solo el backend tiene acceso directo; ni el frontend ni servicios externos pueden conectarse a la base de datos.

Nodo IPFS (Kubo). Mantiene el contenido documental como objetos IPFS. Expone la API HTTP para *pinning* y recuperación de CIDs. Los archivos cifrados se almacenan como bloques de datos sin estructura de directorio; la aplicación gestiona el mapeo entre `documentId` y CID. El nodo conserva los contenidos mientras estén *pinneados*; al eliminar un documento, el backend desancla el CID.

Nodo Ethereum (Hardhat). En el entorno de desarrollo, este nodo proporciona una red EVM local con bloques instantáneos y cuentas pre-fundadas. El contrato `DocumentRegistry` se despliega durante la inicialización del entorno. El backend se conecta a este nodo mediante WebSocket o HTTP para enviar transacciones y escuchar eventos.

Nodo Postfix. Servidor SMTP de salida que recibe los mensajes generados por el backend y los encamina hacia el destinatario final, ya sea por entrega directa o mediante un relay autenticado configurado.

Para el despliegue técnico completo con Docker Compose, configuración de redes, volúmenes y flujo de actualización, véase el Anexo VII, Sección “Despliegue con Docker Compose”.

Referencias

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
3. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
4. OpenZeppelin. (2025). *Smart Contract Patterns*. <https://docs.openzeppelin.com/contracts/>
5. Ethereum Foundation. (2025). *Solidity Documentation*. <https://docs.soliditylang.org/>
6. Protocol Labs. (2025). *IPFS Documentation*. <https://docs.ipfs.tech/>
7. React Team. (2025). *React Documentation*. <https://react.dev/>
8. PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>